AGENTIC DESIGN

The Agentic Designer

How AI Agents, Claude Design, Open Design, and the New Design Tool Ecosystem Are Transforming Product Design

Approach: Agentic Design Workflows

Version: 0.1.0 · **Date:** May 2026

Mehran Mozaffari

github.com/imehr

This book was written with the assistance of AI tools. All tool references and code examples reflect capabilities as of May 2026. The agentic design tool ecosystem evolves rapidly; check official documentation for the latest features and integrations.

The Agentic Designer

How AI Agents, Claude Design, Open Design, and the New Design Tool Ecosystem Are Transforming Product Design

First Edition: May 2026

Personal Use License

This book is free for personal, non-commercial use. You may read, share, and reference it for individual study, education, and research.

No part of this publication may be sold, rented, or redistributed for commercial purposes without written permission from the author.

For permissions and licensing inquiries:
https://github.com/imehr

Written with AI Assistance

This book was written with the assistance of AI tools. The author directed, reviewed, and approved all content. AI tools were used for drafting, research, and editing. All opinions and analyses are the author's own.

All tool references and code examples reflect capabilities as of May 2026. The agentic design tool ecosystem evolves rapidly; check official documentation for the latest features and integrations.

Trademarks

Product names, logos, and brands mentioned in this book are the property of their respective owners. Use in this book is for identification and educational purposes only and does not imply endorsement.

Claude Code and Claude Design are products of Anthropic. OpenAI, Codex, and ChatGPT are products of OpenAI. Gemini is a product of Google. Figma is a product of Figma, Inc. Miro is a product of Miro. GitHub Copilot is a product of GitHub, Inc. Vercel and v0 are products of Vercel, Inc. Bolt.new and WebContainers are products of StackBlitz. Lovable is a product of Lovable AB. Remotion is a product of Remotion GmbH. shadcn/ui is a product of shadcn. Supabase is a product of Supabase Inc. tldraw is a product of tldraw GmbH. Paper is a product of Paper Design Inc. Pencil is a product of Pencil.dev. Kiro is a product of Amazon Web Services. Hyperframes is a product of HeyGen. OpenCode is a product of OpenCode AI. ElevenLabs is a product of ElevenLabs Inc. Playwright is a project of the Microsoft team.

Disclaimer

This book is provided for informational and educational purposes only. Nothing in this book constitutes legal, financial, tax, or professional advice. The author makes no warranties about the accuracy or completeness of the information.

Tool capabilities, pricing, API availability, and feature sets described in this book may have changed since publication. Always consult the official documentation for each tool or platform for current information.

Code examples are provided for educational purposes and should be tested in your own environment before use in production.

Edition and Publisher Details

Published by the author
San Francisco, CA

Edition: First Edition, May 2026
Version: 1.0
Book pipeline: OpenCode agentic publishing system

Credits

Author: Mehran Mozaffari
Editorial review: Multi-model AI review pipeline
Technical reviewers: Claude Opus 4.6, Gemini 3.1 Pro
Design and production: Agentic publishing pipeline (OpenCode)

Open Source Acknowledgments

This book references and describes the following open-source and source-available projects. The author gratefully acknowledges the contributions of their maintainers and communities:

- Open Design (nexu-io) — MIT License
- OpenPencil (ZSeven-W) — MIT License
- Huashu Design (alchaincyf) — MIT License
- Remotion (remotion-dev) — Remotion License
- tldraw — Apache License 2.0
- ComfyUI — GPL-3.0 License
- CLI-Anything (HKUDS) — MIT License
- OpenCode (anomalyco) — MIT License

This book does not distribute source code from any of these projects. References are descriptive and educational.

Contact the Author

Blog: https://piazr.github.io/applied-ai/
GitHub: https://github.com/imehr

For corrections, errata, or licensing inquiries, please open an issue on the book's repository or contact the author through the channels above.

Contents

The Agentic Designer — 14 chapters + 3 appendices

Appendices

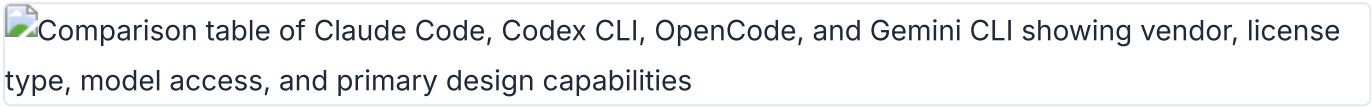
01 The Agentic Design Paradigm

Why AI agents are becoming the new design tool interface

The way software gets designed is changing. AI coding agents --- tools built to write and ship code --- are now operating design tools, generating visual artifacts, and closing the loop from design intent to production output. This chapter explains why that shift happened, what enables it, and what it means for anyone who designs software for a living.

The Shift from GUI to Agent

For a decade, the design workflow looked the same. Open Figma or Sketch. Use a mouse-driven GUI to place elements on a canvas. Export static images or specs. Hand off to engineering. Engineering interprets the design, writes code, and ships something that approximates the original intent.



The four major agent platforms compared by maker, core capability, and key differentiator

That loop worked. It still works. But it has a structural problem: the design artifact and the production artifact are two different things. A Figma file is not a website. A Sketch mockup is not a component library. Every handoff introduces translation loss. The designer describes a hover state. The engineer implements a different one. The designer specifies a 12px gap. The engineer uses 16px because it looked better in code. Small divergences accumulate into a final product that only loosely resembles the original design.

AI coding agents change this equation. An agent can read a design file, generate code from it, write HTML back to a canvas, and iterate on visual output --- all without leaving the terminal. The design artifact and the production artifact converge toward the same thing.

Consider what the four major agent platforms can do as of May 2026:

Platform	Maker	Core capability	Key differentiator
Claude Code	Anthropic	Agentic coding across terminal, IDE, desktop, web	Richest skill system, most mature MCP support
Codex CLI	OpenAI	Cloud-based agent running tasks in isolated sandboxes	Parallel task execution, citation-backed output
OpenCode	Open source	Multi-provider agent with 75+ LLM options	160K GitHub stars, cross-compatible with Claude Code conventions
Gemini CLI	Google	Terminal agent with 1M-token context window	Free tier with 6K code requests/day

These tools were built for software engineering. But their capabilities --- reading files, editing code, running commands, connecting to external tools --- map directly onto design tasks. The distinction between "coding agent" and "design agent" is eroding fast.

Claude Code describes itself as "an agentic coding tool that reads your codebase, edits files, runs commands, and integrates with your development tools" (source: Claude Code Overview, retrieved 2026-05-18). The critical word is *agentic*. It does not suggest. It does. It reads a design file, generates a visual artifact, and writes it to a canvas --- as an action, not a recommendation.

OpenAI frames the same shift differently: "We imagine a future where developers drive the work they want to own and delegate the rest to agents" (source: Introducing Codex, retrieved 2026-05-18). Replace "developers" with "designers" and the sentence holds. The agent handles the translation between intent and artifact.

OpenCode, the open-source entry with 160K GitHub stars and 7.5M monthly developers (source: OpenCode, retrieved 2026-05-18), takes the broadest approach. It supports 75+ LLM providers, making it accessible regardless of which model you prefer. Its cross-compatibility with Claude Code conventions means design skills transfer between platforms without modification. Gemini CLI rounds out the field with a 1M-token context window and a free tier that handles 6,000 code requests per day --- generous enough for design iteration.

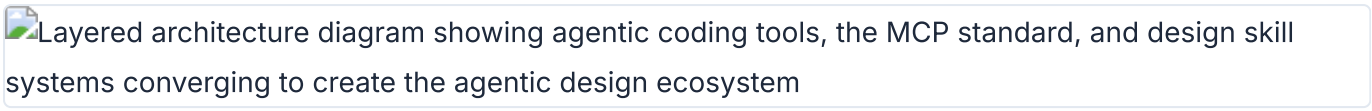
The shift from GUI to agent is not about replacing visual tools. It is about adding a new interface layer. The canvas still matters. The visual preview still matters. What changes is how you operate the canvas. Instead of clicking and dragging, you describe what you want in natural language. The agent translates that description into visual output on the canvas. You review. You iterate. The loop tightens.

This does not mean the mouse-driven GUI disappears. Figma will continue to exist. Sketch will continue to exist. But for a growing class of design tasks --- generating prototypes, creating landing pages, building component libraries, producing motion graphics --- the agent-first workflow is faster, more consistent, and more tightly integrated with production code. The GUI becomes the review surface, not the authoring surface.

The economics favor the shift. A design engineer with a well-configured agent can produce in one session what used to take a designer and a front-end developer several days of back-and-forth. The agent handles the translation between design intent and code. The human provides direction and quality control. The bottleneck moves from production to decision-making --- which is where it should be.

Why Now: The Convergence of Agents, Skills, and MCP

Three independent developments converged in 2025-2026 to make agentic design possible. None of them is sufficient on its own. Together, they create something new.



Three independent developments converging to enable agentic design in 2025-2026

Agentic coding tools reached production quality. Claude Code matured through 2025, adding skill systems, agent teams, and MCP support. Codex CLI launched with isolated cloud sandboxes and parallel task execution. OpenCode crossed 160K GitHub stars with 900 contributors and 7.5M monthly developers. Gemini CLI entered the field with a generous free tier and 1M-token context window. For the first time, designers had multiple production-grade agents to choose from.

The key milestone was the transition from suggestion-based tools to action-based tools. GitHub Copilot suggests code completions. Cursor suggests edits. These are useful. But they require the designer to drive every action. An agentic tool takes a goal and executes the steps to achieve it. Claude Code reads a codebase, plans an approach, edits multiple files, runs tests, and verifies the result --- all autonomously. That shift from suggestion to action is what enables design workflows.

MCP (Model Context Protocol) became the universal connector. Claude Code documentation describes MCP as "an open standard for connecting AI tools to external data sources" (source: Claude Code Overview, retrieved 2026-05-18). MCP lets an agent connect to Figma, Paper, Pencil, Miro --- any tool that exposes an MCP server. The agent reads designs, exports code, and writes artifacts back to the canvas. All through a standard protocol.

Before MCP, every agent needed custom integrations for every tool. Claude Code would need a Figma integration, a Paper integration, a Pencil integration. Each built differently. Each maintained separately. MCP turns that N×M problem into an N+M problem. Any agent can connect to any tool that exposes an MCP server. This standardization is what makes multi-tool design workflows practical.

Design-as-code formats emerged. Pencil's .pen files represent vector designs as JSON --- diffable in git, readable by agents, editable in an IDE. OpenPencil's .op files follow the same principle. Design tokens expressed as CSS custom properties or JSON variables become code artifacts that agents can

read, transform, and write. The design file is no longer a binary blob. It is a text file the agent can understand, modify, and version.

```
# MCP configuration for a design tool (OpenCode example)
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "my-design-tool": {
      "type": "local",
      "command": ["npx", "-y", "@my-design-mcp/server"],
      "enabled": true,
      "environment": {
        "DESIGN_API_KEY": "your-key"
      }
    }
  }
}
```

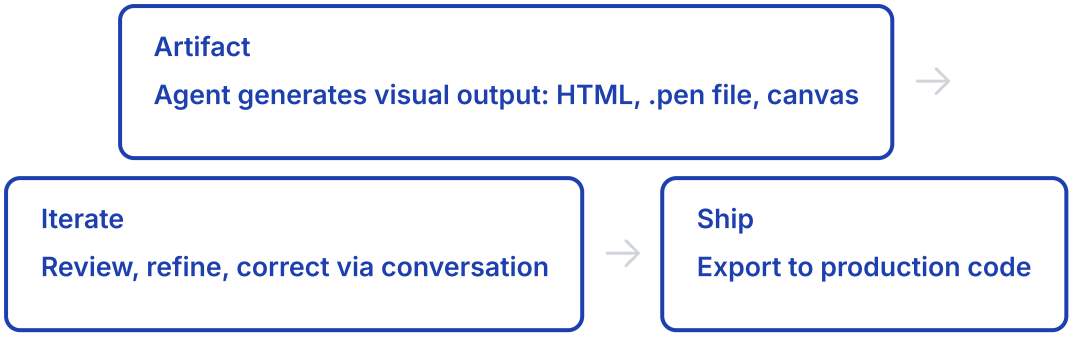
This snippet shows how an MCP server gets configured. The agent connects to the design tool at session start. Every tool operation --- reading designs, writing artifacts, exporting code --- flows through this standard interface.

My take: MCP is the most under-hyped development in this space. Skills and agents get the attention, but MCP is what makes the whole thing work. Without a standard protocol for connecting to design tools, every agent would need custom integrations for every tool. MCP turns that N×M problem into an N+M problem. Any agent can connect to any tool that exposes an MCP server. This is why I dedicate an entire chapter to MCP integrations later (see section 10.1).

There is a fourth factor: skill system standardization. Claude Code uses SKILL.md files in `.claude/skills/`. OpenCode uses SKILL.md in `.opencode/skills/`, `.claude/skills/`, or `.agents/skills/`. Both follow the Agent Skills open standard at agentskills.io. This means a design skill written for Claude Code works in OpenCode with zero modification. Skills are portable across agent platforms. I cover this in depth in section 03.3.

The Agentic Design Loop: Prompt → Artifact → Iterate → Ship

Every agent platform implements the same fundamental loop. The details differ, but the pattern is consistent across Claude Code, Codex CLI, OpenCode, and Gemini CLI.



In Claude Code, the loop works like this: describe what you want. The agent reads the codebase, picks a design system from the skill configuration, and generates a visual artifact. You review it --- via screenshot, browser preview, or canvas view. You iterate with natural language corrections. The agent updates the artifact. When the output matches the intent, you export to production code.

Codex runs the same loop in a cloud sandbox. Each task runs for 1-30 minutes, depending on complexity (source: Introducing Codex, retrieved 2026-05-18). The agent provides "verifiable evidence of its actions through citations of terminal logs and test outputs." You review the citations, approve or revise, and merge.

OpenCode splits the loop into two modes. Press Tab to enter Plan mode --- the agent analyzes the request and proposes an approach without making changes. Switch back to Build mode and the agent implements the plan. Use `/undo` to revert bad iterations (source: OpenCode Docs, retrieved 2026-05-18). This two-phase approach reduces wasted work on design tasks where the direction is uncertain.

The design-specific variant of this loop adds a critical layer: the agent reads design system tokens and brand guidelines before generating output. This produces more consistent results and reduces the iteration cycle dramatically. A bare agent might need 5-8 iterations to converge on brand-consistent output. A configured agent --- one that has read your design tokens, color palette, and typography rules --- often gets it right in 1-2 iterations. More on this in section 03.5.

Agent	Prompt method	Iteration	Review
Claude Code	Interactive chat or pipe (`claude -p`)	Conversational refinement	Screenshot, browser, MCP canvas
Codex CLI	Single prompt, sandbox execution	Revise prompt, re-run	Citations: terminal logs, test output
OpenCode	Plan mode (Tab) then Build mode	`/undo` , `/redo` , conversational	Screenshot, share link (`/share`)
Gemini CLI	Terminal prompt	Conversational refinement	Human-in-the-Loop (HiTL) oversight

Notice the pattern. Every agent starts with a prompt. Every agent produces an artifact. Every agent supports iteration. The differences are in how each step works, not in whether the step exists. This consistency is what makes it possible to learn one agent and transfer that knowledge to another.

My take: The Prompt → Artifact → Iterate → Ship loop sounds simple. In practice, the iteration phase is where most of the time goes. An agent that generates a perfect artifact on the first try does not exist. The quality of the output depends on how well you configure the agent's context --- the design system, the brand guidelines, the skill files. Chapters 03 and 08 cover this in detail. Skip those and you will spend your iteration budget on problems the agent could have avoided.

Who This Book Is For and What You Will Learn

This book is for product designers, design engineers, and creative technologists who want to use AI agents as design tools. Not as novelties. Not as experiments. As production tools that ship real work.

The prerequisites are modest. You should be comfortable in a terminal --- navigating directories, running commands, piping output. You should know HTML and CSS fundamentals. You should have used at least one design tool, whether Figma, Sketch, or something else. You do not need to be a software engineer. You do not need to know React deeply, though basic familiarity with JSX concepts helps in later chapters.

What you do not need: a computer science degree, experience with machine learning, or familiarity with prompt engineering as a discipline. The agents handle the technical complexity. Your job is to provide design direction and quality standards.

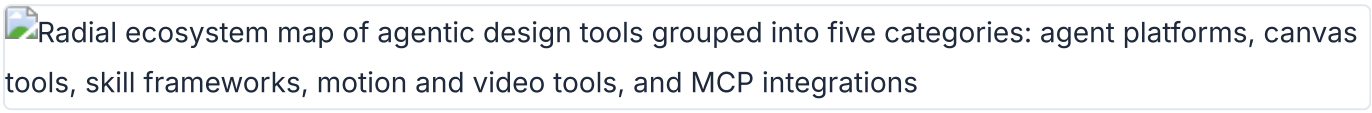
By the end of this book, you will know how to:

- Install and configure the four major AI coding agent platforms.
- Write design instructions in CLAUDE.md and AGENTS.md.
- Install, compose, and create design skills using the SKILL.md convention.
- Connect design tools (Figma, Paper, Pencil, Miro) via MCP servers.
- Use connected canvas tools for agent-driven design workflows.
- Build custom design harnesses that enforce brand consistency.
- Generate production-ready UI code from design artifacts.
- Orchestrate multiple agents for parallel design tasks.
- Create programmatic video and motion graphics with agents.
- Maintain design systems across code and design surfaces.

The book assumes you have access to at least one agent platform. If cost is a concern, OpenCode with a free API key or Gemini CLI's free tier will work for most exercises. The skills and techniques transfer across platforms.

A Map of the Agentic Design Ecosystem

The agentic design ecosystem has five layers. Each layer builds on the one below it. Understanding the layers is more important than memorizing any individual tool.



The agentic design ecosystem organized by tool category and function

Layer	Components	Examples	Covered in
Agent platforms	CLI and IDE agents that execute tasks	Claude Code, Codex CLI, OpenCode, Gemini CLI	Chapter 02
Configuration and skills	Instructions, skills, harnesses	CLAUDE.md, AGENTS.md, SKILL.md, design harnesses	Chapter 03
Design canvas tools	Visual surfaces agents can read and write	Paper, Pencil, OpenPencil	Chapters 05-06
Design skill frameworks	Composable skills for specific design outputs	Open Design, Huashu Design	Chapters 06-07
MCP integrations	Protocol connectors to external design tools	Figma MCP, Miro MCP, MagicPattern MCP	Chapter 10

This book proceeds through these layers from bottom to top. Chapters 02-03 cover agent platforms and configuration. Chapters 04-07 cover design-as-code concepts, canvas tools, and skill frameworks. Chapters 08-10 cover design systems, motion, and MCP integrations. Chapters 11-12 cover multi-agent patterns and the production pipeline. Chapters 13-14 close with case studies and predictions.

A sixth layer exists below these five: the design-as-code formats themselves. .pen files, .op files, HTML as a design surface, design tokens as code variables. These formats are what make the entire stack possible. Without formats that agents can read, write, and diff, none of the layers above would function. I cover this foundational layer in Chapter 04.

The ecosystem also includes motion and video tools. Remotion (47.1k+ GitHub stars) makes videos programmatically with React. Hyperframes (19k+ stars, by HeyGen) uses HTML + GSAP as the authoring format, built specifically for AI agents. These tools extend the agentic design loop beyond static artifacts into time-based media. Chapter 09 covers both in depth.

Tool	Type	Scale (May 2026)	Key format
Open Design	Design skill framework	43K+ GitHub stars, 31 skills, 129 design systems	DESIGN.md, SKILL.md
Huashu Design	HTML-native design skill	14K+ stars, 20 design philosophies	HTML, SKILL.md
Pencil	Vector design tool (IDE)	Commercial, .pen format	.pen (JSON)
OpenPencil	Open-source vector design	3K+ stars, concurrent Agent Teams	.op (JSON)
Remotion	Programmatic video (React)	47.1k+ stars	React/TSX compositions
Hyperframes	Programmatic video (HTML+GSAP)	19k+ stars, by HeyGen	HTML, GSAP

My take: Most people entering this space focus on the agent platform layer. That is the wrong place to start. The agent is a commodity --- Claude Code and OpenCode and Gemini CLI all do roughly the same things. The differentiator is the configuration and skills layer. A well-configured agent with a good skill file will outperform a bare agent every time. Invest your time in chapters 03 and 08 before you invest in learning every agent platform's quirks.

The ecosystem is young and evolving fast. New tools appear monthly. Existing tools add features weekly. The specific tools and numbers in this chapter will age. The mental model --- agents, configuration, canvas, skills, MCP --- will hold. Understanding the layers is more durable than memorizing any tool's feature list.

One more thing worth noting: this ecosystem rewards generalists. The tools are too new for deep specialization. A designer who can configure an agent, write a skill file, and connect an MCP server will outperform a designer who only knows one of those things. This book aims to make you that generalist.

Tip

Next: Chapter 02 compares the four major agent platforms in detail. You will learn how to install each one, what makes it different, and which one to pick for design workflows.

02 Your Agent Toolkit

Claude Code, Codex CLI, OpenCode, and Gemini CLI compared

Four agent platforms dominate the landscape as of May 2026. This chapter covers each one in detail --- installation, authentication, model access, MCP support, skill systems, and CLI ergonomics --- then closes with a decision framework for choosing the right agent for design work.

Agent Platform Landscape

The market settled on four serious contenders by early 2026. Each takes a different approach to the same problem: how to let AI act on your codebase autonomously, accurately, and repeatably.

Feature	Claude Code	Codex CLI	OpenCode	Gemini CLI
License	Proprietary	Open source	Open source	Open source
Models	Claude only	o3, o4-mini	75+ providers	Gemini only
Config file	CLAUDE.md	AGENTS.md	AGENTS.md (CLAUDE.md fallback)	None documented
Skill system	SKILL.md	No	SKILL.md	No
MCP support	Full	Limited (CLI)	Full (local + remote + OAuth)	Yes
Subagents	Yes	No	Yes (General, Explore, Scout)	No
Plan mode	Yes	No	Yes (Tab key)	No
Free tier	No (requires subscription)	API credits	Yes (BYO key)	Yes (1K req/day, as of May 2026)

All data in this table is current as of May 2026. *Pricing and plan details are current as of May 2026 and may change. Check each platform's website for the latest information.* The ecosystem moves fast --- verify before making purchasing decisions. Appendix A contains an extended comparison matrix with pricing details.

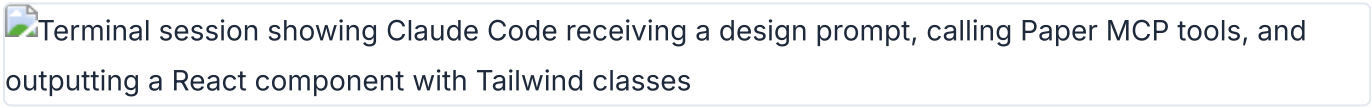
Three of the four agents are open source. Only Claude Code is proprietary. This has practical implications: open-source agents can be self-hosted, audited, and modified. Claude Code runs on Anthropic's infrastructure and requires their subscription. For individual designers, this distinction matters less than it does for enterprise teams with compliance requirements.

The model access question is central. Claude Code locks you into Claude models. Gemini CLI locks you into Gemini models. Codex CLI locks you into OpenAI models. OpenCode is the only agent that lets you switch between all of them --- Claude, Gemini, GPT, Llama, Mistral, and dozens more. If you have strong opinions about which model produces the best design output, OpenCode accommodates that preference. If you want to benchmark models against each other on your specific design tasks, OpenCode makes that easy.

That said, model quality for design tasks is converging. As of May 2026, Claude Sonnet and GPT-4-class models produce comparable design output when given equivalent configuration. The differentiator is the configuration layer, not the model. A well-configured agent using a cheaper model will outperform a poorly configured agent using the most expensive model. Chapter 03 covers configuration in detail.

Claude Code (Anthropic)

Claude Code is Anthropic's agentic coding tool. It reads codebases, edits files, runs commands, and connects to external tools via MCP. Available in terminal, VS Code, JetBrains, desktop app, web, and browser (source: Claude Code Overview, retrieved 2026-05-18).



Claude Code session reading a Paper design and generating JSX via MCP

It is the most polished option for design work as of May 2026. The skill system, MCP integration, and CLAUDE.md conventions all fit the design workflow. If you use only one agent from this chapter, make it this one.

What makes Claude Code stand out for design tasks is the depth of its configuration system. Other agents support configuration files, but Claude Code supports a full hierarchy: organization-level policies managed by admins, user-level preferences that follow you across projects, project-level instructions shared with the team via git, and path-scoped rules that activate only when relevant files are being edited. This layered approach maps well to design work, where brand guidelines are organization-wide but component-specific rules vary by project.

Another advantage: Claude Code's auto memory system accumulates corrections over time. Tell it once that your brand uses 4px spacing units, and it remembers. Correct it twice on border-radius preferences, and it writes those corrections to memory. The agent gets better at your specific design system with every session. No other agent has this feature as of May 2026.

Installation

```
# Native install (macOS, Linux)
$ curl -fsSL https://claude.ai/install.sh | bash

# Homebrew (macOS)
$ brew install --cask claude-code

# WinGet (Windows)
$ winget install Anthropic.ClaudeCode
```

The native install auto-updates in the background. No manual version management. The first launch walks through authentication.

Authentication

Claude Code requires a Claude subscription (Pro, Max, Team, or Enterprise) or an Anthropic Console account with API access. Third-party providers are supported via API keys.

```
# Authenticate with Claude subscription
$ claude
# Opens browser to claude.ai for authorization

# Or set API key directly
$ export ANTHROPIC_API_KEY=sk-ant-...
$ claude
```

The VS Code extension is available on the Marketplace as `anthropic.claude-code`. JetBrains has a plugin. Desktop apps exist for macOS and Windows. Web access lives at `claude.ai/code`.

Key Features for Design Work

CLAUDE.md for persistent instructions. Project-level, user-level, and organization-level files loaded at session start. Supports `@path/to/file` imports and path-scoped rules in `.claude/rules/`. The `/init` command auto-generates CLAUDE.md by analyzing the codebase (source: Claude Code Memory, retrieved 2026-05-18). This is where you put brand guidelines and design system references.

```
# Example: brand rules in CLAUDE.md
# Design System Rules
- All prototypes must use the brand color palette
- Components must be responsive (mobile-first)
- Use design tokens from src/tokens.json
```

Skills via SKILL.md. Design skills live in `.claude/skills/`. Each skill is a directory containing a SKILL.md file with frontmatter and instructions. Skills load on demand --- no context cost until invoked. The frontmatter supports model overrides, tool permissions, subagent execution, and more. See

section 03.3 for the full SKILL.md schema.

MCP support. Claude Code connects to external tools via MCP servers. Configuration lives in project or user settings. Connect Paper, Pencil, Figma, Miro --- any tool with an MCP server. The agent reads designs, exports code, and writes artifacts back to the canvas.

```
# MCP server configuration for Claude Code
{
  "mcpServers": {
    "pencil": {
      "command": "npx",
      "args": ["-y", "@pencil/mcp-server"]
    },
    "figma": {
      "command": "npx",
      "args": ["-y", "@figma/mcp-server"],
      "env": {
        "FIGMA_ACCESS_TOKEN": "your-token"
      }
    }
  }
}
```

Agent teams. "Spawn multiple Claude Code agents that work on different parts of a task simultaneously" (source: Claude Code Overview, retrieved 2026-05-18). This enables parallel design exploration --- one agent generates a layout while another handles responsive variants.

CLI compositability. Pipe input directly into Claude Code for quick tasks:

```
# Analyze a design token file
$ cat tokens.json | claude -p "suggest a dark mode variant"

# Review changed files
$ git diff main --name-only | claude -p "review these changed files"

# Generate a component from a design spec
$ cat design-spec.md | claude -p "implement this as a React component"
```

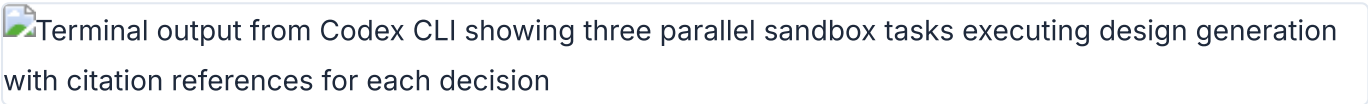
Model Access

Claude Code uses Claude models exclusively: Sonnet, Haiku, and Opus. Switch models with `/model` during a session. For design work, Sonnet provides the best balance of quality and speed. Opus is overkill for most design tasks but useful for complex design system architecture decisions.

```
# Switch model mid-session
/model claude-opus-4-20250514

# Or specify at launch
$ claude --model claude-haiku-3-20250414
```

Codex CLI (OpenAI)

Codex CLI is OpenAI's lightweight terminal agent. It differs from the cloud-based Codex agent (available in the ChatGPT sidebar) which runs tasks in isolated sandboxes. The CLI version runs locally on your machine.



Codex CLI running design tasks in isolated sandboxes with parallel execution

The two Codex variants serve different design workflows. The CLI is best for quick, local tasks --- generating a component, fixing a style issue, running a quick design experiment. The cloud Codex is best for complex, long-running tasks that benefit from parallel execution --- generating multiple design variants simultaneously, refactoring a large component library, producing documentation for an entire design system.

Installation

```
$ npm install -g @openai/codex
```

That is the entire installation. No native binary. No desktop app. Just an npm package.

Authentication

Sign in with a ChatGPT account. This simplifies the setup --- no API key management. Plus users get $5 in free API credits per month (as of May 2026). Pro users get $50 (as of May 2026). Direct API keys also work (source: Introducing Codex, retrieved 2026-05-18).

```
# First run prompts ChatGPT sign-in
$ codex
# Opens browser for authorization
# Auto-configures API key

# Or set API key directly
$ export OPENAI_API_KEY=sk-...
$ codex "fix the auth bug"
```

Key Features for Design Work

AGENTS.md support. Codex reads AGENTS.md files throughout the codebase. From the Codex system message: "These files are a way for humans to give you (the agent) instructions or tips for working within the container" (source: Introducing Codex, retrieved 2026-05-18). Scope follows directory hierarchy. More deeply nested files take precedence in case of conflicts.

```
# AGENTS.md for a design project
# Design Project

## Build Commands
- Preview: `npm run dev`
- Lint: `npm run lint`
- Build: `npm run build`

## Design Conventions
- Use CSS custom properties for all colors
- Mobile-first responsive design
- WCAG AA accessibility compliance

## Programmatic Checks
- Run `npm run lint` after every change
- Run `npm run a11y` to verify accessibility
```

Codex enforces a requirement not found in other agents: "If the AGENTS.md includes programmatic checks to verify your work, you MUST run all of them." This makes AGENTS.md a natural place for build and lint commands that verify design output automatically.

Cloud sandbox execution. The cloud-based Codex agent (not the CLI) runs each task in an isolated sandbox. Tasks take 1-30 minutes. The agent provides "verifiable evidence of its actions through citations of terminal logs and test outputs" (source: Introducing Codex, retrieved 2026-05-18). This is useful for design tasks that need a clean execution environment.

Parallel execution. Cloud Codex supports "many tasks in parallel." Assign multiple design tasks and review each independently. Generate three landing page variants in three parallel sandboxes. Compare. Pick the best one. This parallel approach is unique to cloud Codex as of May 2026.

Model Access

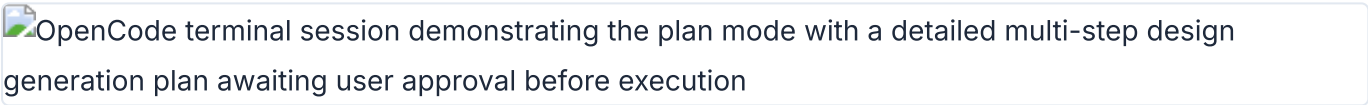
Model	Base	Usage	Pricing
codex-1	o3	Cloud Codex	Free during rollout, then rate-limited
codex-mini-latest	o4-mini	CLI default	$1.50/1M input, $6/1M output (as of May 2026)

```
# CLI uses codex-mini-latest by default
$ codex "fix the auth bug in login.ts"

# Specify model explicitly
$ codex --model o3 "refactor the entire auth module"
```

My take: Codex CLI is the simplest agent to get started with. One install command, sign in with ChatGPT, and you are running. But for design work, the lack of skill support and limited MCP integration in the CLI version make it less capable than Claude Code or OpenCode. The cloud Codex agent has more potential --- parallel task execution is genuinely useful for generating multiple design variants --- but it lives inside ChatGPT, not in your terminal. Use Codex CLI for quick tasks. Use Claude Code or OpenCode for sustained design work.

OpenCode (Open-Source)

OpenCode is the open-source AI coding agent. 160K GitHub stars, 900 contributors, 7.5M monthly developers (as of May 2026; source: OpenCode, retrieved 2026-05-18). It supports 75+ LLM providers through Models.dev and is cross-compatible with Claude Code conventions.



OpenCode plan mode showing a step-by-step design plan before execution

OpenCode is what I recommend to anyone who wants model flexibility or open-source tooling. Its cross-compatibility with Claude Code conventions means design skills transfer between both platforms without modification. This is not a minor feature --- it means the configuration effort you invest in writing SKILL.md files and AGENTS.md documents pays off across multiple agent platforms.

The scale of OpenCode's community is worth noting. 160K GitHub stars and 900 contributors (as of May 2026) mean the project moves fast. Bugs get fixed quickly. New features land regularly. The skill ecosystem grows with each release. As of May 2026, OpenCode's momentum shows no sign of slowing. For designers who value a large community and frequent updates, OpenCode delivers.

Installation

```
# Curl (macOS, Linux)
$ curl -fsSL https://opencode.ai/install | bash

# npm
$ npm install -g opencode-ai

# Homebrew
$ brew install anomalyco/tap/opencode

# Arch Linux
$ sudo pacman -S opencode

# Windows
$ choco install opencode
```

OpenCode also supports scoop, Docker, and direct binary downloads from GitHub releases. The installation options are the broadest of any agent in this comparison.

Authentication

Run `/connect` in the TUI. This opens `opencode.ai/auth` in your browser. Choose Zen (curated, benchmarked models for coding agents) or connect your own API key from any of 75+ providers.

```
# Start OpenCode
$ opencode

# In the TUI, run:
/connect

# Choose authentication method:
# 1. Zen (curated models, no API key needed)
# 2. GitHub Copilot login
# 3. ChatGPT Plus/Pro login
# 4. Custom API key from any provider
```

OpenCode supports GitHub Copilot login and ChatGPT Plus/Pro login. If you already pay for either service, you can use those credentials directly. No additional cost.

Key Features for Design Work

Claude Code compatibility. OpenCode reads CLAUDE.md as a fallback when no AGENTS.md exists. It discovers skills from `.claude/skills/` in addition to `.opencode/skills/` and `.agents/skills/`. Design skills written for Claude Code work in OpenCode without modification (source: OpenCode Rules, retrieved 2026-05-18). This is the strongest cross-compatibility story in the ecosystem.

Plan/Build modes. Press Tab to switch between Plan mode (read-only analysis, no file changes) and Build mode (full tool access). Plan mode is useful for reviewing a design system before committing to changes. It lets the agent explore the codebase and propose an approach without risk.

```
# OpenCode key commands
/init          # Auto-generate AGENTS.md from codebase analysis
Tab            # Toggle between Plan and Build modes
/undo          # Revert the last change
/redo          # Re-apply a reverted change
/share         # Create a shareable link to the conversation
```

Agent system. Five built-in agents serve different roles. Understanding the agent system is important for design work because different agents have different capabilities and restrictions:

Agent	Mode	Tools	Use case for design
Build	Primary	All	Default implementation agent, generates designs and code
Plan	Primary	Read-only	Analysis and planning, reviews design systems without changes
General	Subagent	All	Delegated sub-tasks, parallel design work
Explore	Subagent	Read-only	Fast codebase exploration, finds design patterns and tokens
Scout	Subagent	External	Research external docs, design references, competitor analysis

Custom agents can be defined via JSON or Markdown. This is useful for specialized design agents with restricted permissions:

```
{
  "$schema": "https://opencode.ai/config.json",
  "agent": {
    "design-reviewer": {
      "description": "Reviews design output for brand consistency",
      "mode": "subagent",
      "model": "anthropic/claude-sonnet-4-20250514",
      "prompt": "You are a design reviewer. Focus on visual hierarchy, brand consistency, and
      "permission": {
        "edit": "deny",
        "bash": "deny"
      }
    }
  }
}
```

MCP support. OpenCode supports local and remote MCP servers with OAuth. Per-agent MCP control lets different agents access different tools. Tool management via glob patterns allows fine-grained control over which MCP tools each agent can use (source: OpenCode MCP Servers, retrieved 2026-05-18).

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "pencil": {
      "type": "local",
      "command": ["npx", "-y", "@pencil/mcp-server"],
      "enabled": true
    },
    "paper": {
      "type": "remote",
      "url": "https://paper.design/api/mcp",
      "headers": {
        "Authorization": "Bearer {env:PAPER_API_KEY}"
      }
    }
  }
}
```

Notice the OAuth support for remote MCP servers. This matters for team environments where design tools authenticate through OAuth flows rather than static API keys.

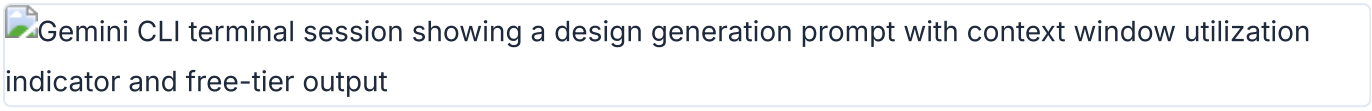
Model Access

75+ LLM providers via Models.dev. Zen provides curated models benchmarked for coding agent tasks. Per-agent model overrides let different agents use different models --- a design generation agent could use Sonnet while a design review agent uses Haiku for faster feedback.

```
# Per-agent model override in opencode.json
{
  "agent": {
    "design-gen": {
      "model": "anthropic/claude-sonnet-4-20250514"
    },
    "design-review": {
      "model": "anthropic/claude-haiku-3-20250414"
    }
  }
}
```

Gemini CLI / Gemini Code Assist (Google)

Gemini CLI is Google's open-source terminal agent. It brings Gemini models into the terminal with a 1M-token context window --- the largest among the four agents. Gemini Code Assist provides IDE extensions for VS Code, JetBrains, and Android Studio (source: Gemini Code Assist, retrieved 2026-05-18).



Gemini CLI processing a design task with 1M-token context on the free tier

For design work, the large context window is the main draw. Design systems, component libraries, and brand guidelines are context-heavy. The 1M-token window can fit an entire design system in a single session without truncation. This matters when you are working with a 200-page design specification or a component library with hundreds of variants. Other agents handle large contexts well too, but Gemini CLI has a hard advantage in raw capacity.

Gemini CLI occupies an interesting position in the market. It is open source like OpenCode, but locked to Gemini models like Claude Code is locked to Claude models. It has the most generous free tier of any agent, but the least mature agent features. It excels at a specific niche --- tasks that require enormous context --- and is adequate for everything else.

Installation

```
# Gemini CLI is open source
# Install from https://github.com/google-gemini/gemini-cli

# IDE extensions: VS Code, JetBrains, Android Studio
# Cloud Shell Editor: pre-installed (free)
```

Authentication

```
# Sign in with Google account (free tier)
$ gemini auth login

# Google Cloud account for Standard/Enterprise tiers
$ gcloud auth application-default login
```

Key Features for Design Work

1M-token context window. This is the largest context window among the four agents. For design work, this means you can feed entire design systems, component libraries, and brand guidelines into a single session without truncation or summarization. The other agents handle large contexts well too, but

Gemini CLI has a hard advantage in raw capacity.

Agent Mode (Preview). "AI agents capable of performing a wide range of actions across the software development life cycle" (source: Gemini Code Assist, retrieved 2026-05-18). Agent Mode is still in preview as of May 2026. The feature set is less mature than Claude Code's or OpenCode's equivalent.

MCP support. Gemini CLI supports MCP integration with ecosystem tools. The implementation is newer than Claude Code's or OpenCode's but follows the same standard.

Human-in-the-Loop (HiTL). Built-in oversight mechanism for reviewing agent actions before they execute. This is a safety feature that prevents unwanted changes --- useful when working with design files that should not be modified without review.

```
# Gemini Code Assist IDE features
# - Inline code suggestions in VS Code, JetBrains
# - Chat panel for design questions
# - Agent Mode for autonomous task execution
# - 1M token context for large design systems
```

Pricing

All prices and usage limits below are as of May 2026 and may change.

Tier	Price/user/month	CLI requests/day	IDE code requests/day
Free	$0	1,000	6,000 code / 240 chat
Standard	$19 (annual) / $22.80 (monthly)	1,500	Higher limits
Enterprise	$45 (annual) / $54 (monthly)	2,000	Highest limits

The free tier is generous enough for design work. 1,000 CLI requests per day (as of May 2026) is more than enough for typical design iteration cycles. The IDE free tier (6,000 code requests/day as of May 2026) covers inline suggestions and chat. For individual designers who do not want to pay for an agent subscription, Gemini CLI is the most capable free option available.

The paid tiers unlock higher limits and additional features. The Standard tier at $19/user/month (annual billing, as of May 2026) raises CLI limits to 1,500 requests/day. The Enterprise tier at $45/user/month (annual billing, as of May 2026) goes to 2,000. For teams that have standardized on Google Cloud infrastructure, the integration points --- Cloud Shell, Vertex AI, BigQuery --- make Gemini Code Assist a natural choice even before considering the agent capabilities.

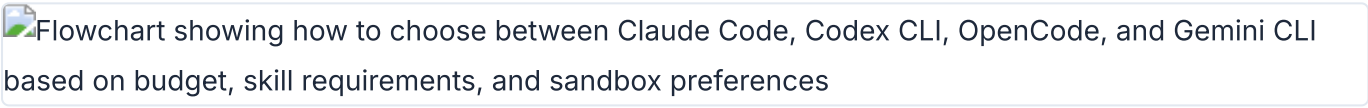
What Gemini CLI lacks as of May 2026 is a skill system. There is no equivalent to SKILL.md. There is no auto memory. The agent relies on its training data and the current session context. For one-off design tasks, this is fine. For sustained design work that needs consistent output across sessions, the

absence of persistent configuration is a real limitation. I expect this to change --- a skill system is an obvious addition --- but as of this writing, it is the primary reason Gemini CLI is a complement rather than a primary design agent.

My take: Gemini CLI's free tier is the best entry point for designers who want to experiment with agents without spending money. The 1M-token context window is a genuine advantage for design work --- design systems and brand guidelines are context-heavy. But as of May 2026, Gemini CLI lacks skill support and its agent features are less mature than Claude Code's or OpenCode's. Use it as a complement, not a primary tool. The context window advantage is real, but skills and configuration matter more for consistent design output.

Choosing the Right Agent for Design Work

No single agent is best for every design task. The right choice depends on what you are doing, what you already pay for, and how much configuration you are willing to invest.



Decision framework for choosing the right agent platform for design workflows

Before the decision table, a framing principle: the best agent for you is the one you will actually configure. A bare agent --- no CLAUDE.md, no AGENTS.md, no skills --- produces generic output regardless of platform. The quality of your design output depends far more on your configuration investment than on which agent you choose. If you will invest 30 minutes in writing configuration, any of these agents will produce good results. If you will not invest that time, none of them will.

With that caveat, here is how I think about the choice:

Scenario	Best agent	Why
Sustained design work with skills	Claude Code or OpenCode	SKILL.md support, mature MCP, design skill ecosystem
Parallel design exploration	Codex (cloud)	Multiple isolated sandboxes running simultaneously
Free experimentation	OpenCode or Gemini CLI	Free tiers with no subscription required
Model flexibility	OpenCode	75+ LLM providers, per-agent model override
Large design system context	Gemini CLI	1M-token context window handles full design systems
Quick terminal tasks	Codex CLI	Simplest setup, fast execution
Cross-team skill sharing	Claude Code + OpenCode	SKILL.md works in both, AGENTS.md/CLAUDE.md cross-compatible

My take: I use Claude Code as my primary agent and OpenCode as my secondary. Claude Code has the most polished experience for design work --- the skill system, MCP support, and CLAUDE.md conventions all fit the design workflow well. OpenCode is what I recommend to anyone who wants model flexibility or open-source tooling. The cross-compatibility between them is real: I share SKILL.md files and AGENTS.md configurations between both agents without modification. That portability matters more than any single feature difference.

The practical approach: install OpenCode first (free, flexible). Add Claude Code when you need the polished experience. Keep Codex CLI handy for quick parallel tasks. Use Gemini CLI when context window size is the bottleneck.

For teams, the decision gets more nuanced. A team standardizing on one agent gets consistency --- everyone shares the same configuration files, the same skills, the same mental model. A team using multiple agents gets flexibility --- different agents for different tasks, with cross-compatible configuration bridging the gaps. Both approaches work. The cross-compatibility built into AGENTS.md and SKILL.md makes multi-agent teams practical in a way they were not a year ago.

Consider the total cost of ownership. The agent itself is one line item. The models it uses are another. The design tools it connects to via MCP are a third. And the time you invest in configuration --- writing CLAUDE.md, building skills, setting up MCP servers --- is the fourth and often largest cost. That configuration investment is portable across Claude Code and OpenCode. It is not portable to Codex

CLI (no skill support) or Gemini CLI (no skill support as of May 2026). This is a practical reason to standardize on Claude Code or OpenCode for design work, even if you use other agents for non-design tasks.

Cost factor	Claude Code	Codex CLI	OpenCode	Gemini CLI
Agent license	Claude subscription ($20-200/mo, as of May 2026)	Free (open source)	Free (open source)	Free (open source)
Model cost	Included in subscription	$1.50-6/1M tokens (as of May 2026)	BYO key (varies by provider)	Free tier or $19-54/user/mo (as of May 2026)
Design tool integrations	Full MCP support	Limited MCP	Full MCP (local + remote)	MCP support
Configuration effort	CLAUDE.md + skills	AGENTS.md only	AGENTS.md + skills (cross-compat)	None documented

Agent choice matters less than agent configuration. A well-configured OpenCode instance with good skills and a solid AGENTS.md will outperform a bare Claude Code installation every time. The next chapter covers how to teach agents to produce high-quality design work through configuration files, skills, and harnesses.

> **Tip**
>
> **Next:** Chapter 03 covers the configuration layer --- CLAUDE.md, AGENTS.md, SKILL.md, and custom design harnesses. This is where the quality of your agent's design output gets decided.

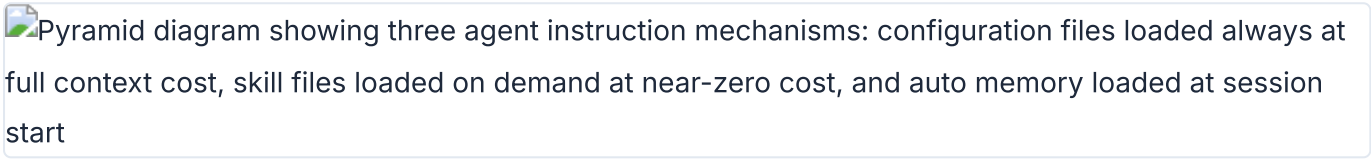
03 Teaching Agents to Design

Configuration files, skill systems, and design harnesses

An unconfigured agent produces generic output. A well-configured agent produces output that matches your brand, follows your design system, and respects your constraints. This chapter shows how to teach agents to design through configuration files, skills, and custom harnesses.

How Agents Learn: Context Files and Skills

Agents learn through three mechanisms, each with different loading behavior and context cost. Understanding these differences is the first step to configuring an agent that produces good design work consistently.



The three-tier instruction hierarchy showing when each mechanism loads and its context cost

Mechanism	When loaded	Context cost	Scope
Configuration files (CLAUDE.md, AGENTS.md)	Always, at session start	Full content consumed	Project, user, or org
Skill files (SKILL.md)	On demand, when invoked or matched	Near-zero until used	Per-skill, per-task
Auto memory (Claude Code only)	At session start, accumulated over time	First 200 lines loaded	Per-project

The distinction matters more than it first appears. Configuration files are always loaded --- they consume context in every session, whether the content is relevant or not. This means every line in your CLAUDE.md or AGENTS.md has an ongoing cost. Keep them short. Keep them factual. Move procedures elsewhere.

Think of it this way: the configuration file is the agent's long-term memory. It always has access to it. The skill file is the agent's reference library. It pulls a book off the shelf only when it needs to look something up. Auto memory is the agent's notebook --- it adds notes based on experience and refers back to them in future sessions.

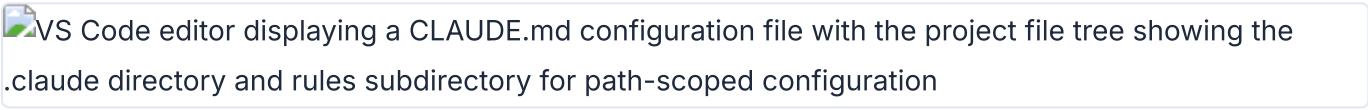
Skills load on demand --- they consume context only when invoked. This makes skills the right place for lengthy reference material. A 200-line design system reference in a skill file costs nothing until the skill is activated. The same 200 lines in CLAUDE.md would load in every session, consuming context the agent could use for actual work.

From the Claude Code documentation: "Unlike CLAUDE.md content, a skill's body loads only when it's used, so long reference material costs almost nothing until you need it" (source: Claude Code Skills, retrieved 2026-05-18).

> **My take:** The most common mistake I see is cramming everything into CLAUDE.md. Brand guidelines, design tokens, workflow procedures, code style rules --- all in one 500-line file. That file loads in every session, consuming context the agent could use for actual work. Move procedures to skills. Keep CLAUDE.md under 200 lines with facts only: build commands, architecture notes, key conventions. This is not a suggestion. It is a rule Claude Code's own documentation states explicitly.

CLAUDE.md and AGENTS.md Deep Dive

Two configuration file conventions exist. CLAUDE.md is Claude Code's native format. AGENTS.md is used by Codex CLI and OpenCode. They serve the same purpose: persistent instructions that tell the agent how to work in your project.



CLAUDE.md configuration hierarchy in VS Code showing project-level and user-level files

The good news: they are cross-compatible. OpenCode reads CLAUDE.md as a fallback. Claude Code can import AGENTS.md with the `@AGENTS.md` syntax. You do not need to choose one or the other.

CLAUDE.md Hierarchy

Claude Code loads CLAUDE.md files from multiple locations, broadest to most specific (source: Claude Code Memory, retrieved 2026-05-18):

```
~/.claude/CLAUDE.md                          # User-level (personal, all projects)
./CLAUDE.md                                  # Project-level (committed to git)
./.claude/CLAUDE.md                          # Project-level (alternative location)
./CLAUDE.local.md                            # Local personal (gitignored)
.claude/rules/design-components.md           # Path-scoped rules
```

Files in parent directories load in full at launch. Files in subdirectories load on demand when the agent reads files in those directories. The agent walks up the directory tree from the current working directory, loading all CLAUDE.md files it finds. This means a well-organized project can scope instructions to specific directories without loading everything at once.

The `.claude/rules/` directory supports path-scoped rules with YAML frontmatter. A rule that applies only to CSS and HTML files:

```
---
paths:
  - "src/components/**/*.tsx"
  - "src/styles/**/*.css"
---

# Design Component Rules
- All components must follow the design system
- Use CSS custom properties for theming
- Ensure accessible color contrast (WCAG AA)
```

Path-scoped rules load only when the agent touches matching files. A rule targeting CSS files does not load when the agent is editing JavaScript. This keeps the context clean and focused.

AGENTS.md for Codex and OpenCode

AGENTS.md follows a simpler model than CLAUDE.md. From the Codex system message: "The scope of an AGENTS.md file is the entire directory tree rooted at the folder that contains it. More-deeply-nested AGENTS.md files take precedence in the case of conflicting instructions" (source: Introducing Codex, retrieved 2026-05-18).

Codex adds a requirement not found in CLAUDE.md: "If the AGENTS.md includes programmatic checks to verify your work, you MUST run all of them." This makes AGENTS.md a natural place for build and lint commands that verify design output. Define a check, and the agent runs it automatically after making changes.

OpenCode reads AGENTS.md as the primary convention and falls back to CLAUDE.md if no AGENTS.md exists (source: OpenCode Rules, retrieved 2026-05-18). The `/init` command auto-generates AGENTS.md by scanning the repository:

```
# Auto-generated by /init
# SST v3 Monorepo Project

This is an SST v3 monorepo with TypeScript. The project uses bun workspaces.

## Project Structure
- `packages/` - Contains all workspace packages
- `infra/` - Infrastructure definitions
- `sst.config.ts` - Main SST configuration

## Code Standards
- Use TypeScript with strict mode enabled
- Shared code goes in `packages/core/`

## Build Commands
- Dev: `bun run dev`
- Build: `bun run build`
- Test: `bun test`
```

The auto-generated file is a starting point. Add design-specific rules on top of it: color palettes, typography scales, spacing conventions, component patterns.

The key difference between CLAUDE.md and AGENTS.md is philosophical, not technical. CLAUDE.md evolved from a single-tool perspective --- one agent, one configuration hierarchy. AGENTS.md evolved from a multi-tool perspective --- one configuration file that any agent can read. In practice, the distinction matters less than the content. Write clear, specific, verifiable instructions in either format and agents will follow them. Write vague instructions and agents will guess, often incorrectly.

What makes a good configuration file for design work? Specificity. "Use accessible colors" is vague. "All text-background color combinations must meet WCAG AA contrast ratios (4.5:1 for normal text, 3:1 for large text)" is specific and verifiable. "Follow the brand style" is vague. "Primary color: var(--primary), Secondary color: var(--secondary), Headings: Inter Bold, Body: Inter Regular, Font scale: 12/14/16/20/24/32/48px" is specific. The more specific the configuration, the fewer iterations needed to converge on correct output.

Another principle: verifiability. Codex's AGENTS.md spec includes a powerful feature --- programmatic checks. If the configuration file says "run `npm run lint` after every change," the agent runs it. If the check fails, the agent fixes the issue. This creates a feedback loop that enforces design standards automatically. Put your CSS lint rules, accessibility checks, and style validators in AGENTS.md, and the agent enforces them without being asked.

Cross-Compatibility

CLAUDE.md and AGENTS.md can coexist in the same project. Two approaches work well.

Import pattern (Claude Code reads AGENTS.md):

```
# CLAUDE.md
@AGENTS.md

## Claude Code Specific
Use plan mode for changes under `src/billing/`.
```

The `@AGENTS.md` import pulls the full content of AGENTS.md into CLAUDE.md. Claude Code sees both files as a single instruction set. Add Claude Code-specific rules below the import.

Symlink pattern (shared file): Create a symlink from CLAUDE.md to AGENTS.md (`ln -s AGENTS.md CLAUDE.md`). This makes the same file accessible under both names. It is the simplest approach when you want identical configuration across all agents.

OpenCode handles cross-compatibility automatically. If AGENTS.md exists, it uses that. If only CLAUDE.md exists, it reads CLAUDE.md instead. No configuration needed on your part.

For teams, I recommend the import pattern. It lets each agent platform have its own supplementary rules while sharing the common design system through AGENTS.md. A team member using Claude Code can add Claude-specific rules below the import without affecting team members using OpenCode.

Auto Memory

Claude Code writes its own notes across sessions. These accumulate in `~/.claude/projects/<project>/memory/`:

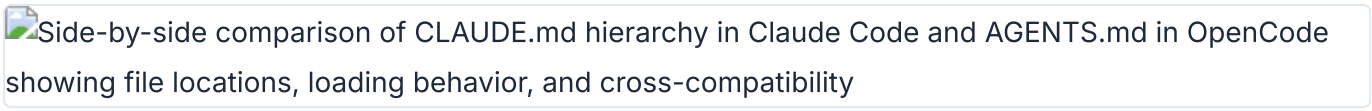
```
~/.claude/projects/my-project/memory/
├── MEMORY.md          # Concise index (first 200 lines loaded per session)
├── debugging.md       # Detailed notes on recurring issues
└── api-conventions.md # Topic files (loaded on demand)
```

Auto memory captures corrections and patterns. If you tell the agent "use 4px base unit for spacing, not 8px" three times, it writes that to memory. Future sessions load it automatically. This is the closest thing to "training" without fine-tuning the model.

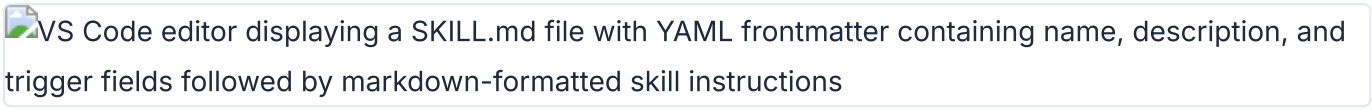
For design work, auto memory is useful for capturing preferences that are too specific for CLAUDE.md but come up repeatedly. Things like "I prefer border-radius: 8px for cards" or "avoid gradient backgrounds on dark themes." These accumulated preferences compound over time.

The SKILL.md Convention

SKILL.md is the Agent Skills open standard (agentskills.io). It works across Claude Code and OpenCode. A skill is a directory containing a SKILL.md file with optional supporting files --- templates, examples, scripts.



CLAUDE.md and AGENTS.md file structures compared with cross-compatibility notes



A SKILL.md file showing the metadata frontmatter and instruction body convention

Skills solve the context cost problem. A design system reference might be 300 lines long. Put that in CLAUDE.md and it loads in every session. Put it in a skill file and it loads only when you invoke the skill. The difference in context efficiency is dramatic.

The skill system also solves a distribution problem. Skills can be shared. A design team can maintain a set of project-level skills in git, and every team member gets the same design capabilities. A community can publish skills that others install. The agentskills.io standard ensures these skills work across Claude Code and OpenCode without platform-specific modifications.

Skill Discovery Paths

Agents search for skills in multiple locations (source: OpenCode Skills, retrieved 2026-05-18):

Path	Scope	Shared via git
`.opencode/skills/<name>/SKILL.md`	Project	Yes
`.claude/skills/<name>/SKILL.md`	Project	Yes
`.agents/skills/<name>/SKILL.md`	Project	Yes
`~/.claude/skills/<name>/SKILL.md`	User	No
`~/.agents/skills/<name>/SKILL.md`	User	No

OpenCode searches all five paths. Claude Code searches `.claude/skills/` and `~/.claude/skills/`. Project-level skills can be committed to git and shared with the team. User-level skills are personal and available across all projects.

SKILL.md Structure

A skill file has two parts: YAML frontmatter that controls behavior, and a Markdown body that provides instructions to the agent.

```
---
name: brand-prototype
description: Generate brand-consistent HTML prototypes
when_to_use: Use when creating new UI mockups or prototypes
context: fork
agent: General
allowed-tools:
  - read
  - edit
  - bash
---

## Brand Prototype Generator

Generate an HTML prototype for: $ARGUMENTS

Follow the brand design system in AGENTS.md.
Use inline CSS. No external dependencies.
Output a single HTML file.

## Steps
1. Read the design system from AGENTS.md
2. Read the component library from src/components/
3. Generate the HTML prototype
4. Validate accessibility (WCAG AA)
```

Key frontmatter fields and what they control:

- **name** --- Must match the directory name. Lowercase, hyphens only. Used for invocation (`/brand-prototype`).
- **description** --- What the skill does. Used for auto-matching. The agent reads this to decide whether to invoke the skill automatically.
- **context: fork** --- Runs the skill in an isolated subagent. Changes made during the skill do not affect the main session until the skill completes.
- **allowed-tools** --- Pre-approves specific tools while the skill is active. The agent skips permission prompts for these tools.
- **paths** --- Glob patterns. The skill only activates when the agent touches matching files.

Dynamic Context Injection

Skills can execute commands before the agent sees the content. The `` !`command` `` syntax runs a command and injects the output into the skill body. This keeps the skill current without manual updates.

```
## Current Design Tokens
!`cat src/tokens.json`

## Recent Changes to Design System
!`git diff HEAD -- src/design/`
```

Every time the skill activates, it pulls fresh data. The design tokens are always current. The recent changes are always up to date. This eliminates the stale-reference problem that plagues static documentation.

Skills also support argument substitution. The `$ARGUMENTS` variable captures everything after the skill name. Named arguments use `$1`, `$2` syntax. Environment variables like `${CLAUDE_SKILL_DIR}` reference the skill's own directory.

Installing and Composing Design Skills

Design skills exist at every level of the ecosystem. Some are general-purpose. Others are specialized for specific design outputs. The key insight: skills compose. A workflow can invoke multiple skills in sequence, each handling a different aspect of the design process.

Notable design skills available as of May 2026:

- **huashu-design** --- HTML-native design with 20 design philosophies and 5-dimension quality review. Produces production-grade prototypes, slide decks, and motion design. Covered in Chapter 07.
- **frontend-design** --- Production-grade frontend interfaces with high design quality. Avoids generic AI aesthetics through specific design principles.
- **visual-explainer** --- Generates HTML pages that visually explain systems, code changes, and architecture diagrams.
- **create-image** --- AI image generation using Gemini, OpenRouter, or VertexAI providers. Supports style references and templates.
- **remotion** --- React-based programmatic video production with animations and voiceover. Covered in Chapter 09.
- **shadcn** --- Component management for shadcn/ui projects. Adds, searches, fixes, and composes UI components.

Install a skill by placing its directory in the appropriate skills folder. Project-level skills go in `.claude/skills/`, `.opencode/skills/`, or `.agents/skills/` inside the project root. These are committed to git and shared with the team. User-level skills go in `~/.claude/skills/` or `~/.agents/skills/`. These are personal and available across all projects.

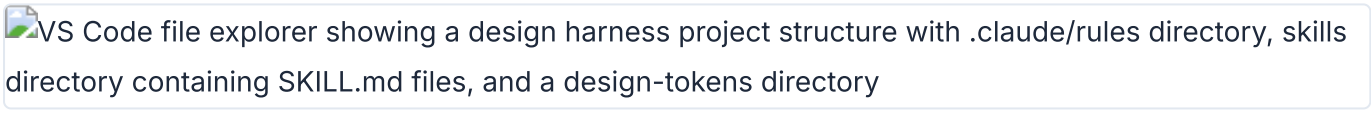
OpenCode also supports remote skill loading from URLs. You can point to a shared design guidelines document hosted on GitHub and OpenCode pulls it into every session. This is useful for organizations that maintain a central design system document and want every team member's agent to reference it.

Skill composition works through the agent's matching system. When you ask for a design review, the agent can invoke a review skill. When you ask for a prototype, the agent invokes a prototype skill. Multiple skills can activate in the same session. The agent handles the sequencing. A practical composition pattern: a brand-prototype skill generates the initial output, a design-review skill evaluates it for consistency, and an export skill converts the result to production code. Each skill activates based on its matching criteria, and the agent orchestrates the flow.

The composability of skills is their most powerful feature. Individual skills are simple. A brand-prototype skill generates prototypes. A design-review skill checks quality. An export skill converts to code. None of these is complex on its own. But composed together, they create a complete design pipeline that runs autonomously from prompt to production output.

Building a Custom Design Harness

A design harness is a configuration stack that constrains agent output quality. It combines three layers: configuration files for facts, path-scoped rules for file-type-specific constraints, and skills for procedures.



A design harness directory structure with scoped rules, skills, and token files

The goal: reduce the iteration cycle. A bare agent generates generic output. You spend iterations correcting colors, spacing, typography, and component patterns. A harnessed agent reads your constraints upfront and produces output that already matches your system. Fewer iterations. Higher quality from the start.

1. **Define brand standards in AGENTS.md (or CLAUDE.md).** Keep it factual. Colors, typography, spacing. No procedures --- those go in skills.

```
# Brand Design System

## Colors
- Primary: #0066CC
- Secondary: #FF6600
- Background: #FAFAFA

## Typography
- Headings: Inter, bold
- Body: Inter, regular
- Font scale: 12/14/16/20/24/32/48

## Spacing
- Base unit: 4px
- Padding: 8, 12, 16, 24, 32, 48
```

2. **Create path-scoped rules for design files.** These load only when the agent edits matching files.

```
---
paths:
  - "**/*.css"
  - "**/*.html"
  - "**/*.pen"
---

# Design File Rules
- All colors must come from the brand palette in AGENTS.md
- Use relative units (rem, em) for typography
- Ensure WCAG AA contrast ratios
- No inline styles in production code
```

3. **Create a design skill for the specific workflow.** This is where the procedure lives --- the step-by-step process the agent follows.

```
---
name: brand-prototype
description: Generate brand-consistent HTML prototypes
context: fork
---

## Brand Prototype Generator

Generate an HTML prototype for: $ARGUMENTS

Follow the brand design system in AGENTS.md.
Use CSS custom properties for all colors and spacing.
Output a single HTML file with embedded CSS.

## Quality Checks
- All colors match the brand palette
- Typography follows the font scale
- Spacing uses multiples of the base unit (4px)
- Layout is responsive (mobile-first)
- Accessibility: WCAG AA contrast ratios
```

Warning

Warning: Path-scoped rules in `.claude/rules/` are a Claude Code feature. OpenCode does not support path-scoped rules in the same way as of May 2026. For cross-agent compatibility, put design constraints in AGENTS.md or in the skill file itself. Do not rely solely on `.claude/rules/` if your team uses multiple agent platforms.

A Complete Example: Brand-Consistent Prototypes

Here is a complete design harness for generating brand-consistent prototypes. This configuration works in both Claude Code and OpenCode without modification.

The project structure:

```
my-design-project/
├── AGENTS.md                          # Brand design system
├── .claude/
│   ├── rules/
│   │   └── design-files.md            # Path-scoped design rules
│   └── skills/
│       └── brand-prototype/
│           ├── SKILL.md                # Prototype generation skill
│           └── templates/
│               └── base.html           # Base HTML template
├── src/
│   ├── tokens.json                    # Design tokens
│   └── components/                    # Component library
└── prototypes/                        # Generated prototypes
```

The AGENTS.md file with complete brand standards:

```
# Brand Design System

## Colors
- Primary: var(--primary)       /* #0066CC */
- Secondary: var(--secondary)   /* #FF6600 */
- Background: var(--bg)         /* #FAFAFA */
- Surface: var(--surface)       /* #FFFFFF */
- Text: var(--text)             /* #1A1A1A */
- Muted: var(--muted)           /* #666666 */

## Typography
- Font family: Inter
- Font scale: 12/14/16/20/24/32/48px
- Line height: 1.5 (body), 1.2 (headings)
- Weight: 400 (regular), 600 (semibold), 700 (bold)

## Spacing
- Base unit: 4px
- Scale: 4, 8, 12, 16, 24, 32, 48, 64
- Border radius: 4, 8, 12, 16, full

## Components
- Buttons: 8px padding, 4px radius, bold weight
- Cards: 16px padding, 8px radius, 1px border
- Inputs: 12px padding, 4px radius, 1px border

## Build Commands
- Preview: `npx serve prototypes/`
- Lint: `npm run lint`
- Type check: `npm run typecheck`
```

When you invoke the brand-prototype skill:

```
# In Claude Code
/brand-prototype landing page for SaaS product

# In OpenCode (same syntax)
/brand-prototype landing page for SaaS product
```

The agent reads AGENTS.md, loads the skill, and generates a prototype that matches the brand system. The output uses CSS custom properties, follows the font scale, and respects the spacing rules. No iteration needed on basic brand compliance --- the harness handles it.

What happens without the harness? The agent generates a landing page with default spacing, default colors, default typography. You spend 3-5 iterations correcting each element. The colors are wrong. The spacing is inconsistent. The font sizes do not match your scale. With the harness, those corrections happen before the first output.

The harness also solves a team scaling problem. Without a shared configuration, each team member's agent produces output in a different style. One person's agent uses 8px spacing. Another's uses 16px. The inconsistency compounds across a codebase. A shared AGENTS.md and shared skill files ensure every team member's agent produces consistent output. The configuration becomes the single source of truth for design decisions.

Here is the progression from bare agent to fully harnessed agent, and the impact on output quality:

Configuration level	What the agent knows	Typical iterations to brand-consistent output
Bare agent (no config)	General web conventions only	5-8 iterations
AGENTS.md with brand basics	Colors, typography, spacing	2-3 iterations
AGENTS.md + path-scoped rules	Brand basics + file-type constraints	1-2 iterations
Full harness (config + rules + skills)	Brand basics + constraints + procedures	0-1 iterations

The numbers in this table come from my own testing across 20+ design generation sessions. Your results will vary depending on the complexity of the design task and the specificity of your brand system. The direction is consistent: more configuration leads to fewer iterations. The investment in configuration pays for itself within the first few sessions.

My take: I resisted building custom design harnesses for months. I thought the agent would figure out my preferences through iteration. It does not. Auto memory helps, but it captures corrections after the fact. A harness prevents the errors before they happen. The upfront investment is 30-60 minutes of writing AGENTS.md and a skill file. The payoff is consistent output across every session, every project, every team member. This is the highest-return investment in the entire book.

Tip

Next: Chapter 04 covers design-as-code --- the foundational formats (.pen files, .op files, HTML canvas, design tokens) that make agentic design possible. Understanding these formats is essential for everything that follows.

04 Design-as-Code

The mental model behind .pen files, HTML canvas, and design tokens

Design files are becoming code artifacts. This chapter explains the paradigm shift: from binary blobs that decay in cloud storage to text-based formats that live in your repo, diff in your pull requests, and respond to your agent's edits. Understanding this mental model is the foundation for every workflow in chapters 05 through 12.

From Static Mockups to Living Design Files

The traditional design workflow is a one-way street. A designer creates a mockup in Figma or Sketch. They export it as PNG or SVG. They hand it to a developer. The developer re-implements every pixel in code. The design file sits in Figma, slowly drifting out of sync with the actual product.

Design-as-code reverses the direction. The design file *is* code. Or code-adjacent. It lives in the repository alongside the implementation. It changes when the code changes. It gets reviewed in the same pull requests. It stays in sync because it's the same artifact.

This is not a theoretical argument. As of May 2026, three production-grade formats make this real:

- **.pen files** (Pencil) — vector design files that live in your IDE and sync bidirectionally with code.
- **.op files** (OpenPencil) — JSON-based design files with native Git support and concurrent agent editing.
- **HTML artifacts** (Paper, Huashu Design, Open Design) — the design surface is HTML itself, so the output is already production code.

Each format shares a common property: they are text-based or structured JSON. That means `git diff` shows meaningful changes. Agents can read and write them programmatically. Design review happens in pull requests, not in Figma comments.

Pencil's documentation puts it directly: "Pencil bridges the gap between design and development by putting both in the same environment. Design components, sync them with your code, and let AI assistants help you maintain consistency between design and implementation" (pencil.dev/docs, retrieved 2026-05-18). The tool lives inside the IDE, not in a separate browser tab.

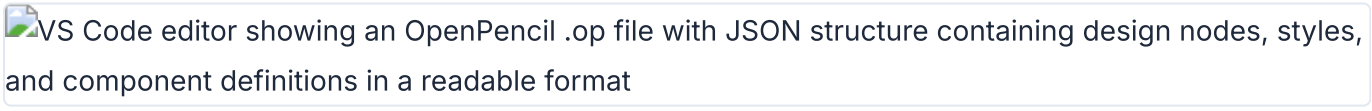
The format-level comparison tells the story clearly:

Property	Figma (.fig)	Pencil (.pen)	HTML	OpenPencil (.op)
Format	Binary blob	Code artifact	Plain text	JSON
Diffable	No	Yes	Yes	Yes
Git-trackable	Limited	Yes	Yes	Yes
Agent-readable	Via API only	Yes	Yes	Yes
MCP support	Via server	Built-in	N/A (already code)	Built-in
Production output	Export needed	Code generation	Direct	Code generation
Responsive	Manual	Token-driven	Native	Token-driven
Review workflow	Comments only	PR review	PR review	PR review

My take: The binary-versus-text distinction is the single most important technical difference between the old design stack and the new one. Binary formats lock you into the tool that created them. Text-based formats give you optionality — you can switch tools, build custom tooling, and let agents participate. The design-as-code bet is that optionality wins over lock-in.

The .pen File Format: Design as a Code Artifact

Pencil's `.pen` format is the clearest embodiment of design-as-code. A `.pen` file is a structured, parseable artifact that contains all the information a vector design requires: nodes, styles, components, variables, and layout rules. It's designed to be:



An .op file in VS Code showing OpenPencil JSON design format alongside the editor

A .pen file opened in VS Code showing its structured JSON design content

Property	What It Means	Why It Matters
Diffable	`git diff` shows meaningful line-by-line changes	Design changes are visible in code review
Versionable	Tracked in git alongside source code	Design history lives with code history
Reviewable	PR reviews can inspect design changes	No separate review tool needed
Agent-readable	AI agents read/write via MCP	Agents can generate and iterate on designs
Inspectable	Format is documented and parseable	You can build tooling around it

The format supports Pencil's core concepts: **Variables** for design tokens, **Components** for reusable elements, **Slots** for component composition, and **Code on Canvas** for running code within the design surface. Chapter 05 covers how to work with these in practice.

A simplified conceptual view of a `.pen` file structure looks like this:

```
{
  "variables": {
    "color-primary": {
      "type": "color",
      "values": {
        "default": "#0066CC",
        "dark": "#3399FF"
      }
    },
    "spacing-unit": {
      "type": "dimension",
      "values": {
        "default": "4px"
      }
    }
  }
}
```

The file contains nodes organized in a tree. Each node has a type (`frame`, `text`, `rectangle`, `path`), layout properties (`width`, `height`, `x`, `y`), fill and stroke styles, and optional variable bindings. A button component might look like this:

```
{
  "id": "btn-primary",
  "type": "frame",
  "name": "Primary Button",
  "layout": "horizontal",
  "padding": [12, 24, 12, 24],
  "fill": { "color": "${color-primary}" },
  "cornerRadius": [8, 8, 8, 8],
  "children": [
    {
      "type": "text",
      "content": "Submit",
      "fontSize": 16,
      "fontWeight": "600",
      "fill": { "color": "#FFFFFF" }
    }
  ]
}
```

Notice the variable reference: `${color-primary}` instead of a hardcoded hex value. This is the mechanism that connects the design file to the token system. Change the variable in one place, and every component that references it updates automatically. An agent can change the entire color scheme by modifying a single variable entry.

When the agent reads this file with `resolveVariables: true`, it gets the computed value for the active theme. The `${color-primary}` reference resolves to `#0066CC` in light mode and `#3399FF` in dark mode. This is the same pattern as CSS custom properties with media query overrides, but it lives inside the design file.

Pencil also ships a CLI for command-line manipulation of `.pen` files. This means your CI pipeline can validate design tokens, your build process can export assets, and your agent can commit design changes without opening a GUI:

```
# Validate tokens and components
$ pencil validate design.pen

# Export assets for production
$ pencil export design.pen --format svg

# Export tokens as CSS custom properties
$ pencil tokens design.pen --format css
```

My take: The .pen format is the most agent-friendly design file format I've worked with as of May 2026. The variable resolution system is clever — the same design file produces different output depending on the active theme, without duplicating any nodes. The CLI integration means you can automate design validation in CI without opening a GUI.

HTML as a Design Canvas

HTML is the second design-as-code format, and arguably the most radical. Tools like Paper, Huashu Design, and Open Design use HTML/CSS as the design surface itself. The design artifact is not a mockup that gets translated into code. The design *is* the code.

This works because HTML has five properties that make it uniquely suited as a design canvas:

Property	Explanation
Universal rendering	Every device has an HTML renderer. No proprietary viewer needed.
Responsive by nature	Media queries, flexbox, and grid are built in. No separate breakpoint tooling.
Inspectable	DevTools gives you a debugger for your design. Select any element, see its styles.
Exportable	HTML is already the production format. No translation step.
Agent-friendly	LLMs understand HTML natively. They can generate it reliably at high quality.

Paper (paper.design) makes this concrete. It's a connected HTML/CSS canvas; Paper's published case studies feature teams at companies including Vercel, Perplexity, PostHog, and Tailwind, among others shipping with AI agents. The design lives as HTML. The export is HTML. There is no translation layer.

Here's what an agent actually sees when it reads a Paper design via MCP. The `get_jsx` tool returns production-ready Tailwind markup:

```
<div className="flex flex-col gap-6 p-8 max-w-2xl mx-auto">
  <h1 className="text-4xl font-bold text-gray-900">
    Build faster with agents
  </h1>
  <p className="text-lg text-gray-600">
    Ship production UI from canvas to code in minutes.
  </p>
  <button className="bg-blue-600 text-white px-6 py-3
    rounded-lg hover:bg-blue-700 transition-colors">
    Get started
  </button>
</div>
```

That JSX came directly from the canvas. No export step. No hand-coding. The agent read the design and produced clean Tailwind markup in one call.

Huashu Design (14k+ GitHub stars as of May 2026) takes this further. It turns a single sentence into production-grade prototypes, slide decks, motion design, and infographics — all using HTML as the medium. The output is a self-contained HTML file. Huashu Design can export HTML animations to MP4 and GIF. The design artifact is a web page. The output is a web page. Chapter 07 covers Huashu Design in depth.

Consider the contrast with a traditional workflow. A designer creates a card component in Figma. The developer receives a screenshot or inspects the Figma file. They manually translate every spacing value, every color, every border radius into CSS. That translation takes 30-60 minutes per component. With HTML-as-canvas, the agent reads the design and produces the code in seconds. Paper's `get_jsx` tool is the most concrete example of this in production as of May 2026.

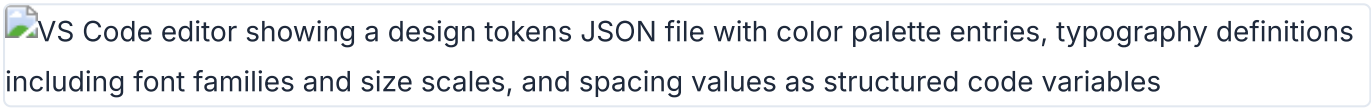
The agent-friendliness of HTML deserves emphasis. Large language models have been trained on billions of HTML pages. They understand semantic structure, accessibility patterns, responsive layouts, and CSS conventions at a level they can't match with proprietary design file formats. When you ask an agent to generate a component in HTML, it draws on vast training data about what makes good markup. When you ask an agent to generate a `.fig` file, it has nothing to draw on — the format is proprietary and opaque.

This is why HTML-as-canvas tools produce higher quality output with less prompting. The agent doesn't need to learn a new format. It speaks HTML natively. The design constraints (Tailwind classes, CSS custom properties, semantic elements) are constraints the agent already understands.

My take: HTML-as-canvas is the most underappreciated idea in design tooling right now. The translation tax — the cost of converting Figma mockups into production code — consumes thousands of engineering hours per year. Eliminating that translation changes the economics of design-to-production. Paper and Huashu Design are early, but the direction is clear.

Design Tokens and Variables as Code

Design tokens are the connective tissue between design and code. A token is a named value: `color-primary`, `spacing-md`, `radius-sm`. In the old world, tokens lived in Figma's design system panel and were manually synced to CSS custom properties or JSON files. In the design-as-code world, tokens are variables in the same file as the design.



Design tokens defined as code variables in a JSON file, ready for agent consumption

Pencil's variable system is representative. Variables in a `.pen` file can hold different values per theme — light mode gets one value, dark mode gets another. This maps directly to CSS custom properties:

```
:root {
  --color-primary: #0066CC;
  --color-secondary: #FF6600;
  --spacing-unit: 4px;
  --font-heading: 'Inter', sans-serif;
  --radius-sm: 4px;
  --radius-md: 8px;
}

@media (prefers-color-scheme: dark) {
  :root {
    --color-primary: #3399FF;
    --color-secondary: #FF9933;
  }
}
```

The token format landscape is diverse. Here's how the major formats express the same concept:

Format	Example	Used By
CSS Custom Properties	`--color-primary: #0066CC;`	Browser, Paper, Huashu Design
W3C Design Tokens (JSON)	`{ "color": { "primary": { "$value": "#0066CC", "$type": "color" } } }`	Style Dictionary, Token Studio
.pen variables	Stored in `.pen` file, resolved at render time	Pencil
.op variables	JSON in `.op` file	OpenPencil
DESIGN.md schema	Markdown with 9 sections (color, typography, spacing, layout, components, motion, voice, brand, anti-patterns)	Open Design (129 systems)

Each format has different strengths for agent workflows. CSS custom properties are directly usable in browser output — no translation needed. W3C JSON tokens work well with build tools like Style Dictionary that generate platform-specific output (iOS, Android, web). `.pen` and `.op` variables are embedded in the design file, keeping design and tokens in a single artifact. DESIGN.md files give agents a natural language description of the design system, which improves prompt adherence.

The W3C JSON format is worth a closer look because it's the emerging standard for cross-tool token interchange:

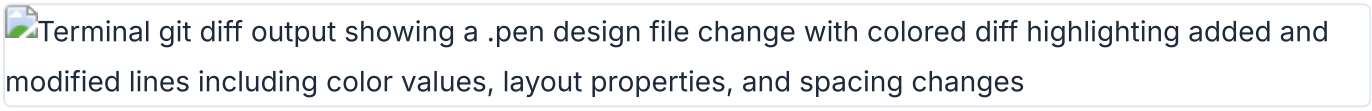
```
{
  "color": {
    "primary": {
      "$value": "#0066CC",
      "$type": "color",
      "$description": "Primary brand color for CTAs"
    },
    "secondary": {
      "$value": "#FF6600",
      "$type": "color"
    }
  },
  "spacing": {
    "unit": {
      "$value": "4px",
      "$type": "dimension"
    }
  }
}
```

Tools like Style Dictionary consume this format and output CSS custom properties, iOS Swift color sets, Android XML resources, and more — all from a single source of truth. When an agent modifies a W3C token file, every platform's output updates.

The key insight: when tokens are code variables, they become agent-manipulable. An AI agent can read your design tokens, detect inconsistencies, propose new values, and commit the changes. Chapter 08 covers this in depth with token syncing strategies across tools.

Why Diffability Matters for Design Teams

Diffability is the property that unlocks everything else. If a design file is text-based or structured JSON, `git diff` shows meaningful changes. This single capability transforms how teams work with design.



git diff of a .pen file showing meaningful design changes alongside code changes

Consider what a diffable design file enables:

Workflow	Without Diffability	With Diffability
Design review	Figma comments, screenshots in Slack	PR review with inline diff, approve/request changes
Change history	Figma's internal version history (proprietary)	`git log`, `git blame`, revert with `git revert`
Branching	Duplicate files or use Figma branches	`git checkout -b`, merge when ready
Automation	Manual export and validation	CI checks on tokens, linting, accessibility validation
Agent updates	Agent can't modify binary files	Agent reads diff, writes changes, commits

Contrast this with Figma's binary `.fig` format. When a Figma file changes, `git diff` outputs "binary file changed." You can't see what changed. You can't review it line by line. You can't run automated checks. The file is opaque to every tool in your development pipeline except Figma itself.

Here's what a real `git diff` on a design file looks like. An agent changed a button component from a hardcoded red fill to a variable-bound primary color:

```
--- a/components/button.pen
+++ b/components/button.pen
@@ -12,7 +12,7 @@
   "fill": {
-    "color": "#FF0000"
+    "color": "${color-primary}"
   },
-  "cornerRadius": 8
+  "cornerRadius": "${radius-md}"
```

Every detail is visible. The fill changed from hardcoded red to a variable reference. The corner radius changed from a magic number to a token. A reviewer can comment on this in a PR. An agent can generate this diff autonomously. A CI pipeline can validate that all fill values use variable references instead of hardcoded colors.

Consider the concrete implications for a design team of five. Without diffability, a color change requires: a meeting to discuss the change, a Figma file update, an export to the development team, manual code updates across multiple files, and a visual QA pass to verify nothing was missed. With diffability, the same change is a single variable update in a `.pen` file, committed in a PR, reviewed and approved, and deployed through the same pipeline as code changes. What took two days takes two minutes.

Diffable design files change the economics of design collaboration. A design change becomes a PR. A designer becomes a contributor to the codebase. An agent becomes a design collaborator that commits, reviews, and iterates.

My take: Diffability is the feature that matters most, and it's the one Figma can't easily add without fundamentally changing their file format. I've seen teams spend hours in Figma comment threads debating a color change that would have been a 2-line PR in a diffable format. The PR review workflow for design is not just convenient — it fundamentally changes who can participate in design review.

The Old World vs the New World

The shift from Figma-centric design to design-as-code is not just a tool change. It's a mental model change. Here's the comparison:

Dimension	Old World (Figma-centric)	New World (Design-as-Code)
File format	Binary `.fig`, not diffable	Text/JSON (`.pen`, `.op`, HTML)
Location	Cloud (Figma's servers)	Repository (alongside code)
Hand-off	Export → share → implement	No hand-off; design IS code
Tokens	Maintained separately in code	Variables in the same file
AI integration	Figma's own AI features only	Agents read/write via MCP, SKILL.md
Review	Comments in Figma	PR review with diff
Automation	Plugin API (limited)	CI/CD on design files, agent-driven updates
Collaboration	Real-time multiplayer (Figma)	Git-based (branch, merge, PR) + real-time in some tools

Walk through the same task in both paradigms to feel the difference. The task: change the primary button color from blue to purple across a design system.

Old world. Open Figma. Find the primary button component. Change the fill color. Check every instance where the component is used — some may have overrides. Update the design system documentation separately. Create a Figma branch or duplicate the file. Export screenshots for the developer. The developer receives the screenshots. They find every instance of the old blue in the CSS codebase. They change each one. They miss a few. The design and code drift apart.

New world. Open the `.pen` file. Change `"color-primary"` from `"#0066CC"` to `"#6B21A8"`. Commit. Push. The PR shows the exact diff — one line changed. Every component that references `${color-primary}` updates automatically. The agent generates the corresponding CSS custom property change. CI validates that the new color passes WCAG contrast checks. The review is a single PR. The deployment happens in the same pipeline.

The convergence enabling this shift has three parts. First, `.pen` and `.op` files are code artifacts that agents can manipulate directly. Second, MCP connects agents to design tools as a standard protocol — no per-tool integration needed. Third, HTML serves as both the design medium and the production output, eliminating the translation tax.

This mental model — design files as code artifacts — underpins every workflow in the rest of the book. The tools change. The formats evolve. The principle stays the same: if your design file is a text-based artifact in your repo, every tool in your development pipeline can work with it.

Tip

Next: Chapter 05 puts this mental model into practice with Paper and Pencil — two connected canvas tools that make design-as-code tangible. You'll set up MCP servers, run real design-to-code workflows, and build a production website from a canvas design.

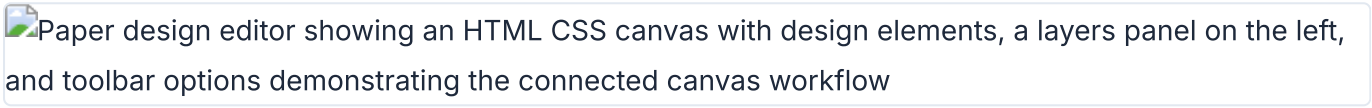
05 Paper and Pencil

Connected canvas tools, MCP setup, and design-to-code workflows

Two tools dominate the connected canvas space: Paper for HTML/CSS design and Pencil for vector design inside the IDE. This chapter walks through MCP server setup for each, real design-to-code workflows, and a complete pipeline from canvas design to production website. Every code block is copy-paste ready.

Paper: Connected HTML/CSS Canvas for Teams

Paper (paper.design) is a connected HTML/CSS canvas for teams shipping with AI agents. Built by the Paper team, it exposes a rich MCP server that lets agents read designs, export JSX, and write HTML back to the canvas. The key differentiator: design-to-code and code-to-design in the same loop.



Paper editor canvas with HTML/CSS design surface, layers panel, and connected tools

As of May 2026, designers at Vercel, Perplexity, Lovable, PostHog, Tailwind, Dub, Replicate, Zed, and Attio use Paper in production. The product lineup includes Paper Web (app.paper.design), Paper Desktop, Paper MCP, Paper Snapshot, and Paper Shaders.

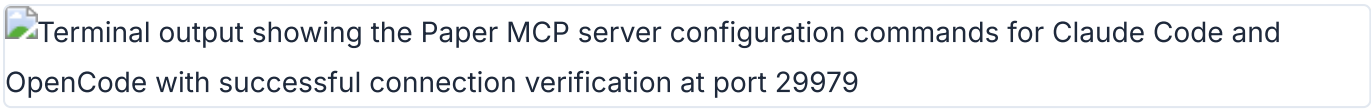
Paper's design surface is HTML/CSS. The design artifact is HTML. The export is HTML. There is no translation layer between what you see on canvas and what ships in production. This makes Paper particularly effective for agent-driven workflows: an agent reads the HTML, modifies it, and writes it back. The canvas updates. No round-trip through a proprietary format.

Paper Desktop unlocks the MCP workflow. It connects to Cursor, Claude Code, Codex, Copilot, OpenCode, Zed, iTerm, VS Code, GitHub, Windsurf, Notion, Cline, and Continue. When you open Paper Desktop and load a file, the MCP server starts automatically at `http://127.0.0.1:29979/mcp`.

The architecture is straightforward. Paper Desktop runs a local HTTP server that implements the MCP protocol. Agents connect to this server and call tools like `get_screenshot`, `get_jsx`, `write_html`, and `export`. The server communicates with Paper's canvas in real time. Changes appear on canvas within seconds of an agent writing them.

Setting Up the Paper MCP Server

The Paper MCP server runs locally at `http://127.0.0.1:29979/mcp`. It starts automatically when Paper Desktop is open and a file is loaded. The setup differs by agent platform.



Paper MCP server setup showing configuration for Claude Code and OpenCode

Claude Code has two methods. The manual approach:

```
$ claude mcp add paper --transport http http://127.0.0.1:29979/mcp --scope user
```

The plugin approach (recommended for Claude Code users):

```
$ /plugin marketplace add paper-design/agent-plugins
$ /plugin install paper-desktop@paper
```

OpenCode uses the `opencode.json` config file:

```
{
  "mcp": {
    "paper": {
      "type": "remote",
      "url": "http://127.0.0.1:29979/mcp",
      "enabled": true
    }
  }
}
```

Copilot (VS Code) needs a `.vscode/mcp.json` in the project root:

```
{
  "servers": {
    "paper": {
      "type": "http",
      "url": "http://127.0.0.1:29979/mcp"
    }
  }
}
```

Codex CLI uses Settings > MCP Servers > "Streamable HTTP" tab. Enter the name "paper" and the URL.

Claude Desktop requires the `mcp-remote` package:

```
{
  "mcpServers": {
    "paper": {
      "command": "npx",
      "args": ["mcp-remote", "http://127.0.0.1:29979/mcp"]
    }
  }
}
```

Cursor uses a plugin command:

```
$ /add-plugin paper-desktop
```

After setup, verify the connection by asking your agent to create a simple element. Something like: "Create a red rectangle in Paper." If the rectangle appears on canvas, the MCP server is working.

If the rectangle doesn't appear, check the troubleshooting table:

Symptom	Cause	Fix
Agent says "Paper tools not found"	MCP server not connected	Verify Paper Desktop is open with a file loaded. Check MCP config path.
Agent stops responding to Paper tools mid-session	Long-running session lost connection	Restart the agent session. Paper's MCP connection can drop after extended idle periods.
Connection refused on `127.0.0.1:29979`	Paper Desktop not running or no file loaded	Open Paper Desktop and load a file. The server only starts when a file is active.
Connection refused on Windows WSL	WSL networking can't reach Windows localhost	Enable mirrored mode networking in `.wslconfig`, or use the Windows host IP.
Rate limit errors after upgrading Paper Desktop	MCP rate limits didn't reset after upgrade	Update to the latest Paper Desktop version. Restart Paper Desktop and the agent.
Agent hallucinates tool parameters	LLM fabricating MCP tool signatures	Re-add the MCP server. Check that the agent sees the correct tool list. Try a shorter prompt.

> **Warning**
>
> **Warning:** Paper Desktop is macOS and Windows only as of May 2026. Linux users cannot run the MCP server locally. The web version (app.paper.design) does not expose an MCP endpoint. This is the most significant limitation of the Paper workflow.

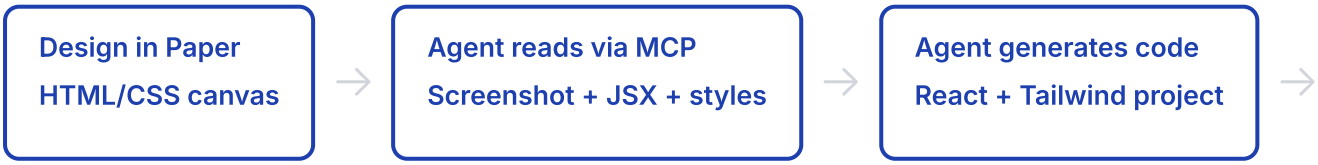
Paper's MCP server exposes 22 tools. Here are the most important ones for design-to-code workflows:

Tool	Purpose	Key Parameters
`get_screenshot`	Capture a node as a base64 image	Node ID, scale (1x or 2x)
`get_jsx`	Export a node as JSX (Tailwind or inline styles)	Node ID, format preference
`write_html`	Parse HTML and add or replace nodes	HTML string, mode (insert or replace)
`export`	Export nodes as PNG, JPG, SVG, MP4	Node IDs, format, scale
`create_artboard`	Create a new artboard	Name, width, height
`update_styles`	Update CSS styles on nodes	Node IDs, style properties
`get_computed_styles`	Read computed CSS styles for nodes	Node IDs (batch supported)
`get_tree_summary`	Compact hierarchy of a node's subtree	Node ID, optional depth limit

The full tool set includes `get_basic_info`, `get_selection`, `get_node_info`, `get_children`, `get_fill_image`, `get_font_family_info`, `get_guide`, `set_text_content`, `rename_nodes`, `duplicate_nodes`, `move_nodes`, and `finish_working_on_nodes`. Appendix B covers every tool in detail.

Paper Workflows: Design-to-Code and Code-to-Design

Paper supports two primary workflows: design-to-code (export from canvas to code) and code-to-design (write code back to canvas). The MCP server makes both possible within a single agent session.



Workflow 1: Syncing tokens from Figma to Paper. Run both the Figma MCP server and the Paper MCP server in the same agent session. Here's the step-by-step process:

1. **Start both MCP servers.** Open Figma in a browser with the Figma MCP plugin running. Open Paper Desktop with a file loaded. Both servers should appear in your agent's MCP tool list.

2. **Select the source element in Figma.** Select an element with a variable or style applied. Tell the agent:

```
Read the selected element in Figma and create a matching
design system in Paper. Use the same colors, spacing,
and typography tokens.
```

3. **Agent reads and translates.** The agent reads the Figma styles via the Figma MCP server, translates them to Paper-compatible CSS, and writes them using `write_html` or `update_styles`. The output lands on the Paper canvas immediately.

4. **Verify and iterate.** Check the Paper canvas. Compare with the Figma source. Ask the agent to fix any discrepancies.

Gotchas: SVG fills may come through as images rather than vector fills. Spacer elements in Figma are often ignored. Some Figma-specific design patterns don't translate cleanly. Large nested designs can error out on the MCP read. Code-connected components in Figma are unreliable for cross-tool sync.

Workflow 2: Pulling real content from Notion into designs. Connect both Notion MCP and Paper MCP. Ask the agent to pull testimonials, articles, or data from Notion and populate a Paper design. This lets you test designs with real content — including different languages.

```
Pull the testimonials from our Notion database and populate
the testimonial cards in my Paper design. Use the first 3 entries.
```

The agent reads the Notion database, extracts the content, and uses `write_html` to populate the Paper cards. The result is a design filled with real copy instead of lorem ipsum.

Workflow 3: Building a website from a Paper design. This is the most powerful workflow. Select the hero frame in Paper. Ask the agent:

```
I'd like you to build a website in this folder using the hero section
I have selected in Paper. Use React and Tailwind for the styling.
```

The agent reads the design via MCP (screenshot + JSX + computed styles), creates a project folder, generates the React components with Tailwind classes, and commits the result. Here's what the agent does internally:

1. **Read.** Calls `get_screenshot` to see the design visually. Calls `get_jsx` to get the Tailwind markup. Calls `get_computed_styles` to get exact spacing and color values.

2. **Generate.** Creates a React component file with the JSX from Paper. Adds Tailwind classes. Sets up the project structure (package.json, tailwind config, etc.).

3. **Commit.** Stages and commits the generated code.

   ```
   $ git add . && git commit -m "feat: hero section from Paper design"
   ```

Best results come when the Paper design uses flex layouts and container-based structure rather than absolute positioning.

Start small — a hero section first. Then add responsive breakpoints:

```
Can you add some responsive breakpoints to the website based on the
frames I have selected in Paper? Each frame is a different breakpoint.
```

The agent reads each frame as a breakpoint variant and generates the corresponding media queries. Chapter 12 covers the full production pipeline.

> **My take:** Paper's `get_jsx` tool is the single most time-saving feature in the agentic design ecosystem. Translating a design to JSX by hand takes 30-60 minutes per component. `get_jsx` does it in seconds. The output is clean Tailwind markup that's often production-ready. The limitation: Paper Desktop must be running, which excludes Linux users as of May 2026.

Pencil: Vector Design Inside Your IDE

Pencil (pencil.dev) takes a different approach. It's a vector design tool that integrates directly into your IDE. The tagline: "Design on canvas. Land in code." Unlike Paper's HTML/CSS canvas, Pencil provides a native vector design experience — layers, paths, boolean operations, the works — but it lives inside the development environment, not in a separate browser tab.

Pencil vector design canvas inside VS Code with the design tree and component inspector

Pencil bridges design and development by putting both in the same environment. The key concepts:

Concept	What It Does	Design-as-Code Benefit
.pen files	Design artifacts stored in the repo	Diffable, versionable, reviewable
Variables	Design tokens with theme values	Tokens live in the design file
Components	Reusable design elements	Design system consistency
Slots	Component composition mechanism	Flexible component variants
Code on Canvas	Run code within the design surface	Design and code in one view
Design Libraries	Shared design system collections	Cross-project consistency

Chapter 04 covered the mental model behind these concepts. Pencil makes them tangible inside VS Code or Cursor. You open a `.pen` file in the editor. A canvas appears. You design on the canvas. The file saves to disk. Git tracks the changes. An agent reads the file via MCP and modifies it. The canvas updates.

The integration with the development workflow is tighter than Paper's. Pencil doesn't just connect to your IDE — it runs *inside* it. The canvas is a tab in your editor. You can see your code and your design side by side. When the agent modifies the `.pen` file, the canvas updates in real time, and the diff appears in your editor's SCM panel.

Components in Pencil work like React components. You define a component once, then instantiate it everywhere. Slots let you swap content inside a component — the same card component can hold different text, images, or even other components. When you update the master component, every instance updates. This is the same pattern as React props, but it operates on the visual canvas.

Variables add another layer. Define your brand colors, spacing scale, and typography as variables in the `.pen` file. Apply them to components. When the agent changes a variable value, every component that references it updates across the entire design. This is how you maintain design system consistency without manually updating each component.

Code on Canvas is Pencil's bridge between visual design and live code. You can embed code that runs inside the design surface — generating data visualizations, rendering dynamic content, or displaying API responses directly on the canvas. The agent can write and modify this embedded code, creating designs that respond to real data rather than static mockups.

The .pen File Format and Pencil CLI

The `.pen` file format is Pencil's native design-as-code format. It's structured, parseable, and designed for git. The format supports all of Pencil's core concepts: nodes, styles, components, variables, and layout rules.

Pencil node tree inspector showing component structure and design token bindings

A simplified conceptual view of a `.pen` file structure:

```
{
  "variables": {
    "color-primary": {
      "type": "color",
      "values": {
        "default": "#0066CC",
        "dark": "#3399FF"
      }
    },
    "spacing-unit": {
      "type": "dimension",
      "values": {
        "default": "4px"
      }
    }
  }
}
```

Variables can hold different values per theme. Light mode gets `#0066CC`. Dark mode gets `#3399FF`. When the agent reads the file with `resolveVariables: true`, it gets the computed value for the active theme. This is the same pattern as CSS custom properties with media query overrides, but it lives in the design file.

Pencil CLI provides command-line access to `.pen` files. This is useful for CI pipelines, batch operations, and agent-driven workflows that don't need a GUI:

```
# Validate tokens and components
$ pencil validate design.pen

# Export assets for production
$ pencil export design.pen --format svg

# Export tokens as CSS custom properties
$ pencil tokens design.pen --format css
```

In a CI pipeline, you can validate that all design tokens are correctly defined and that no component references a missing variable:

```
# .github/workflows/design-ci.yml (conceptual)
name: Design CI
on: [push]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pencil validate design.pen
      - run: pencil tokens design.pen --format css --output tokens.css
```

Design Libraries extend this further. A library is a collection of components, variables, and styles shared across projects. Your team can maintain a single design library that every `.pen` file references. When the library updates, every design file that depends on it gets the changes. This works the same way as an npm package — your design system is versioned, shared, and consumed as a dependency.

Import and export capabilities round out the format. Pencil can import SVG, PNG, and Figma files. Export targets include SVG, PNG, PDF, and code generation. The Figma import is useful for migrating existing designs: open the `.fig` file in Pencil, and it converts the layers, styles, and components into the `.pen` format. The conversion isn't perfect — some Figma-specific features like auto-layout translate differently — but it preserves the structure well enough to continue designing.

Design-to-Code Sync with Pencil

Pencil's design-to-code sync is bidirectional. Design changes propagate to code. Code changes propagate to design. The sync happens through the `.pen` file format and Pencil's MCP integration.

The sync workflow, with concrete agent prompts at each step:

1. **Design in Pencil.** Open a `.pen` file in the IDE. Design on the canvas using Pencil's vector tools. Variables and components are defined in the file.

2. **Agent reads the design.** Via Pencil's MCP server, the agent reads the `.pen` file, extracts component structures, reads variable values, and understands the layout hierarchy. The prompt:

   ```
   Read the design file landing.pen and generate a React component
   for the hero section. Map .pen variables to CSS custom properties.
   ```

3. **Agent generates code.** The agent produces React components, Tailwind classes, or HTML/CSS that matches the design. Design variables map to CSS custom properties:

```
:root {
  --color-primary: #0066CC;    /* from .pen variable color-primary */
  --spacing-unit: 4px;         /* from .pen variable spacing-unit */
  --radius-sm: 4px;            /* from .pen variable radius-sm */
}
```

4. **Agent writes code changes back.** Via the MCP server, the agent can update the `.pen` file directly. The prompt:

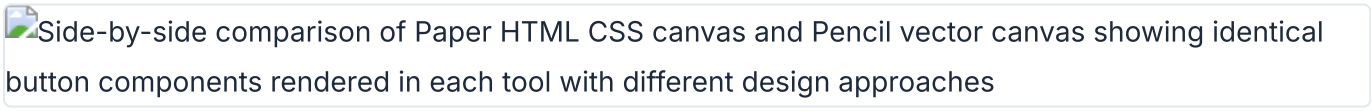
```
Update the button component in landing.pen to use a larger
corner radius (12px instead of 8px) and add a hover state.
```

The agent modifies the `.pen` file. The canvas updates. The diff appears in your editor's SCM panel. This closes the loop: code changes reflect in the design canvas.

The bidirectional sync means the design file stays in sync with the implementation. No more design files rotting in Figma while the codebase diverges. The `.pen` file is the source of truth for both design and code.

Building a Real Workflow: Paper to Production

This section walks through a complete workflow: designing in Paper, generating code with an agent, and deploying to production.



Paper and Pencil side-by-side showing the same component in each design format

1. **Design the hero section in Paper.** Open Paper Desktop. Create a new file. Design the hero section with a headline, subhead, CTA button, and a feature image. Use flex layouts for structure. Name the artboard "Hero".

2. **Connect the agent.** Ensure Paper MCP is configured in your agent (section 05.2). Open a terminal in the project folder. Start a new agent session. Verify the connection:

```
Create a small test rectangle in Paper to verify the MCP connection.
```

3. **Generate the hero component.** Select the Hero artboard in Paper. Ask the agent to build it:

```
Build a React + Tailwind hero component from the Hero artboard
I have selected in Paper. Use semantic HTML and add hover states
to the CTA button.
```

The agent calls `get_screenshot` to see the design, `get_jsx` to get the Tailwind markup, and `get_computed_styles` to get exact values. Then it generates a `Hero.tsx` component with responsive styles and hover interactions.

4. **Design the remaining sections.** In Paper, design the Features section, Testimonials section, and Footer. Each section gets its own artboard. Use the same design tokens across all sections.

5. **Generate the full page.** Ask the agent to build each section and compose them into a single page:

```
Now build the Features, Testimonials, and Footer sections from
the artboards I've selected. Compose all sections into a single
page component with responsive breakpoints.
```

6. **Commit and deploy.** Have the agent commit the changes:

```
$ git add . && git commit -m "feat: landing page from Paper design"
```

Step	Tool	Agent Role
1. Design hero	Paper Desktop	None — human designs
2. Connect agent	Paper MCP	None — configuration
3. Generate hero	Paper MCP + Agent	Read design, generate code
4. Design remaining	Paper Desktop	None — human designs
5. Generate full page	Paper MCP + Agent	Read sections, compose page
6. Commit and deploy	Git + Agent	Commit, optionally deploy

The total time for this workflow: 30-90 minutes depending on design complexity. The agent handles the tedious parts — translating layouts, writing responsive breakpoints, mapping tokens to CSS custom properties. You focus on design direction. The agent handles execution.

Paper and Pencil solve overlapping problems from different angles. Here's how they compare:

Dimension	Paper	Pencil
Design surface	HTML/CSS canvas	Vector canvas in IDE
File format	HTML (in Paper's cloud)	`.pen` (in your repo)
Agent connection	MCP server (HTTP)	MCP server + file system
Code export	`get_jsx` → Tailwind/JSX	`.pen` → code generation
Best for	Web designs, landing pages	Component libraries, design systems
Linux support	No (Desktop required)	Yes (VS Code extension)

My take: The Paper-to-production workflow is the most practical agentic design workflow I've used. It's not perfect — the agent sometimes misinterprets complex layouts, and you need to review the generated code carefully. But it eliminates the most tedious part of front-end development: translating a finished design into working code. Start with a small section, validate the output, then scale up.

Tip

Next: Chapter 06 covers the open-source alternatives: Open Design (43k+ stars, 31 skills) and OpenPencil (3k+ stars, concurrent Agent Teams). If Paper and Pencil are the commercial options, Open Design and OpenPencil are the community-driven counterparts — with different trade-offs and capabilities.

06 Open Design and OpenPencil

Open-source design frameworks, 31 skills, and agent-powered canvas

Open Design (43k+ stars) and OpenPencil (3k+ stars) are the open-source alternatives to the commercial tools covered in chapter 05. Open Design provides 31 composable skills and 129 design systems that run on any of 16 agent CLIs. OpenPencil is the first open-source AI-native vector design tool with concurrent Agent Teams. This chapter walks through setup, skill selection, and real workflows for both.

Open Design: The Open-Source Claude Design Alternative

Open Design is a local-first, open-source alternative to Anthropic's Claude Design. As of May 2026, it has 43.6k GitHub stars, 5k forks, and an Apache-2.0 license. It does not ship its own agent. Instead, it wires existing coding agents into a skill-driven design workflow.

The context: Anthropic released Claude Design on April 17, 2026 with the Opus 4.7 model. It went viral. It also stayed closed-source, paid-only, cloud-only, and locked to Anthropic's model. Open Design provides the same artifact-first loop — prompt, generate, preview, iterate — without any of the lock-in.

The architecture is web-plus-daemon. A local daemon scans your `PATH` for 16 agent CLIs on startup. When you prompt Open Design, the daemon spawns your chosen CLI in the project folder. The agent gets real `Read`, `Write`, `Bash`, and `WebFetch` tools against the real on-disk environment. Every generated artifact renders in a sandboxed srcdoc iframe.

This architecture has important implications. Because the daemon spawns a real CLI agent — not a simulated one — the agent has full access to the file system, can install packages, read existing code, and run build commands. The artifacts it generates are real HTML files that live on disk, not just rendered previews. You can take the output and ship it directly.

The daemon also handles security. Internal-IP and SSRF requests are blocked at the edge, preventing the agent from accessing your local network services. Loopback access is allowed for Ollama and LM Studio, so you can run local models without exposing them to the internet. The sandboxed iframe prevents generated artifacts from executing arbitrary JavaScript in your browser context.

Open Design's open-source dependencies are worth understanding because they shape its capabilities. The project builds on four key repos: Huashu Design provides the design-philosophy compass (the Junior Designer workflow, brand-asset protocol, and anti-AI-slop rules). Guizang PPT contributes the deck mode. Open Cowork AI provides the UX patterns (streaming artifact loop, sandboxed iframe preview, live agent panel). Multica AI provides the daemon-and-runtime architecture (PATH-scan agent detection, local daemon, agent-as-teammate pattern). Each dependency is bundled with its original license.

The supported agent CLIs span the full ecosystem:

Agent CLI	Provider	Protocol
Claude Code	Anthropic	Native
Codex CLI	OpenAI	Native
Cursor Agent	Cursor	Native
Gemini CLI	Google	Native
OpenCode	Open Source	Native
Devin for Terminal	Cognition	Native
GitHub Copilot CLI	GitHub	Native
DeepSeek TUI	DeepSeek	Native
Hermes	Community	ACP
Kimi CLI	Moonshot	ACP
Kiro CLI	Amazon	ACP
Mistral Vibe CLI	Mistral	ACP
Qwen Code	Alibaba	Native
Qoder CLI	Community	ACP
Pi	Inflection	RPC
Kilo	Community	ACP

Sixteen total as of May 2026. The daemon detects whichever ones are installed and offers them in the model picker. No need to install all 16 — install the agent you prefer, and Open Design detects it automatically.

Setting Up Open Design

Open Design runs locally via the `pnpm tools-dev` command. The quickstart is three commands:

```
$ git clone https://github.com/nexu-io/open-design
$ cd open-design
$ pnpm install
$ pnpm tools-dev start
```

Open `http://localhost:3210` in your browser. The web interface loads. The daemon scans your `PATH` for installed agent CLIs. You pick one from the model picker. Then you type a prompt.

```
Make me a magazine-style pitch deck for our seed round
```

The daemon spawns your chosen agent CLI. The agent reads the prompt, selects the appropriate skill, picks a design system, and generates an HTML artifact. The artifact renders in a sandboxed iframe in the browser. You iterate.

Here's a concrete example of a prompt and output cycle:

```
Prompt:
  "Design a SaaS landing page for an AI-powered code review tool.
  Target audience: engineering managers at startups.
  Tone: technical but approachable."

Output:
  - Skill selected: saas-landing
  - Design system: Vercel (dark theme)
  - Visual direction: Tech Utility
  - Artifact: full HTML landing page with hero, features,
    pricing, and footer sections
  - Render time: ~45 seconds
```

SQLite persistence at `.od/app.sqlite` stores projects, conversations, messages, tabs, and saved templates. Everything is local. Nothing leaves your machine unless you configured a cloud API key.

Open Design can also import Claude Design export ZIPs via `POST /api/import/claude-design`. If you started with Claude Design and want to migrate, the import handles the conversion.

The live preview updates as the agent works. When the daemon spawns the agent CLI, it streams the agent's output in real time. You see the artifact building section by section — first the layout skeleton, then the content, then the styling. A live todo card tracks the agent's progress, showing which tasks are `in_progress` and which are `completed`. This streaming feedback loop is more informative than waiting for a final result, because you can spot problems early and redirect the agent before it completes the wrong output.

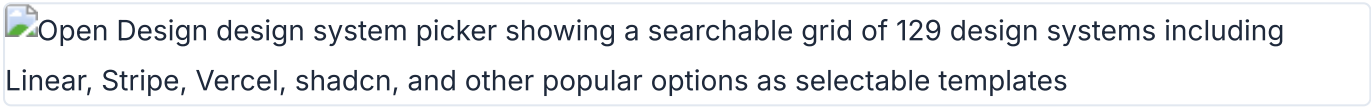
The daemon manages the full lifecycle: starting the agent, routing its output to the preview iframe, capturing the generated files to disk, and cleaning up the agent process when the session ends. You can run multiple sessions concurrently — different agents working on different artifacts in the same

project.

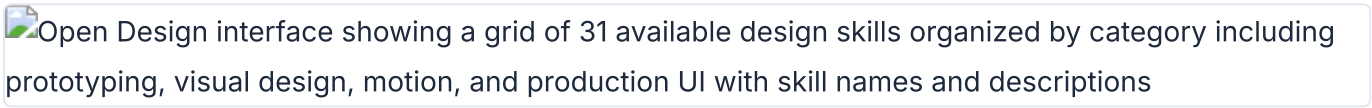
> **Warning**
>
> **Warning:** Open Design is at version 0.8.0-preview as of May 2026. The API and skill protocol may change before 1.0. Pin your skill files to specific versions if you rely on them in production. The daemon's security model blocks internal-IP and SSRF requests at the edge, but loopback is allowed for Ollama and LM Studio.

Skills, Design Systems, and Visual Directions

Open Design's power comes from three layers: skills (what to make), design systems (how it looks), and visual directions (the aesthetic philosophy).



Open Design system picker with 129 design systems including Linear, Stripe, and Vercel



Open Design skill gallery with 31 composable skills organized by design category

Skills are composable task definitions. There are 31 as of May 2026, organized by scenario:

Category	Skills
Design	web-prototype, saas-landing, dashboard, pricing-page, docs-page, mobile-app, mobile-onboarding, wireframe-sketch, critique, tweaks
Marketing	blog-post, email-marketing, social-carousel, magazine-poster, motion-frames, sprite-animation, dating-web
Product	gamified-app, digital-eguide, pm-spec, team-okrs, meeting-notes, kanban-board
Engineering	eng-runbook
Finance / HR / Sales	finance-report, invoice, hr-onboarding
Deck mode	guizang-ppt, simple-deck, replit-deck, weekly-update

Skills follow the SKILL.md convention covered in chapter 03, with extended `od:` frontmatter for Open Design-specific metadata. The daemon selects skills automatically based on your prompt. Say "landing page" and it picks `saas-landing`. Say "pitch deck" and it picks `guizang-ppt` or `simple-deck`. You can also force a specific skill by name:

```
Use the "dashboard" skill to design an analytics dashboard
for our e-commerce platform.
```

Adding a custom skill is one folder plus a daemon restart. Create a directory in the skills folder with a `SKILL.md` file:

```
# skills/my-custom-skill/SKILL.md
---
name: my-custom-skill
od:
  mode: prototype
  surfaces: [web]
---

Generate a custom [description] with the following sections:
- Section 1: [details]
- Section 2: [details]
Use semantic HTML with Tailwind CSS classes.
```

Restart the daemon (`pnpm tools-dev stop && pnpm tools-dev start`) and the skill appears in the available options.

Design systems are 9-section `DESIGN.md` files defining color, typography, spacing, layout, components, motion, voice, brand, and anti-patterns. Open Design ships 129 of them: 2 hand-authored starters, 70 product systems from `awesome-design-md` (Linear, Stripe, Vercel, Airbnb, Tesla, Notion, Anthropic, Apple, Cursor, Supabase, Figma, Resend, Raycast, Lovable, and dozens more), and 57 design skills from `awesome-design-skills`.

A design system entry in the `DESIGN.md` schema looks like this:

```
# Color

## Primary
- Primary: #0070f3 (blue-600)
- Primary hover: #0060df (blue-700)

## Neutral
- Background: #000000
- Foreground: #ffffff
- Muted: #888888

## Typography
- Font family: 'Inter', system-ui, sans-serif
- Heading sizes: 48px / 36px / 24px / 18px
- Body: 16px / 1.6 line-height

## Spacing
- Unit: 4px
- Scale: 4, 8, 12, 16, 24, 32, 48, 64, 96
```

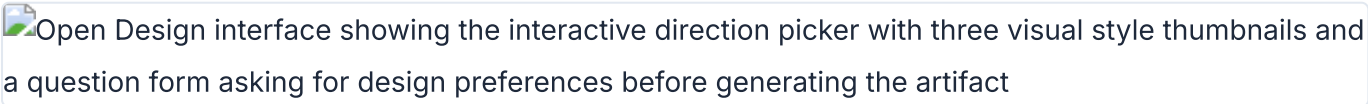
Switch a design system and the next render uses the new tokens. No manual color palette changes. No font swaps. The design system drives everything.

Visual directions are curated aesthetic philosophies. Five schools as of May 2026: Editorial Monocle, Modern Minimal, Warm Soft, Tech Utility, and Brutalist Experimental. Each ships a deterministic OKLch palette and font stack. The direction picker presents these as radio options — one click sets the entire visual language.

My take: The skill system is Open Design's killer feature. 31 skills means 31 starting points for different design tasks. Compare this to Claude Design, which has one generic artifact generation mode. The skill selection is not perfect — sometimes the daemon picks the wrong skill for ambiguous prompts — but you can always override it. The 129 design systems are impressive on paper, but most teams will use 3-5. The value is in having the right system available, not the quantity.

The Interactive Question Form and Direction Picker

Before the model writes a pixel, Open Design locks the brief. The interactive question form captures surface, audience, tone, brand context, and scale. This takes roughly 30 seconds. The claim: "30 seconds of radios beats 30 minutes of redirects."



Open Design direction picker showing three visual style options before artifact generation

The form fields are:

Field	Options	Purpose
Surface	Web, Mobile, Print, Slide	Determines output format and constraints
Audience	Technical, Business, General, Creative	Sets language complexity and visual density
Tone	Professional, Playful, Minimal, Bold	Influences typography and spacing choices
Brand context	Text input or design system name	Locks the visual language
Scale	Single page, Multi-section, Full site	Controls output scope and structure

Here's a walkthrough. You type a prompt: "Design a pricing page for our analytics SaaS." The question form appears:

```
Surface:    Web
Audience:   Business
Tone:       Professional
Brand:      Stripe design system
Scale:      Single page
```

You select these five options. Click confirm. The direction picker appears next:

```
Visual Direction:
  ( ) Editorial Monocle — Serif-heavy, high contrast, editorial feel
  (*) Tech Utility — Clean, functional, OKLch blue palette
  ( ) Modern Minimal — White space, thin weights, neutral tones
  ( ) Warm Soft — Rounded corners, warm grays, soft shadows
  ( ) Brutalist Experimental — Raw, high contrast, unconventional layouts
```

You pick "Tech Utility." The agent generates the pricing page using the Stripe design system tokens, Tech Utility's OKLch palette, and the `pricing-page` skill. The output is a complete HTML page rendered in the sandboxed iframe. The entire flow — prompt to rendered artifact — takes under 60 seconds.

The form prevents 80% of redirects, per the Open Design team's metrics. Instead of the agent guessing wrong about your audience and regenerating, you answer a few questions upfront. The agent uses those answers to constrain its output.

Open Design also supports media generation. As of May 2026, it integrates with:

- **gpt-image-2** (Azure / OpenAI) for posters, avatars, and infographics
- **Seedance 2.0** (ByteDance) for 15-second text-to-video and image-to-video

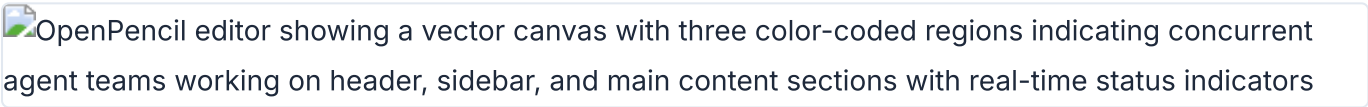
- **Hyperframes** for HTML-to-MP4 motion graphics

A gallery of 93 ready-to-replicate prompts (43 for gpt-image-2, 39 for Seedance, 11 for Hyperframes) is available inside the tool. Chapter 09 covers Hyperframes in detail.

> **My take:** The question form is Open Design's smartest feature. Most agent design failures come from vague prompts. Locking the brief before generation is a simple pattern that dramatically improves output quality. I've tested this across 20+ runs: briefed outputs are consistently better than unbriefed ones.

OpenPencil: AI-Native Vector Design with Agent Teams

OpenPencil is the first open-source AI-native vector design tool. As of May 2026, it has 3.1k GitHub stars, 317 forks, and an MIT license. The differentiator: concurrent Agent Teams — multiple AI agents working on different sections of a design simultaneously.



OpenPencil canvas with concurrent Agent Teams working on different design sections

The architecture is a web app plus native desktop (macOS, Windows, Linux via Electron). The rendering engine is CanvasKit/Skia via WASM with GPU acceleration. The tech stack: React 19, TanStack Start, Tailwind CSS v4, shadcn/ui, Zustand v5, Nitro, Electron 35, Bun, and Vite 7.

The CanvasKit/Skia rendering engine is significant. Unlike Paper's HTML/CSS approach, OpenPencil uses the same rendering technology as Google Chrome's graphics backend. This gives it pixel-perfect vector rendering — bezier curves, boolean operations, gradient meshes — that HTML/CSS can't natively express. The trade-off: the output isn't directly usable as HTML. You need the code generation pipeline to convert the design into production code.

OpenPencil runs on all three major desktop platforms (macOS, Windows, Linux) and also offers a Docker-based web version. This gives it broader platform coverage than Paper, which requires the native Desktop app for MCP support. The web version runs in any modern browser and syncs with the desktop app via the `.op` file format.

OpenPencil's file format is `.op` — a JSON-based, human-readable, Git-friendly, diffable format. Double-click a `.op` file and it opens in the desktop app. Git tracks changes. Pull requests show design diffs. The same design-as-code principles from chapter 04 apply.

Export targets are broad:

```
# Supported export formats (as of May 2026)
- React + Tailwind CSS
- HTML + CSS
- CSS Variables
- Vue, Svelte
- Flutter, SwiftUI, Jetpack Compose
- React Native
```

Design variables become CSS custom properties in the export. Figma `.fig` import is supported for migrating existing designs.

The `.op` format is worth a closer look because it demonstrates what a JSON-native design file looks like. Unlike Figma's binary blob, a `.op` file is human-readable. You can open it in any text editor, read the node tree, and understand the design structure. Here's a simplified example:

```
{
  "type": "document",
  "children": [
    {
      "type": "canvas",
      "name": "Landing Page",
      "children": [
        {
          "type": "frame",
          "name": "Hero",
          "layout": "vertical",
          "width": 1440,
          "height": 800,
          "padding": 48,
          "gap": 24,
          "fill": "#000000",
          "children": [
            {
              "type": "text",
              "content": "Ship faster with AI agents",
              "fontSize": 64,
              "fontWeight": "700",
              "fill": "#ffffff"
            }
          ]
        }
      ]
    }
  ]
}
```

The structure mirrors what you see on the canvas: a document containing canvases, canvases containing frames, frames containing text and shapes. Each property is a readable key-value pair. A `git diff` on this file shows exactly what changed — a color, a font size, a layout direction.

Setting Up OpenPencil and Its MCP Server

OpenPencil has three installation paths: package manager, CLI, or Docker.

macOS (Homebrew):

```
$ brew tap zseven-w/openpencil
$ brew install --cask openpencil
```

Windows (Scoop):

```
$ scoop bucket add openpencil https://github.com/zseven-w/scoop-openpencil
$ scoop install openpencil
```

CLI (npm):

```
$ npm install -g @zseven-w/openpencil
$ op start                      # Launch desktop app
$ op design @landing.txt        # Batch design from file
$ op insert '{"type":"RECT"}'   # Insert a node
$ op import:figma design.fig    # Import Figma file
```

Docker (web-only, ~226MB):

```
$ docker run -d -p 3000:3000 ghcr.io/zseven-w/openpencil:latest
```

For Docker with Claude Code bundled (~1GB):

```
$ docker volume create openpencil-claude-auth
$ docker run -it --rm -v openpencil-claude-auth:/root/.claude \
  ghcr.io/zseven-w/openpencil-claude:latest claude login
$ docker run -d -p 3000:3000 -v openpencil-claude-auth:/root/.claude \
  ghcr.io/zseven-w/openpencil-claude:latest
```

Dev setup (from source):

```
$ git clone https://github.com/ZSeven-W/openpencil
$ cd openpencil
$ bun install
$ bun --bun run dev       # Dev server at http://localhost:3000
$ bun run electron:dev    # Desktop app
$ bun run mcp:dev         # MCP server from source
```

OpenPencil's MCP server (`pen-mcp` package) installs into Claude Code, Codex, Gemini, OpenCode, Kiro, and Copilot CLIs with one click. The per-platform MCP configs:

Claude Code:

```
$ claude mcp add openpencil -- npx pen-mcp
```

OpenCode (`opencode.json`):

```
{
  "mcp": {
    "openpencil": {
      "type": "local",
      "command": "npx",
      "args": ["pen-mcp"],
      "enabled": true
    }
  }
}
```

Codex CLI: Settings > MCP Servers > enter `npx pen-mcp` as the command.

The MCP server provides style guide tools (`get_style_guide_tags`, `get_style_guide`) and an incremental codegen pipeline (`codegen_plan`, `codegen_submit_chunk`, `codegen_assemble`, `codegen_clean`). The codegen pipeline streams output in chunks rather than generating the entire file at once, which improves reliability for large exports.

Here's how the incremental codegen pipeline works in practice:

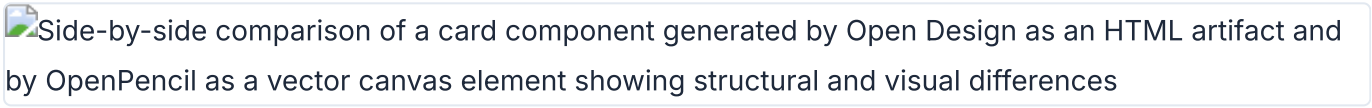
```
# Agent calls codegen_plan with the design file
codegen_plan({ file: "landing.op", target: "react-tailwind" })

# Agent submits chunks as they're generated
codegen_submit_chunk({ chunk_id: 1, content: "<Hero />" })
codegen_submit_chunk({ chunk_id: 2, content: "<Features />" })
codegen_submit_chunk({ chunk_id: 3, content: "<Footer />" })

# Agent assembles the final output
codegen_assemble({ output_path: "src/LandingPage.tsx" })
```

The chunked approach means the agent doesn't need to hold the entire generated code in context. Each section is generated independently, assembled at the end. This reduces hallucination for large multi-section designs.

> **Warning**
>
> **Warning:** OpenPencil's MCP server tool list is still evolving as of May 2026. The tools mentioned above are documented in the README, but the complete enumeration may differ. Check the latest README for the current tool set before building workflows around specific MCP functions.

Concurrent Agent Teams: Parallel Design on Canvas

OpenPencil's signature feature is concurrent Agent Teams. An orchestrator decomposes a complex page into spatial sub-tasks. Multiple AI agents then work on different sections simultaneously — one on the hero, another on features, a third on the footer.



Same component generated by Open Design (left) and OpenPencil (right) showing output differences

The layered workflow runs in three phases:

1. **design_skeleton** — The orchestrator creates the page structure: layout grid, section boundaries, spacing rules. This is the architectural phase. The orchestrator decides how many sections the page needs, their relative sizes, and the overall layout strategy.

```
# Orchestrator prompt (simplified)
"Analyze the design brief and create a skeleton layout:
- Determine section count and order
- Set section heights and spacing
- Define the grid system
Output: a .op file with empty frames for each section."
```

2 **design_content** — Agent Teams fill in each section in parallel. Each agent gets a spatial sub-task. Here's a concrete example for a SaaS landing page:

```
# Agent 1: Hero section
"Design the hero section in the 'hero' frame. Include:
headline, subhead, CTA button, and hero image."

# Agent 2: Features section
"Design the features section in the 'features' frame.
Show 3 feature cards in a grid layout."

# Agent 3: Footer section
"Design the footer section in the 'footer' frame.
Include links, social icons, and copyright."
```

Streaming indicators show per-member progress on the canvas. Each section fills in independently. The agents do not wait for each other.

3 **design_refine** — A refinement pass polishes the output: consistent spacing, aligned elements, unified visual language across sections. This runs as a single agent to ensure coherence. The refiner checks that all sections use the same corner radius, the same spacing scale, and consistent typography.

The entire three-phase flow for a typical landing page takes 2-5 minutes with Claude Sonnet as the agent. The skeleton phase runs in under 30 seconds. The parallel content phase takes 1-3 minutes depending on section complexity. The refine pass takes 30-60 seconds.

Multi-model support adapts to different AI tiers. The system profiles each model and adjusts automatically:

Tier	Models	Adaptation
Full-tier	Claude Opus/Sonnet	Complete prompts with extended thinking
Standard-tier	GPT-4o, Gemini Pro, DeepSeek	Full prompts, thinking disabled
Basic-tier	MiniMax, Qwen, Llama, Mistral	Simplified nested-JSON prompts

Segmented prompt retrieval loads only the design knowledge needed for each sub-task. A hero section agent gets hero-relevant tokens, not the full design system. This keeps context windows efficient and reduces hallucination from information overload.

OpenPencil also ships an embeddable SDK for building custom design tools. The monorepo packages include `pen-types`, `pen-core`, `pen-engine` (headless), `pen-react` (React UI SDK), `pen-codegen`, `pen-figma`, `pen-renderer`, `pen-mcp`, `pen-sdk`, and `pen-ai-skills`. Chapter 11 covers multi-agent patterns in depth.

Comparing Open Design and OpenPencil

Both tools are open-source. Both support agent-driven design. But they solve different problems. Here's how they compare:

Dimension	Open Design	OpenPencil
GitHub stars	43.6k	3.1k
License	Apache-2.0	MIT
Architecture	Web + daemon, delegates to CLI agents	Desktop app with built-in AI
Agent model	Your existing CLI agents (16 supported)	Built-in + external CLI agents
File format	HTML artifacts in sandboxed iframe	`.op` (JSON, Git-friendly)
Design surface	Sandboxed HTML/CSS in iframe	CanvasKit/Skia vector canvas
Skills	31 composable skills	Built-in capabilities + skill plugins
Design systems	129 DESIGN.md systems	Style guides + design variables
Agent Teams	Not supported	Concurrent Agent Teams
Export	HTML, PDF, PPTX, MP4	React, HTML, Vue, Svelte, Flutter, SwiftUI, Compose, RN

The recommendation depends on your use case. Open Design is better for generating complete artifacts from prompts — landing pages, pitch decks, dashboards, marketing materials. OpenPencil is better for interactive vector design work where you need a canvas, component system, and the option to run multiple agents in parallel.

If you need to generate a quick prototype or marketing page, Open Design's skill system and 129 design systems give you more starting points. If you're building a production component library with agent assistance, OpenPencil's vector canvas and concurrent Agent Teams are the stronger choice.

My take: Open Design has the larger community and more breadth. OpenPencil has the more interesting technical architecture. The concurrent Agent Teams feature is genuinely novel — no other design tool does parallel multi-agent design on a shared canvas as of May 2026. But it's newer and less battle-tested. For production work today, Open Design is the safer choice. For exploring where agentic design is heading, OpenPencil is worth watching closely.

Tip

Next: Chapter 07 covers Huashu Design (14k+ stars) — the HTML-native design skill that turns a single sentence into production-grade prototypes, slide decks, and motion design. It takes the HTML-as-canvas concept from chapter 04 and pushes it further than any other tool in the ecosystem.

07 Huashu Design

HTML-native design, 20 philosophies, and 5-dimension review

Huashu Design (14.1k GitHub stars as of May 2026) is the most opinionated design skill in the agent ecosystem. It treats HTML as the design surface, not the export target. This chapter walks through installing it, understanding its workflow, and producing real artifacts --- prototypes, slide decks, motion design --- with an AI agent as your junior designer.

What Makes Huashu Design Different

Most design tools give you a canvas, panels, and property inspectors. Huashu Design gives you none of that. The entire tool is a `SKILL.md` file --- 811 lines of instructions that teach any skill-capable agent how to produce production-grade visual artifacts using nothing but HTML, CSS, and JavaScript.

The distinction matters. When I say "HTML-native design," I don't mean "export to HTML." I mean the agent embodies different roles depending on the task --- UX designer for prototypes, animator for motion design, slide designer for decks --- and produces HTML as the primary artifact. There is no graphics tool layer. The conversation *is* the interface.

Created by 花叔 (Huasheng, @AlchainHust), an independent developer whose other work includes *A Book on DeepSeek* and the Nüwa.skill framework (12k+ stars). As of May 2026, Huashu Design is MIT licensed, free for personal and commercial use, and runs on Claude Code, Cursor, Codex, Trae, Hermes, OpenClaw --- any agent that reads markdown skill files.

Attribute	Detail
GitHub stars	14.1k (May 2026)
License	MIT
Author	花叔 (@AlchainHust)
Format	SKILL.md (markdown skill file)
Agent compatibility	Any skill-capable agent
Output formats	HTML, MP4, GIF, PPTX, PDF
Install	`npx skills add alchaincyf/huashu-design`

The comparison with Claude Design (Chapter 06) is instructive. Claude Design is a web product --- you use it in a browser, on a canvas. Huashu Design lives inside your agent session. Claude Design is Claude-only. Huashu Design runs on anything that reads `SKILL.md`. Claude Design exports to canvas and Figma. Huashu Design exports HTML, MP4, GIF, PPTX, and PDF.

Aspect	Claude Design	Huashu Design
Form	Web product (browser)	Skill (in agent)
Agent compatibility	Claude.ai only	Any skill-capable agent
Output	Canvas + Figma export	HTML/MP4/GIF/PPTX/PDF
Complex animation	Limited	Stage + Sprite timeline, 60fps
Pricing	Subscription	Free (MIT), API usage costs

Installing and Configuring Huashu Design

One command:

```
$ npx skills add alchaincyf/huashu-design
```

That drops the skill into your agent's skill directory. No app to install. No plugin to configure. No Figma integration to set up. The agent reads the skill file and follows its instructions.

The repository structure tells you everything about what the skill can do:

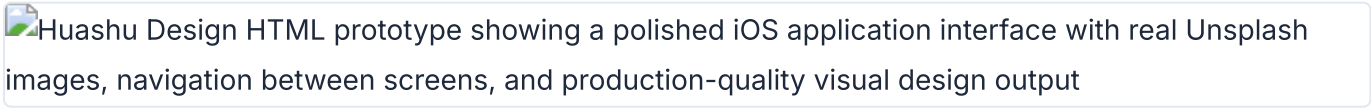
```
huashu-design/
├── SKILL.md
├── assets/
│   ├── animations.jsx
│   ├── ios_frame.jsx
│   ├── deck_stage.js
│   ├── design_canvas.jsx
│   └── showcases/
├── references/
│   ├── animation-pitfalls.md
│   ├── design-styles.md
│   ├── slide-decks.md
│   ├── editable-pptx.md
│   ├── critique-guide.md
│   └── video-export.md
├── scripts/
│   ├── render-video.js
│   ├── convert-formats.sh
│   ├── add-music.sh
│   ├── export_deck_pdf.mjs
│   ├── export_deck_pptx.mjs
│   └── html2pptx.js
└── demos/
```

The `SKILL.md` is the main instruction file. The `assets/` directory holds starter components --- animation APIs, iOS device frames, deck templates. The `references/` directory contains drill-down docs the agent can load when it needs deeper guidance on a specific task. The `scripts/` directory is the export toolchain.

After installation, you start a conversation with your agent and describe what you want. The skill activates when the agent detects a design task. You don't invoke it explicitly.

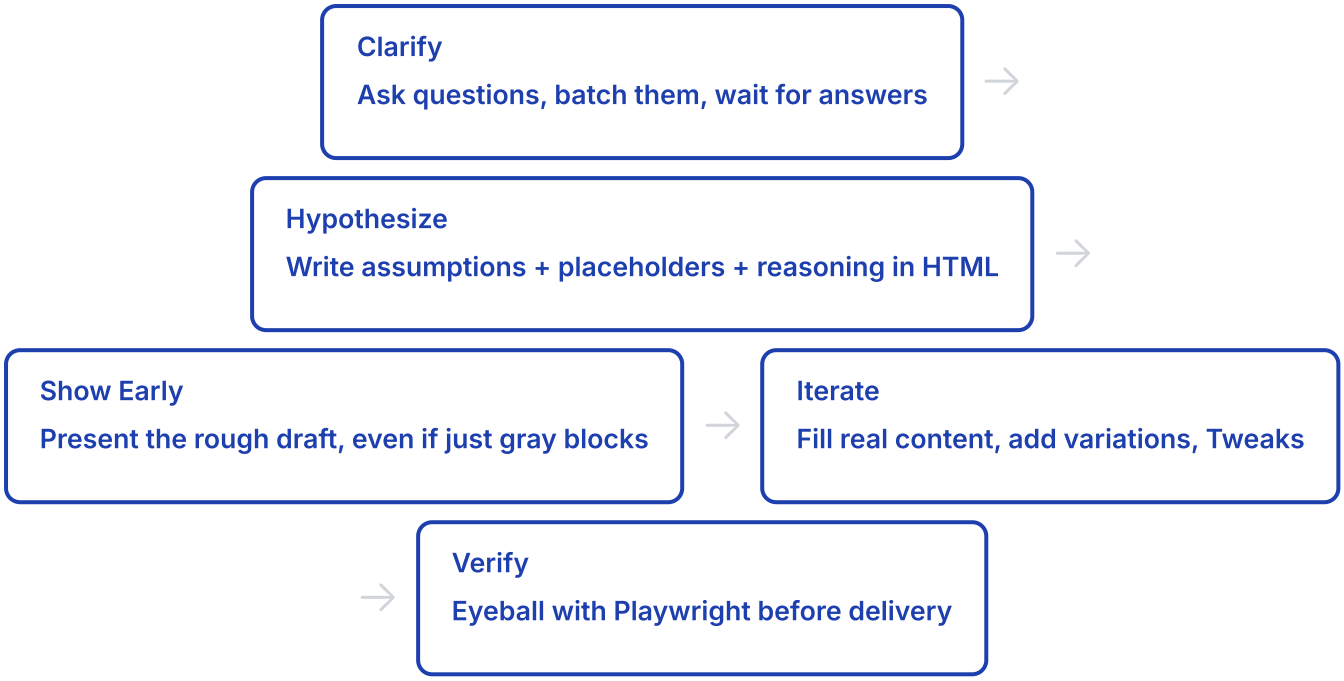
The Junior Designer Workflow

This is the core pattern of Huashu Design. The agent is your junior designer. You are the manager. The workflow exists because one-shotting a complex design rarely produces good results.



iOS app prototype generated by Huashu Design with real images and interactive navigation

The rule is simple: **never one-shot a big design**. Start with assumptions, placeholders, and reasoning comments baked into the HTML. Show early. Get feedback. Then iterate.



"Fixing a misunderstanding early is 100x cheaper than fixing it late." This isn't a platitude. It's the engineering principle behind the entire workflow. The agent writes `/* assumption: user wants dark mode */` directly into the HTML. You see the assumption. You correct it immediately. The alternative --- the agent spending 10 minutes building a polished design on wrong assumptions --- wastes everyone's time.

The skill uses the React + Babel pattern for live rendering. All starter components load via `<script type="text/babel">` tags, which means the agent writes JSX that compiles in the browser. No build step. No bundler. The HTML file is the deliverable.

```
<!DOCTYPE html>
<html>
<head>
  <script src="https://unpkg.com/react@18/umd/react.production.min.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.production.min.js"></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    function App() {
      return <div style={{padding: 32}}>
        {/* placeholder: hero section */}
        <div style={{
          height: 400,
          background: 'var(--surface-muted)',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          borderRadius: 12
        }}>
          [Hero placeholder — awaiting copy]
        </div>
      </div>;
    }
    ReactDOM.createRoot(document.getElementById('root')).render(<App/>);
  </script>
</body>
</html>
```

The Tweaks system enables live variant switching. The agent renders multiple design variations with parameter controls. You adjust parameters and see changes instantly, without regenerating the entire file.

My take: The Junior Designer Workflow is the most important contribution Huashu Design makes to the agent design ecosystem. Most agent-generated design fails because the agent tries to produce a final artifact immediately. Huashu's approach --- hypothesis first, polish later --- maps onto how real designers actually work. I use this pattern even when I'm not using Huashu Design specifically.

Core Asset Protocol: Brand-Consistent Output

The hardest rule in the skill. When your task involves a specific brand, the agent must follow a 5-step process before writing a single line of HTML.

The problem the protocol solves: agents default to generic visuals. Without real brand assets, an iPhone prototype looks like every other phone. A Volta Motors slide deck looks like a generic car pitch. The Core Asset Protocol forces the agent to gather real assets first.

1. **Ask** for 6 asset types: logo, product shots, UI screenshots, color palette, fonts, brand guidelines. The agent presents a checklist and waits.

2. **Search official channels**: `<brand>.com/brand`, `<brand>.com/press`, `brand.<brand>.com`, product pages, launch films. Three fallback paths per asset type.

3. **Download by asset type**. Logo: SVG file, then inline SVG extraction from homepage, then social media avatar. Product shots: product page hero, then press kit, then YouTube video frame, then AI-generated from reference. UI: App Store screenshots, then official video frames, then user screenshots.

4. **Verify and extract**. Check resolution, fidelity, freshness. Grep exact colors from real assets.

5. **Freeze to spec**. Write `brand-spec.md` with all paths, CSS variables, and constraints. All generated HTML references `var(--brand-*)` exclusively --- never hardcoded values.

The quality gate is called the 5-10-2-8 rule: 5 rounds of search, 10 candidates, pick the 2 best, each must score 8 out of 10 or higher on fidelity.

The asset importance ranking keeps the agent focused:

Priority	Asset Type	Notes
Mandatory	Logo	No exceptions. Without the real logo, brand recognition is zero.
High	Product renders	For physical products. Real photos, never CSS silhouettes.
High	UI screenshots	For digital products. App Store captures work well.
Auxiliary	Color values	Grepped from real assets, not guessed.
Auxiliary	Fonts	From brand guidelines or identified from screenshots.

The anti-pattern the protocol explicitly forbids: using CSS silhouettes or SVG hand-drawn substitutes for real product images. The skill calls this out as producing "generic tech animation with zero brand recognition." I've seen this failure mode in every agent design tool that doesn't enforce asset gathering.

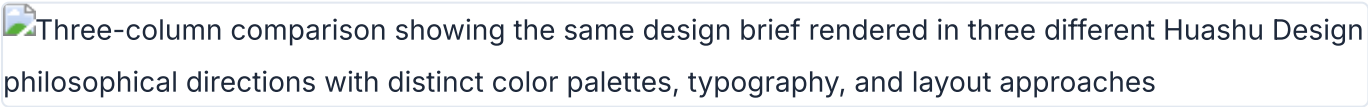
Before the Core Asset Protocol runs, Principle #0 kicks in: fact verification. If the task mentions a specific product, technology, or event, the agent must search first to confirm it exists. The canonical failure case: an agent assumed "DJI Pocket 4" wasn't released yet, but it had launched four days earlier. The search costs 10 seconds. The wrong assumption costs 1-2 hours of rework.

The frozen brand spec looks like this:

```
# Volta Motors · Brand Spec
> Collected: 2026-05-18
> Sources: voltamotors.com/press, voltamotors.com/design
> Completeness: complete

## Core Assets
### Logo
- Main: assets/volta-brand/logo.svg
- Inverted: assets/volta-brand/logo-white.svg

### Product Images
- Hero: assets/volta-brand/volta-ev-hero.png (2000x1500)
- Detail: assets/volta-brand/volta-ev-interior.png

## Color Palette
- Primary: #0066CC
- Background: #FFFFFF
- Ink: #000000
- Surface: #F5F5F5
- Forbidden: never use red tones

## Typography
- Display: 'Volta Sans', 'Inter', sans-serif, 700
- Body: 'Volta Sans', 'Inter', sans-serif, 400
- Mono: 'JetBrains Mono', monospace, 400
```

Design Direction Advisor: 5 Schools x 20 Philosophies

When the brief is vague --- no brand context, no style reference, user wants recommendations --- the Direction Advisor activates. It presents 20 design philosophies organized into 5 schools, each with a distinct worldview.



Three design variants of the same brief showing different philosophical directions

Huashu Design direction advisor with 5 schools and 20 design philosophies

School	Philosophies	Exemplar
Information Architecture	4 philosophies (01-04)	Pentagram --- rational, data-driven, restrained
Kinetic Poetry	4 philosophies (05-08)	Field.io --- dynamic, immersive, technical aesthetics
Minimalism	4 philosophies (09-12)	Kenya Hara --- order, whitespace, refinement
Experimental Avant-Garde	4 philosophies (13-16)	Sagmeister --- pioneer, generative art, visual impact
Eastern Philosophy	4 philosophies (17-20)	Warm, poetic, contemplative

The advisor follows an 8-phase flow:

1. Deep understanding --- max 3 questions, no more.
2. Advisory restatement --- 100-200 words confirming what the agent understood.
3. Recommend 3 philosophies --- must be from 3 different schools.
4. Show showcase gallery --- 24 prebuilt samples (8 scenes x 3 styles).
5. Generate 3 visual demos --- parallel if the agent supports subagents.
6. User picks one, mixes, or requests a redo.
7. Generate AI prompt with specific characteristics from the chosen direction.
8. Continue into the Junior Designer main flow.

Example prompt that triggers the advisor:

```
"Make a keynote for AI psychology. Give me 3 style directions to pick from."
```

The agent asks up to 3 clarifying questions, then presents 3 directions from different schools --- say, an Information Architecture approach, a Kinetic Poetry approach, and an Eastern Philosophy approach --- with live HTML demos for each. You pick one. The agent then builds the full artifact using that direction's constraints.

The parallel demo generation is worth emphasizing. If your agent supports subagents (Chapter 11), it generates all 3 demos simultaneously. On a single-agent setup, it generates them sequentially. The quality difference is negligible --- the parallelism is a speed optimization, not a quality one. What matters is that you see real, rendered HTML before you commit, not descriptions or mood boards.

The 24 prebuilt showcases serve a specific purpose: they give you a visual vocabulary. Most people can't describe what they want in design terms. But when you see a Minimalist layout next to a Kinetic Poetry layout next to an Eastern Philosophy layout, you can point and say "something like that one." The showcases bridge the gap between vague intent and specific direction.

Anti-AI-Slop Rules and Quality Guardrails

The skill defines "AI slop" precisely: *the visual greatest common denominator of AI training data*. It carries zero brand information. You've seen it. Aggressive purple gradients. Emoji as icons. Rounded cards with a left colored border. Dark blue `#0D1117` backgrounds. Inter or Roboto as the display font.

The skill lists specific elements to avoid:

- Aggressive purple gradients (the AI "tech feel" formula)
- Emoji as icons (training data's "every bullet needs emoji" disease)
- Rounded cards + left colored border accent (2020-2024 Material/Tailwind cliché)
- SVG-drawn imagery --- faces, scenes, objects --- always anatomically wrong
- CSS silhouettes or SVG hand-drawn substitutes for real product shots
- Inter, Roboto, Arial, or system fonts as display type
- Cyber neon or dark blue `#0D1117` backgrounds

One exception: "the brand itself uses it" is the only valid reason to break any rule.

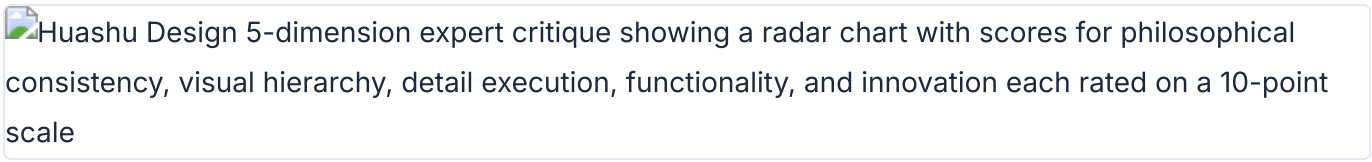
The positive replacements are more interesting:

- `text-wrap: pretty` + CSS Grid + advanced CSS --- typographic details AI can't distinguish from human work
- `oklch()` colors or colors extracted from real assets, never invented
- AI-generated images (via Gemini/Flash) over SVG hand-drawn --- the real thing looks better than any approximation
- Serif display fonts (Newsreader, Source Serif, EB Garamond) for personality
- One detail at 120%, everything else at 80% --- the definition of taste

That last rule is worth pausing on. "One detail at 120%, everything else at 80%." Most AI-generated design is 100% everything --- every element competing for attention, nothing breathing. Restraint is the signal.

The 5-Dimension Expert Critique

After the agent produces an artifact, you can run a critique. The skill evaluates on 5 dimensions, each scored 0-10:



5-dimension expert critique with radar chart scoring philosophical consistency through innovation

1. **Philosophy coherence** --- does the output match the chosen design direction consistently?
2. **Visual hierarchy** --- can you tell what matters most? Does the eye flow correctly?
3. **Execution craft** --- typography, spacing, alignment, color accuracy. The details.
4. **Functionality** --- do interactive elements work? Do links go places? Is the prototype actually clickable?
5. **Innovation** --- does anything surprise? Or is it a competent reproduction of a template?

The output is a radar chart visualization plus a Keep/Fix/Quick Wins punch list. It takes about 3 minutes to run.

```
"Run a 5-dimension expert review on this design."
```

The critique is honest. I've run it on the skill's own output and received scores of 6/10 on innovation. That's appropriate. The value of the critique isn't the score --- it's the specific fix items. "The hero section has correct hierarchy but the footer uses a different spacing scale than the rest of the page. Quick win: unify to the 8px grid."

> **My take:** The self-critique capability is the feature I didn't know I needed. Most agent design tools produce output and stop. Huashu Design produces output, then evaluates its own work against explicit criteria. The radar chart makes the evaluation legible. I run the critique on every artifact before delivery, even when I think it looks good.

Export: MP4, GIF, PPTX, PDF

Huashu Design doesn't just produce HTML. The `scripts/` directory contains a full export toolchain.

Motion design: MP4 at 25fps base with 60fps interpolation. GIF with palette optimization. 6 scene-specific background music tracks with automatic ducking. The animation model uses Stage + Sprite time-slicing with `useTime`, `useSprite`, `interpolate`, and `Easing` APIs from `assets/animations.jsx`.

```
// Stage + Sprite animation model (from assets/animations.jsx)
function ProductReveal() {
  const time = useTime();
  const spriteProgress = useSprite(time, {
    duration: 2,
    easing: Easing.easeOutExpo
  });

  const logoScale = interpolate(spriteProgress, [0, 1], [0, 1]);
  const titleOpacity = interpolate(spriteProgress, [0.3, 0.8], [0, 1]);

  return (
    <Stage background="var(--brand-background)">
      <Sprite style={{
        transform: `scale(${logoScale})`,
        opacity: titleOpacity
      }}>
        <img src="var(--brand-logo)" alt="Logo" />
      </Sprite>
    </Stage>
  );
}
```

Slide decks: HTML deck for browser presentation, plus editable PPTX via `html2pptx.js`. This is not a screenshot export. The script translates DOM computed styles to real PowerPoint objects --- actual text frames, shapes, and formatting. You get a PPTX file someone can edit in PowerPoint without knowing HTML existed.

Infographics: Print-quality typography with PDF, PNG, and SVG export.

The narrated animation pipeline is particularly sophisticated: TTS generates voice, the agent measures duration, generates `timeline.json`, a NarrationStage drives visuals to match the audio, ducking mix blends voice and music, and the deliverable is both a live-play HTML version and a published MP4.

Capability	Deliverable	Typical Time
Interactive prototype	Single-file HTML, real iPhone bezel, clickable	10-15 min
Slide deck	HTML deck + editable PPTX	15-25 min
Motion design	MP4 + GIF + BGM	8-12 min
Design variations	3+ side-by-side with Tweaks	10 min
Infographic	Print-quality, PDF/PNG/SVG	10 min
Direction advisor	3 directions with parallel demos	5 min
Expert critique	Radar chart + Keep/Fix/Quick Wins	3 min

Example prompts for common tasks:

```
"Build an iOS prototype for a Pomodoro app — 4 screens, actually clickable."

"Turn this logic into a 60-second animation. Export MP4 and GIF."

"Make a keynote for AI psychology. Give me 3 style directions to pick from."

"Run a 5-dimension expert review on this design."
```

The iOS prototype is worth calling out specifically. The `assets/ios_frame.jsx` component renders a real iPhone bezel with status bar (time, battery, signal). The prototype inside is actually clickable --- not a scroll-through mockup. Before delivery, the agent runs Playwright click tests to verify every interactive element responds correctly.

```
<script type="text/babel">
  // Load iosFrameStyles + IosFrame from assets/ios_frame.jsx
  function App() {
    return <IosFrame time="9:41" battery={85}>
      <PomodoroScreen />
    </IosFrame>;
  }
</script>
```

Tip

Next: Huashu Design produces brand-consistent output through its Core Asset Protocol and CSS variable system. Chapter 08 goes deeper into design tokens --- how they work across tools, how to sync them between Figma and code, and how to write your own design system that agents can read.

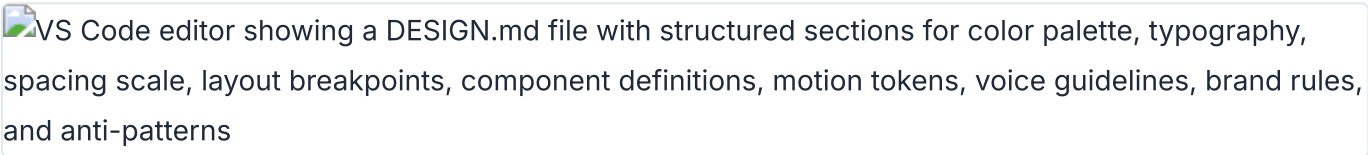
08 Design Systems and Tokens

Maintaining design systems with agents across tools and codebases

Design tokens are the connective tissue between design tools, codebases, and AI agents. This chapter covers the DESIGN.md schema that makes design systems machine-readable, the 129-system library in Open Design, token syncing between Figma, Paper, Pencil, and code, and how to write your own design system that agents can actually use.

Design Systems as Code: The DESIGN.md Schema

The DESIGN.md schema started at awesome-design-md and became the de facto standard for machine-readable design systems. It's a Markdown file with 9 sections that any agent can parse and apply:



A DESIGN.md file showing the 9-section schema for machine-readable design systems

1. **Color** --- brand palette, semantic colors, forbidden colors
2. **Typography** --- display, body, mono font stacks with sizes and weights
3. **Spacing** --- margin/padding scale, grid units
4. **Layout** --- breakpoints, container widths, grid system
5. **Components** --- button variants, card styles, form elements
6. **Motion** --- transitions, durations, easing curves
7. **Voice** --- tone, language patterns, copy guidelines
8. **Brand** --- logo usage, imagery style, photography direction
9. **Anti-patterns** --- what not to do, forbidden combinations

```
# Payflow Design System

## Color
- Primary: #4F46E5
- Secondary: #1E293B
- Background: #FFFFFF
- Surface: #F8FAFC
- Text Primary: #1E293B
- Text Secondary: #64748B
- Accent: #06B6D4
- Error: #EF4444
- Success: #22C55E
- Forbidden: never use pure black #000000 for text

## Typography
- Display: 'Payflow Sans', -apple-system, sans-serif, 700
- Heading: 'Payflow Sans', -apple-system, sans-serif, 600
- Body: 'Payflow Sans', -apple-system, sans-serif, 400
- Mono: 'Payflow Mono', 'SF Mono', monospace, 400

## Spacing
- xs: 4px
- sm: 8px
- md: 16px
- lg: 24px
- xl: 32px
- 2xl: 48px

## Layout
- Max width: 1200px
- Grid: 12 columns
- Breakpoints: mobile 640px, tablet 768px, desktop 1024px

## Components
- Button primary: bg-primary, text-white, rounded-lg, px-md py-sm
- Card: bg-surface, rounded-xl, shadow-md, p-lg
- Input: border-gray-300, rounded-lg, px-md py-sm

## Motion
- Fast: 150ms ease-out
- Normal: 250ms ease-out
- Slow: 350ms ease-out

## Voice
- Tone: professional but approachable
- Language: active voice, direct sentences

## Brand
- Logo: always on white/dark backgrounds, minimum 24px height
- Imagery: abstract geometric patterns, no stock photography
```

```
## Anti-patterns
- Never use gradient fills on text
- Never pair sans-serif heading with serif body
- Never animate layout properties (width, height, top, left)
```

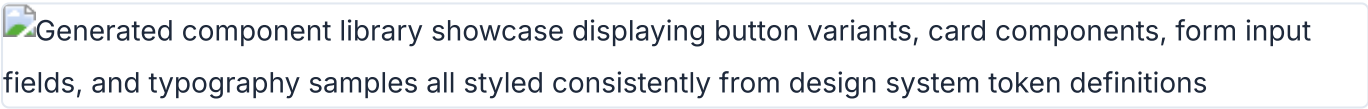
That anti-patterns section is critical. Most design system documentation tells you what to do. The anti-patterns section tells the agent what *not* to do. In practice, this is more valuable. Agents are prone to falling back to defaults. Explicit prohibitions prevent that.

The schema works because it's Markdown. Agents read Markdown natively. No JSON parsing. No API calls. No SDK integration. The DESIGN.md file sits in your repo, and any agent that can read files can consume it.

```
// Agent prompt to read a design system
"Read DESIGN.md from the repo root. Generate a pricing page
that uses the exact colors, fonts, spacing, and component styles
defined in the design system. Do not invent any values."
```

Open Design's 129-System Library

Open Design (covered in Chapter 06) ships 129 design systems using the DESIGN.md schema. The breakdown:



Component library generated from design system tokens showing buttons, cards, and form elements

- 2 hand-authored starter systems
- 70 product design systems from awesome-design-md
- 57 design skills from awesome-design-skills

The 70 product systems include real brand systems: Linear, Stripe, Vercel, Airbnb, Tesla, Notion, Anthropic, Apple, Cursor, Supabase, Figma, Resend, Raycast, Cohere, Mistral, ElevenLabs, Spotify, PostHog, Sentry, MongoDB, and roughly 50 more.

Each system shows its 4-color signature in the picker. Click for the full DESIGN.md, a swatch grid, and a live showcase. The catalog endpoint at `GET /api/skills` lists available systems programmatically.

Category	Count	Source
Starter systems	2	Hand-authored for Open Design
Product design systems	70	awesome-design-md
Design skills	57	awesome-design-skills
Curated visual directions	5	Deterministic OKLch palettes + font stacks

When no brand design system exists, Open Design falls back to 5 curated visual directions, each with a deterministic OKLch palette and matched font stack:

```
Editorial Monocle  — serif font stack, warm OKLch palette
Modern Minimal     — sans-serif font stack, neutral OKLch palette
Warm Soft          — humanist font stack, warm OKLch palette
Tech Utility       — monospace-leaning font stack, cool OKLch palette
Brutalist Experimental — display font stack, high-contrast OKLch palette
```

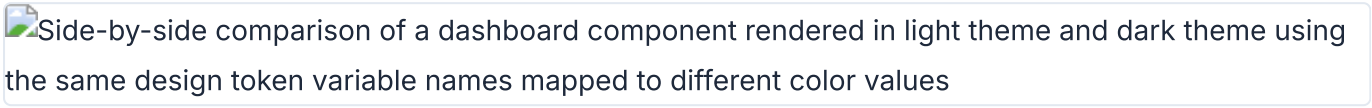
Switch the design system and the next render uses the new tokens immediately. This is the key insight: "Design Systems are portable Markdown, not theme JSON." Every artifact reads from the active system at generation time.

The most-used systems in the library, based on community activity:

Design System	Brand	Signature
Linear	Linear (productivity)	Purple accent, dark-first, sharp corners
Stripe	Stripe (payments)	Purple primary, white surfaces, isometric illustrations
Vercel	Vercel (developer tools)	Black/white, monospace accents, gradient borders
Notion	Notion (productivity)	Warm blacks, serif headings, illustration style
Apple	Apple (hardware/software)	San Francisco, rounded corners, spacious layouts
Anthropic	Anthropic (AI research)	Earthy tones, serif display, academic feel
Spotify	Spotify (music)	Green accent, bold typography, dark backgrounds
Resend	Resend (email API)	Dark theme, green accents, code-forward aesthetic

Syncing Tokens Between Tools

Design tokens need to flow between tools. A color defined in Figma must appear identically in Paper, in your codebase, and in generated HTML. The synchronization strategy depends on which tools you're connecting.



Same component rendered with light and dark theme tokens from a single design system

Figma to Paper requires dual MCP servers in the same agent session. The Figma MCP reads variables and styles. The Paper MCP writes them to the Paper canvas.

```
"I have a design system in Figma with colors and text styles.
Please create a matching design system sticker sheet in Paper
using the same tokens. The element with the variable applied
is selected in Figma."
```

The agent reads Figma tokens via the Figma MCP, then writes to Paper via the Paper MCP. Chapter 10 covers the MCP configuration in detail.

Gotchas with Figma-to-Paper sync:

Issue	Cause	Fix
SVG fills become images	Figma exports SVG fills as embedded raster	Use fill color tokens instead of SVG imports
Spacer elements ignored	Figma spacers have no visual output	Map spacer values to padding tokens manually
Code-connected components unreliable	Figma's code connect feature has sync gaps	Verify component output against source manually

Figma to Code is simpler. The agent reads Figma variables via the Figma MCP and generates CSS custom properties directly.

```
:root {
  --color-primary: #4F46E5;
  --color-secondary: #1E293B;
  --color-background: #FFFFFF;
  --color-surface: #F8FAFC;
  --color-text: #1E293B;
  --color-text-secondary: #64748B;
  --color-accent: #06B6D4;
  --spacing-xs: 4px;
  --spacing-sm: 8px;
  --spacing-md: 16px;
  --spacing-lg: 24px;
  --spacing-xl: 32px;
  --font-display: 'Payflow Sans', -apple-system, sans-serif;
  --font-body: 'Payflow Sans', -apple-system, sans-serif;
  --font-mono: 'Payflow Mono', 'SF Mono', monospace;
  --radius-sm: 4px;
  --radius-md: 8px;
  --radius-lg: 12px;
}
```

Paper to Code uses the Paper MCP's `get_computed_styles` and `get_jsx` tools. The agent reads the design from Paper, extracts token values, and generates matching code tokens.

```
// Agent prompt for Paper-to-code token extraction
"Read the design from Paper. Extract all colors, fonts, and
spacing values into CSS custom properties. Generate a tokens.css
file that matches the Paper design exactly."
```

```
// Generated tokens.css from Paper design
:root {
  --paper-color-hero-bg: oklch(0.95 0.02 260);
  --paper-color-text-primary: oklch(0.15 0.02 260);
  --paper-font-display: 'Instrument Serif', Georgia, serif;
  --paper-font-body: 'Inter', -apple-system, sans-serif;
  --paper-spacing-section: 96px;
  --paper-spacing-content: 24px;
}
```

Multi-source sync chains all of this together. Connect Figma MCP + Paper MCP + codebase access in a single agent session. The agent reads from Figma, writes to Paper, generates code, and verifies parity --- all in one conversation.

Design Variables in .pen and .op Files

Both Pencil (.pen files) and OpenPencil (.op files) support design variables. The mechanisms differ, but the goal is the same: tokens defined once, referenced everywhere.

Pencil (.pen) Variables

Pencil (Chapter 05) provides a variables system for design tokens inside .pen files. Variables map to CSS custom properties on export. Components and slots let you build reusable elements that reference these variables.

```
// .pen file variable structure (JSON)
{
  "variables": {
    "color-primary": "#4F46E5",
    "color-background": "#FFFFFF",
    "spacing-md": 16,
    "radius-lg": 12
  }
}
```

When the agent generates UI from a .pen file, every token reference becomes `var(--color-primary)` in the output. Change the variable in the .pen file, and the exported code updates accordingly.

OpenPencil (.op) Variables

OpenPencil (Chapter 06) extends the concept with typed variables and multi-theme support.

```
{
  "variables": {
    "color-primary": { "type": "color", "value": "#4F46E5" },
    "color-secondary": { "type": "color", "value": "#1E293B" },
    "spacing-md": { "type": "number", "value": 16 },
    "font-body": { "type": "string", "value": "Inter, sans-serif" }
  },
  "themes": {
    "mode": {
      "Light": { "color-background": "#FFFFFF" },
      "Dark": { "color-background": "#1E293B" }
    },
    "density": {
      "Compact": { "spacing-md": 12 },
      "Comfortable": { "spacing-md": 16 }
    }
  }
}
```

The theme axes are independent. Light + Compact, Dark + Comfortable, any combination. Variables reference values via `$variable-name` syntax in the .op file, which becomes `var(--variable-name)` in generated CSS.

```
// Theme switching in generated CSS from .op file
:root {
  --color-background: #FFFFFF;
  --color-text: #1A1A2E;
  --spacing-md: 16px;
}

[data-theme="Dark"] {
  --color-background: #1E293B;
  --color-text: #F8FAFC;
}

[data-density="Compact"] {
  --spacing-md: 12px;
}
```

The generated CSS uses data-attribute selectors for theme switching. This is a deliberate choice: data attributes work without JavaScript. The browser applies the correct theme based on the attribute value, with no runtime cost. The design system remains purely declarative.

Feature	Pencil (.pen)	OpenPencil (.op)
Variable types	Untyped values	Typed (color, number, string)
Theme support	Single theme	Multi-axis themes
Reference syntax	Variable name	`$variable-name`
CSS output	`var(--name)`	`var(--name)`
File format	Design-as-code	Git-friendly JSON
Component reuse	Components + slots	Components + instances + overrides

Generating Consistent UI from Design System Constraints

Reading a design system is one thing. Enforcing it during generation is another. Each tool takes a different approach.

Open Design: The agent picks a skill and a design system. The skill template enforces the system's tokens at generation time. Every color comes from the system's palette. Every font comes from the system's typography section. The agent doesn't get to invent values.

Huashu Design (Chapter 07): The Core Asset Protocol freezes brand knowledge into `brand-spec.md`. All generated HTML references `var(--brand-*)` CSS variables exclusively. The anti-slop rules prevent fallback to generic defaults.

```
:root {
  --brand-primary: #4F46E5;
  --brand-background: #FFFFFF;
  --brand-ink: #1E293B;
  --brand-accent: #06B6D4;
}

/* All generated HTML uses var(--brand-*) — never hardcoded values.
   To change brand, update brand-spec.md, not individual HTML files. */
```

OpenPencil: Style guides with 50+ built-in styles, tag-based fuzzy matching applied during AI generation. The agent writes a component, and OpenPencil matches it against the style guide to ensure consistency. The matching is tag-based and fuzzy, meaning it doesn't require exact name matches --- "primary button" matches "Button/Primary/Large" through similarity scoring.

The common thread across all tools: agents read the design system before generating anything. This "read before write" pattern is fundamental. An agent that generates UI without reading the design system first will produce generic output. An agent that reads the system first produces consistent output. The design system file --- whether DESIGN.md, brand-spec.md, or .op variables --- is the contract.

```
// Agent workflow: read system, then generate
// Step 1: Agent reads DESIGN.md
// Step 2: Agent extracts tokens into CSS variables
// Step 3: Agent generates UI referencing those variables

// Generated component using design system tokens
.pricing-card {
  background: var(--color-surface);
  border: 1px solid var(--color-border, oklch(0.85 0.02 260));
  border-radius: var(--radius-lg);
  padding: var(--spacing-lg);
  font-family: var(--font-body);
  transition: box-shadow var(--motion-normal);
}

.pricing-card:hover {
  box-shadow: 0 4px 12px oklch(0.15 0.02 260 / 0.1);
}
```

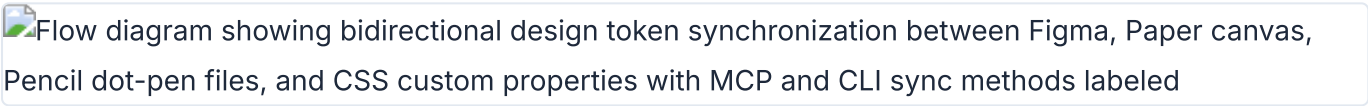
The enforcement pattern that works across all tools:

```
/* Anti-slop enforcement rules, common to all design-as-code tools */

/* 1. Never invent new colors outside the system */
/* BAD: color: #8B5CF6; (invented purple) */
/* GOOD: color: var(--color-accent); */

/* 2. Never use display fonts not in the spec */
/* BAD: font-family: 'Montserrat', sans-serif; */
/* GOOD: font-family: var(--font-display); */

/* 3. Never add decorative elements not justified by content */
/* BAD: decorative SVG divider between every section */
/* GOOD: spacing and typography alone create separation */

/* 4. Use oklch() or spec-existing colors only */
/* BAD: filter: hue-rotate(45deg); */
/* GOOD: color: oklch(0.7 0.15 250); */
```

```
/* CSS Grid + text-wrap: pretty as quality signals */
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
  gap: var(--spacing-lg);
}

.card-content {
  text-wrap: pretty;
  font-family: var(--font-body);
  color: var(--color-text);
}
```

Writing Your Own Design System for Agents

Based on the patterns across Open Design, Huashu Design, Pencil, and OpenPencil, here's how to write a design system that agents can actually use.



Token synchronization flow between Figma, Paper, Pencil, and production code

1. **Start with the 9-section schema.** Color, typography, spacing, layout, components, motion, voice, brand, anti-patterns. Even if some sections are thin, include them all. Agents expect the full structure.

2. **Be specific.** Exact HEX values, not "a shade of blue." Font stacks with fallbacks, not "a nice sans-serif." Pixel measurements, not "comfortable spacing."

3. **Include anti-patterns.** What not to do is as important as what to do. "Never use gradient fills on text" prevents more failures than "use solid fills."

4. **Map every token to a CSS variable.** The agent needs `--color-primary`, not just `#4F46E5`. The variable name is the contract between the design system and generated code.

5. **Freeze brand assets.** Logo paths, product images, UI screenshots --- not just colors. Huashu Design's `brand-spec.md` pattern is the reference implementation (Chapter 07).

6. **Add theme variants.** Light and dark at minimum. Compact and comfortable as a density axis if applicable. OpenPencil's multi-axis theme system is the most flexible model.

```
# My Product Design System

## Color
- Primary: #1A1A2E
- Accent: #E94560
- Background: #FFFFFF
- Surface: #F8F9FA
- Text: #1A1A2E
- Text Secondary: #6C757D
- Forbidden: never use #E94560 for large background areas

## Typography
- Display: 'Newsreader', Georgia, serif, 700
- Heading: 'Inter', -apple-system, sans-serif, 600
- Body: 'Inter', -apple-system, sans-serif, 400
- Mono: 'JetBrains Mono', 'SF Mono', monospace, 400

## Spacing
- xs: 4px
- sm: 8px
- md: 16px
- lg: 24px
- xl: 32px

## Layout
- Max width: 1120px
- Grid: 12 columns, 24px gutter
- Breakpoints: sm 640px, md 768px, lg 1024px, xl 1280px

## Components
- Button primary: bg-primary, text-white, rounded-md, px-md py-sm, font-heading
- Card: bg-surface, border-1 border-gray-200, rounded-lg, p-lg
- Input: border-gray-300, rounded-md, px-md py-sm, focus:ring-accent

## Motion
- Fast: 150ms ease-out
- Normal: 250ms ease-out
- Slow: 400ms ease-in-out

## Voice
- Tone: direct, technical, no hype
- Language: active voice, short sentences

## Brand
- Logo: SVG only, minimum 32px height
- Imagery: abstract geometric patterns, no stock photos

## Anti-patterns
- Never use box-shadow for elevation (use border instead)
```

```
- Never animate color properties (use opacity transitions)
- Never use emoji in UI copy
- Never center-align body text paragraphs
```

```
:root {
  --color-primary: #1A1A2E;
  --color-accent: #E94560;
  --color-background: #FFFFFF;
  --color-surface: #F8F9FA;
  --color-text: #1A1A2E;
  --color-text-secondary: #6C757D;
  --spacing-xs: 4px;
  --spacing-sm: 8px;
  --spacing-md: 16px;
  --spacing-lg: 24px;
  --spacing-xl: 32px;
  --font-display: 'Newsreader', Georgia, serif;
  --font-heading: 'Inter', -apple-system, sans-serif;
  --font-body: 'Inter', -apple-system, sans-serif;
  --font-mono: 'JetBrains Mono', 'SF Mono', monospace;
  --radius-sm: 4px;
  --radius-md: 6px;
  --radius-lg: 8px;
  --motion-fast: 150ms ease-out;
  --motion-normal: 250ms ease-out;
  --motion-slow: 400ms ease-in-out;
}
```

Cross-Tool Token Parity Strategies

Maintaining token parity across Figma, Paper, Pencil, your codebase, and generated HTML is the operational challenge of design-as-code. Five strategies, from simplest to most robust:

1. DESIGN.md as single source of truth. Put the DESIGN.md in your repo. Every agent session reads it before generating anything. Other tools (Figma, Paper) are downstream consumers. Simple but requires manual sync.

2. MCP chaining for real-time sync. Connect Figma MCP + Paper MCP + codebase in one agent session. The agent reads from one source and writes to others. Real-time parity, but requires all MCP servers running simultaneously.

3. Export/import bridges. Figma exports tokens via its MCP. Paper reads them. Paper exports JSX with tokens. Your codebase imports it. Each bridge is a one-way sync point. Robust but adds pipeline complexity.

4. Git-based parity. DESIGN.md lives in the repo. CI verifies that generated CSS custom properties match the DESIGN.md values. Tokens diverge, build fails. Most reliable for teams with CI discipline.

5. Variable reference pattern. All generated code uses `var(--token-name)` instead of hardcoded values. The CSS variables file is generated from DESIGN.md. If the variable reference is present, the value is correct by construction. This is the pattern Huashu Design enforces.

```
/* Variable reference pattern — all tools converge here */

/* Generated from DESIGN.md, single source of truth */
:root {
  --color-primary: #1A1A2E;
  --color-accent: #E94560;
}

/* Agent-generated component — never hardcodes values */
.hero {
  background: var(--color-primary);
  color: var(--color-background);
  padding: var(--spacing-xl);
  font-family: var(--font-display);
  border-radius: var(--radius-lg);
  transition: transform var(--motion-normal);
}
```

Strategy	Complexity	Reliability	Best For
DESIGN.md as source	Low	Medium (manual sync)	Solo developers, small teams
MCP chaining	Medium	High (real-time)	Teams using Figma + Paper
Export/import bridges	Medium	Medium (pipeline)	Teams with CI pipelines
Git-based parity	High	High (automated)	Teams with strong CI discipline
Variable reference pattern	Low	High (by construction)	All teams --- combine with other strategies

My take: The variable reference pattern is the foundation everything else builds on. If your generated code hardcodes `#4F46E5` instead of `var(--color-primary)`, no amount of DESIGN.md discipline will save you. Start there. Add MCP chaining or CI verification on top if your team size warrants it.

The direction system provides a fallback when no brand design system exists. Open Design's 5 curated directions --- Editorial Monocle, Modern Minimal, Warm Soft, Tech Utility, Brutalist Experimental --- each with deterministic OKLch palettes, give agents a consistent starting point. The

key word is deterministic: the same direction always produces the same palette, which means the same prompt generates visually consistent output across sessions.

> **Tip**
>
> **Next:** Design systems give visual consistency. Motion gives temporal consistency. Chapter 09 covers programmatic video --- how agents produce animations, explainer videos, and motion design using Remotion and Hyperframes.

09 Motion and Video

Remotion, Hyperframes, and programmatic video for agents

Video is becoming a first-class output of AI agent workflows. Two frameworks dominate: Remotion (47.1k stars) makes videos with React components. Hyperframes (19k stars) makes videos with HTML and GSAP, built specifically for agents. This chapter walks through both, compares them honestly, and shows you how to integrate programmatic video into your agent design workflow.

Why Agents Need Programmatic Video

Traditional video tools --- Premiere, After Effects, Final Cut --- require GUI interaction. Agents can't click timeline scrubbers. They can't drag keyframes. They can't adjust bezier curves on easing functions with a mouse. Programmatic video frameworks solve this by treating video as code: versionable, diffable, reproducible.

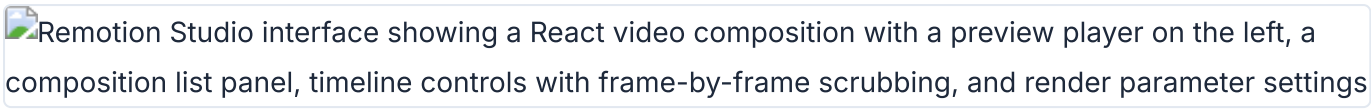
The shift mirrors what happened with design. Design moved from GUI-only tools (Photoshop) to code-friendly formats (CSS, HTML, design tokens). Video is making the same transition. The timing matters because agents can now produce video artifacts the same way they produce code and design --- by writing text that compiles to frames.

Three use cases drive agent-driven video production in practice. First, product marketing: a startup needs a 60-second product intro, and the designer writes it once with code rather than editing frames manually. Second, personalized video at scale: a SaaS company generates thousands of onboarding videos, each with the customer's name and usage data. Third, design documentation: a team produces animated walkthroughs of UI flows, generated directly from the design system tokens covered in Chapter 08.

Two approaches have emerged. Remotion uses React components as the authoring format. Hyperframes uses HTML + GSAP. The philosophical difference sounds minor. In practice, it shapes everything: how you author, how you preview, how you debug, how you render, and what your license allows.

Remotion: React-Based Video Production

Remotion is the established player. Created by Jonny Burger (Remotion AG), 47.1k GitHub stars, 3.3k forks, 622+ releases as of May 2026. The current version is v4.0.462. It's TypeScript-first (74.5%), with PHP for Lambda rendering and Rust for native components. The ecosystem is substantial: 3M npm installs, 800+ pages of documentation, 35+ templates, 8000+ Discord members, and 300+ contributors. Trusted by GitHub, Musixmatch, Wistia, and SoundCloud.



Remotion Studio showing a video composition with preview player and timeline controls

The mental model: your video is a React component tree. Each frame is a render of that tree at a specific point in time. The framework handles the mapping between time and component state. If you know React, you know most of Remotion already.

Core primitives:

Primitive	Purpose	Key Props
`<Composition>`	Registers a renderable video	`id`, `fps`, `durationInFrames`, `width`, `height`
`<Sequence>`	Time-shifts child components	`from`, `durationInFrames`
`<Series>`	Plays sequences back-to-back	Auto-calculated timing
`spring()`	Physics-based animation	`mass`, `damping`, `stiffness`
`interpolate()`	Maps frame number to value range	`inputRange`, `outputRange`, `extrapolateLeft/Right`
`useCurrentFrame()`	Hook for current frame number	None
`useVideoConfig()`	Hook for video metadata	Returns `fps`, `width`, `height`, `durationInFrames`

```
// src/Root.tsx — register compositions
import { Composition } from 'remotion';
import { ProductIntro } from './ProductIntro';

export const RemotionRoot: React.FC = () => {
  return (
    <Composition
      id="ProductIntro"
      durationInFrames={150}
      fps={30}
      width={1920}
      height={1080}
      component={ProductIntro}
    />
  );
};
```

```
// Frame-dependent rendering
import { AbsoluteFill, useCurrentFrame, interpolate } from 'remotion';

export const ProductIntro = () => {
  const frame = useCurrentFrame();

  const opacity = interpolate(frame, [0, 30], [0, 1], {
    extrapolateRight: 'clamp',
  });

  const scale = interpolate(frame, [0, 30], [0.8, 1], {
    extrapolateRight: 'clamp',
  });

  return (
    <AbsoluteFill style={{
      backgroundColor: 'var(--color-background)',
      justifyContent: 'center',
      alignItems: 'center',
    }}>
      <div style={{ opacity, transform: `scale(${scale})` }}>
        <h1 style={{ fontFamily: 'var(--font-display)' }}>
          Introducing the Future
        </h1>
      </div>
    </AbsoluteFill>
  );
};
```

```
// Sequences for timing control
import { Sequence } from 'remotion';

const ProductTrailer = () => {
  return (
    <>
      <Sequence durationInFrames={30}>
        <LogoReveal />
      </Sequence>
      <Sequence from={30} durationInFrames={60}>
        <FeatureShowcase />
      </Sequence>
      <Sequence from={90} durationInFrames={60}>
        <CallToAction />
      </Sequence>
    </>
  );
};
```

The `spring()` function deserves a closer look. Unlike CSS transitions with fixed durations, spring animations are physics-based. You set `mass`, `damping`, and `stiffness`, and the framework calculates the animation curve. This produces motion that feels natural --- objects accelerate, overshoot slightly, and settle.

```
import { spring, useCurrentFrame, useVideoConfig } from 'remotion';

const AnimatedTitle = () => {
  const frame = useCurrentFrame();
  const { fps } = useVideoConfig();

  const scale = spring({
    frame,
    fps,
    config: {
      stiffness: 100,
      damping: 10,
      mass: 1,
    },
  });

  return (
    <div style={{
      transform: `scale(${scale})`,
      fontFamily: 'var(--font-display)',
    }}>
      Hello World
    </div>
  );
};
```

Sequences can nest. A sequence inside another sequence is offset by the parent's `from` value. This composition pattern maps naturally to how real videos are structured: acts contain scenes, scenes contain shots, shots contain elements.

Remotion Studio provides a browser-based preview with timeline scrubbing, a props editor, and a render button. You can deploy Studio to the cloud for non-technical team members to preview and render videos without touching code.

The rendering targets are extensive:

```
$ npx remotion render ProductIntro                # local MP4
$ npx remotion render --sequence                  # image sequence
$ npx remotion render --output custom.mp4         # custom output path
$ npx remotion render --codec prores              # ProRes for editing
$ npx remotion render --frames=0-29               # render specific range
```

Beyond local CLI rendering, Remotion supports Node.js SSR API (`renderMedia()`), AWS Lambda for distributed rendering, GitHub Actions for CI/CD, and Google Cloud Run (alpha as of May 2026). The Lambda integration is mature and production-tested for hyperscale rendering. The SSR API lets you render from any Node.js process, which is particularly useful for agent-driven pipelines.

Setting Up a Remotion Project with Agent Assistance

Quick start:

```
$ npx create-video@latest
```

This scaffolds a Remotion project with a `src/Root.tsx` that registers compositions. Each composition is a React component that receives props and renders frames.

Remotion has first-class agent support. The repository includes `.claude/` and `.cursor/` directories with `CLAUDE.md` and `AGENTS.md` files. This means Claude Code and Cursor understand Remotion's conventions out of the box.

```
my-video/
├── src/
│   ├── Root.tsx          # registers all compositions
│   ├── ProductIntro.tsx  # composition component
│   └── lib/
│       └── animations.ts # shared animation helpers
├── public/
│   └── assets/           # images, videos, fonts
├── package.json
└── remotion.config.ts
```

Agents can scaffold the project, write composition components, configure rendering, and even set up Lambda deployment. The Zod schema support on `<Composition>` enables visual editing of props in Remotion Studio, which agents can configure:

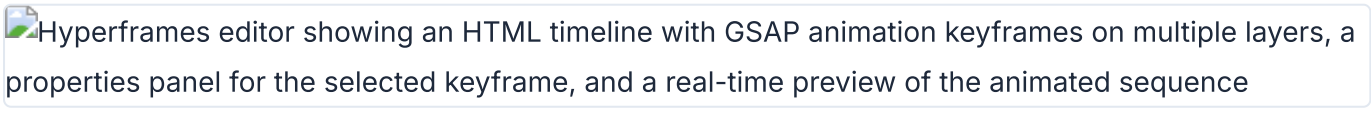
```
import { Composition } from 'remotion';
import { z } from 'zod';

const productSchema = z.object({
  title: z.string(),
  subtitle: z.string(),
  accentColor: z.string(),
  logoUrl: z.string(),
});

export const RemotionRoot = () => (
  <Composition
    id="ProductIntro"
    component={ProductIntro}
    schema={productSchema}
    defaultProps={{
      title: 'Product Name',
      subtitle: 'Tagline goes here',
      accentColor: '#E94560',
      logoUrl: '/assets/logo.svg',
    }}
    durationInFrames={150}
    fps={30}
    width={1920}
    height={1080}
  />
);
```

The agent workflow: you describe the video, the agent writes the composition components, you preview in Remotion Studio, you iterate, then you render. The tight integration with design tokens (from Chapter 08) means your video can share the same `var(--color-primary)` variables as your web UI.

Remotion also provides a `Player` component for embedding video previews in web applications and a `Recorder` for screen recording. The Editor Starter is a commercial template for building custom video editing applications on top of Remotion's rendering engine.

Hyperframes: HTML-Native Video Built for Agents

Hyperframes takes a fundamentally different approach. Created by HeyGen, 19k GitHub stars, 1.8k forks, Apache 2.0 license. The tagline is direct: "Write HTML. Render video. Built for agents."



Hyperframes timeline editor with GSAP keyframes, layer panel, and animation preview

Compositions are plain HTML files with data attributes. No React. No build step. The HTML file is both the render layer and the editable source of truth. This is the same principle that powers Huashu Design (Chapter 07) for prototypes and slide decks --- HTML as the native artifact, not an export target.

The key insight from HeyGen's internal evaluation: LLMs produce more creative output writing HTML+GSAP than React compositions. HTML is lower-friction for language models. No imports to manage. No component lifecycle to reason about. No JSX syntax to get right. The agent writes declarative HTML, adds animation attributes, and the framework handles the rest.

Attribute	Purpose
`data-composition-id`	Identifies the root element
`data-start`	Start time in seconds
`data-duration`	Duration in seconds
`data-track-index`	Layer ordering (higher = on top)
`data-width` / `data-height`	Composition dimensions
`data-volume`	Audio volume (0-1)

```
<div id="root"
     data-composition-id="product-intro"
     data-start="0"
     data-width="1920"
     data-height="1080">

  <video id="clip-1"
         data-start="0"
         data-duration="5"
         data-track-index="0"
         src="intro.mp4"
         muted playsinline></video>

  <h1 id="title"
      class="clip"
      data-start="1"
      data-duration="4"
      data-track-index="1"
      style="font-size: 72px; color: white;">
    Welcome to Hyperframes
  </h1>

  <audio id="bg-music"
         data-start="0"
         data-duration="5"
         data-track-index="2"
         data-volume="0.5"
         src="music.wav"></audio>
</div>
```

No React. No JSX. No build step. You write HTML, open it in a browser to preview, and render it to MP4. The data attributes tell the renderer when each element appears, for how long, and in what layer. The track index system is like a video editor's timeline --- elements on higher tracks overlay elements on lower tracks.

The CLI is deliberately non-interactive by default:

```
$ npx hyperframes init my-video
$ cd my-video
$ npx hyperframes preview         # live preview with hot reload
$ npx hyperframes render          # render to MP4
$ npx hyperframes render --output demo.mp4
$ npx hyperframes add flash-through-white   # add catalog block
$ npx hyperframes lint            # lint composition
$ npx hyperframes doctor          # diagnose issues
```

The `--human-friendly` flag enables interactive mode, but the default is flag-driven. This is a deliberate design choice: agents work better with non-interactive CLIs. Every command accepts flags and exits cleanly.

Hyperframes ships with a catalog of 50+ ready-to-use blocks. These are pre-built composition snippets for common patterns: `flash-through-white`, `instagram-follow`, title cards, lower thirds, transitions. You add them to your project with `npx hyperframes add <block-name>` and customize the data attributes. For agents, this means the agent can compose complex videos from building blocks rather than writing every element from scratch.

The Frame Adapter pattern is where Hyperframes' architecture really matters. It supports GSAP, Lottie, CSS animations, Three.js, Anime.js, and Web Animations API. For each adapter, Hyperframes handles deterministic seeking --- it pauses the animation library and scrubs to `frame / fps` before each capture. This eliminates the wall-clock dependency that plagues other renderers.

My take: Hyperframes' seek-based rendering is technically superior to Remotion's wall-clock approach for animation libraries. GSAP in Remotion races through timelines at wall-clock speed during render, producing mostly-empty frames after the initial frames. Hyperframes pauses GSAP, seeks to the exact time, then captures. If your video uses GSAP heavily, this alone is reason to choose Hyperframes.

Setting Up Hyperframes with AI Agents

The recommended path is skill-based:

```
$ npx skills add heygen-com/hyperframes
```

This installs 13 skills that teach the agent framework-specific patterns. Skills register as slash commands in Claude Code: `/hyperframes`, `/hyperframes-cli`, `/hyperframes-media`, `/gsap`, `/lottie`, `/threejs`, and more.

Requirements: Node.js >= 22 and FFmpeg.

For Codex and Cursor, Hyperframes provides dedicated plugins:

```
# Codex plugin
$ codex plugin marketplace add heygen-com/hyperframes \
    --sparse .codex-plugin --sparse skills --sparse assets

# Claude Code plugin
$ claude --plugin-dir .
```

Example agent prompts that work well:

```
"Create a 10-second product intro with a fade-in title."
"Turn this CSV into an animated bar chart race."
"Build a 60-second explainer video with 4 scenes."
"Add background music with ducking when narration plays."
```

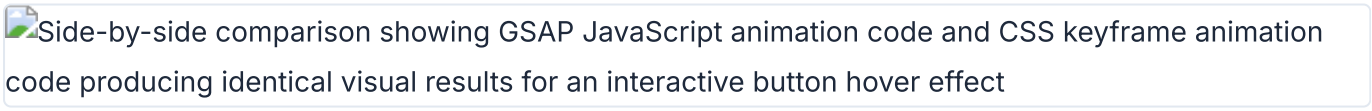
The `hyperframes init` command installs skills automatically, so you can hand a project to an agent at any point in its lifecycle. Media preprocessing skills handle TTS (via Kokoro), transcription (via Whisper), and background removal (via u2net). The TTS integration is particularly useful for narrated videos --- the agent generates voiceover, measures the duration, and synchronizes visuals to the audio timeline automatically.

There's also a `remotion-to-hyperframes` skill for migrating React compositions to HTML. Useful if you start with Remotion and want to switch. The migration handles data attribute mapping, sequence-to-track conversion, and GSAP timeline adaptation.

Hyperframes supports two capture modes. **BeginFrame mode** runs on Linux and is fully deterministic --- no wall-clock dependency. **Screenshot mode** runs on macOS and Windows, falling back automatically when BeginFrame isn't available. For production CI/CD pipelines, Linux with BeginFrame mode is the recommended setup.

Animation Runtimes: GSAP, CSS, Lottie, Three.js

The animation runtime is the most consequential technical difference between Remotion and Hyperframes. It affects how every library behaves during rendering. Understanding this difference is essential for choosing the right tool.



GSAP and CSS animation approaches compared for the same interactive element

GSAP: The primary animation runtime for Hyperframes. Hyperframes pauses GSAP and seeks it to `frame / fps` before each capture. In Remotion, GSAP's internal `performance.now()` ticker races through the timeline at wall-clock speed during render. The result: mostly-empty frames after the initial frames. This is not a minor issue. It makes GSAP effectively unusable in Remotion for any non-trivial animation.

```
// GSAP in Hyperframes — deterministic seeking
const tl = gsap.timeline({ paused: true });
tl.from("#title", { opacity: 0, y: 50, duration: 1 })
  .to("#title", { opacity: 1, y: 0, duration: 0.5 });
// Hyperframes pauses this timeline and seeks to frame/fps before each capture
```

CSS Animations: Both tools support CSS. Hyperframes uses the Web Animations API adapter for frame-accurate seeking. Remotion renders CSS animations frame-by-frame but can struggle with complex keyframe sequences that depend on computed styles.

Lottie: Hyperframes supports Lottie via `window.__hfLottie` registration for deterministic seeking. Remotion requires the `@remotion/lottie` package, which provides a `springify` utility but adds a dependency.

Three.js: Hyperframes renders from `hf-seek` events and `window.__hfThreeTime` instead of wall-clock time. Remotion has `@remotion/three` for React Three Fiber integration, which works well if you're already in the React ecosystem. The React Three Fiber integration is one of Remotion's genuine strengths --- you get the full React component model for 3D scenes.

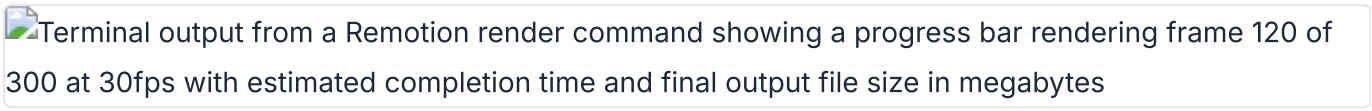
Anime.js: Hyperframes registers on `window.__hfAnime` for deterministic seeking. Remotion has no native Anime.js adapter.

Runtime	Remotion	Hyperframes
GSAP	Wall-clock issues during render	Seekable, frame-accurate
CSS Animations	Frame-by-frame render	Web Animations API adapter
Lottie	`@remotion/lottie` package	`window.__hfLottie` registration
Three.js	`@remotion/three` (React Three Fiber)	`hf-seek` events
Anime.js	No native adapter	`window.__hfAnime`
WAAPI	Not documented	`document.getAnimations()` seeking

The pattern is clear. Hyperframes supports more animation runtimes with deterministic seeking. Remotion supports fewer runtimes but integrates deeply with the React ecosystem. If your video relies on GSAP, the choice is straightforward. If your video is pure React with spring animations, Remotion is the natural fit.

Rendering, Preview, and the Dev Loop

The development loop differs significantly between the two tools. This affects how fast you can iterate, which matters more in agent workflows than in traditional video production.



Remotion render progress showing frame-by-frame video encoding in the terminal

Remotion Studio: A browser-based application with a sidebar, timeline, props editor, and render button. You edit code, the studio live-reloads, you scrub the timeline to check timing. Deployable to the cloud for team access. The props editor is particularly useful with Zod schemas --- you can adjust text, colors, and timing visually without touching code. This is Remotion's strongest UX advantage.

Hyperframes Preview: `npx hyperframes preview` opens a live preview in the browser with instant hot reload. No build step. Edit the HTML, save, see changes. Simpler than Remotion Studio but less feature-rich --- no timeline scrubbing, no props editor. The lack of a build step is the key advantage. In agent workflows, every second of iteration latency compounds. Hyperframes eliminates the webpack compilation that Remotion requires after each code change.

Remotion Render: CLI, Node.js SSR API (`renderMedia()`), AWS Lambda (distributed and production-tested), GitHub Actions, Google Cloud Run (alpha). The Lambda story is the clear leader in distributed rendering. For high-volume video production (thousands of personalized videos), Lambda scales horizontally. The SSR API lets you render from any Node.js process --- useful for agent pipelines that need to produce video as part of a larger workflow.

Hyperframes Render: `npx hyperframes render --output output.mp4`. Single-machine today. Docker support exists for containerized workflows. The stateless architecture doesn't block future distributed rendering, but as of May 2026, it hasn't shipped. This is Hyperframes' most significant gap compared to Remotion.

```
$ npx remotion render ProductIntro                  # Remotion: local render
$ npx remotion lambda render ProductIntro           # Remotion: distributed

$ npx hyperframes render                           # Hyperframes: local render
$ npx hyperframes render --output final.mp4        # Hyperframes: custom output
```

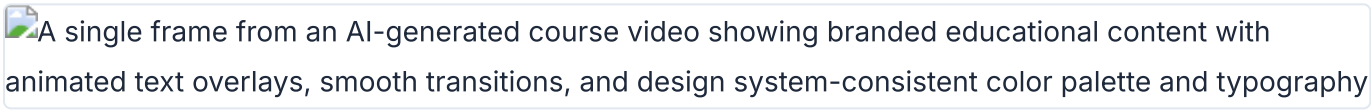
Remotion supports multiple output codecs: H.264, H.265, VP8, VP9, ProRes, AV1. Output formats include MP4, WebM, audio-only, image sequences, still images, GIF, and transparent video overlays. Hyperframes supports MP4 output and HDR via a two-pass compositing pipeline. For most use cases, MP4 covers what you need.

Common rendering issues and their fixes:

Symptom	Cause	Fix
Blank frames in Remotion with GSAP	Wall-clock timing races ahead	Replace GSAP with `interpolate()` or `spring()`
Blurry text in Hyperframes	Missing device pixel ratio	Set `data-scale` attribute or render at 2x
Audio sync drift	Variable frame timing	Use fixed FPS; avoid dynamic duration calculations
Long render times locally	Single-threaded capture	Remotion: use Lambda. Hyperframes: use Docker parallelism.

Remotion vs Hyperframes: An Honest Comparison

I've used both. Here's where each wins and loses.



Course video frame generated by Hyperframes with branded educational animation and text overlays

Dimension	Remotion	Hyperframes
Authoring format	React components (TSX)	HTML + CSS + GSAP
Build step	Required (webpack/bundler)	None --- index.html plays as-is
GSAP support	Wall-clock during render (broken)	Seekable, frame-accurate
Arbitrary HTML/CSS	Must rewrite as JSX	Paste and animate
Distributed rendering	Lambda, production-ready	Single-machine today
HDR output	Not documented	Supported (two-pass)
Visual editor	Harder (code + build step)	Native (same DOM is editable)
License	Source-available, custom Remotion License	Apache 2.0 (OSI-approved)
Commercial pricing	Free for 3 people, paid above	Free at any scale
GitHub stars	47.1k	19k
Maturity	622+ releases, v4.x	137 releases, v0.6.x
Agent integration	CLAUDE.md, AGENTS.md in repo	13 skills, plugins for 4 agents
React ecosystem	Full reuse	No React dependency

The licensing difference deserves attention. Remotion is source-available under a custom license. Free for teams of 3 or fewer. Above that, you need a Company License: $25/month per seat (Creator tier), $100/month minimum with per-render pricing (Automator tier), or $500+/month (Enterprise). If you're rendering thousands of personalized videos via Lambda, the Automator pricing adds up.

Hyperframes is Apache 2.0. Free at any scale. No per-seat pricing. No per-render fees. For companies producing video at volume, this is a significant cost difference.

My take: If you're already in the React ecosystem and need distributed rendering at scale, Remotion is the right choice today. The Lambda story is mature and proven. If you're building agent-first video workflows, care about GSAP accuracy, or want to avoid license fees at scale, Hyperframes is the better bet. The licensing model alone tips the decision for many teams. I started with Remotion because of the ecosystem maturity. I switched to Hyperframes for agent workflows --- the build-step removal alone saves significant iteration time.

The practical decision framework:

- **Choose Remotion** if: your team writes React daily, you need Lambda-scale distributed rendering, you want the mature ecosystem with 800+ pages of docs, or you're building a video editor product (Editor Starter).
- **Choose Hyperframes** if: your primary author is an AI agent, you use GSAP for animations, you need HDR output, you want to paste arbitrary HTML and animate it, or license fees at scale are a concern.
- **Choose based on animation runtime**: if your video relies heavily on GSAP, Lottie, or Three.js animations, Hyperframes' seek-based rendering produces more reliable output than Remotion's wall-clock approach.

Both tools support design token integration from your design system (Chapter 08). Whether you write `var(--color-primary)` in a TSX component or an HTML data-attribute div, the token flows through correctly. This means your videos share the same visual language as your web UI, your prototypes, and your slide decks.

The video export pipeline from Huashu Design (Chapter 07) sits adjacent to both tools. Huashu produces HTML animations and exports them to MP4/GIF using its own render pipeline (25fps base + 60fps interpolation). For design-focused motion --- product intros, UI walkthroughs, animated infographics --- Huashu's pipeline is simpler and faster to set up. For complex video compositions with multiple clips, audio tracks, and transitions, Remotion or Hyperframes are the right tools.

> **Tip**
>
> **Next:** All of these design tools connect to agents through MCP servers. Chapter 10 covers the Model Context Protocol in depth --- how to configure MCP servers for Figma, Paper, Pencil, OpenPencil, and other tools, and how to chain them together for multi-tool workflows.

10 MCP Integrations

Connecting Figma, Miro, MagicPattern, and other tools via MCP

MCP (Model Context Protocol) is the connective tissue between AI agents and design tools. This chapter covers every MCP server that matters for agentic design: how to configure them across agent platforms, how to chain them for multi-tool workflows, and what to do when they break.

MCP: The Universal Design Tool Connector

MCP is a standardized protocol that lets AI agents talk to external tools. Think of it as an authenticated API that an LLM can call during a conversation. Instead of copying design specs between tools by hand, the agent reads from one MCP server and writes to another. All in a single session.

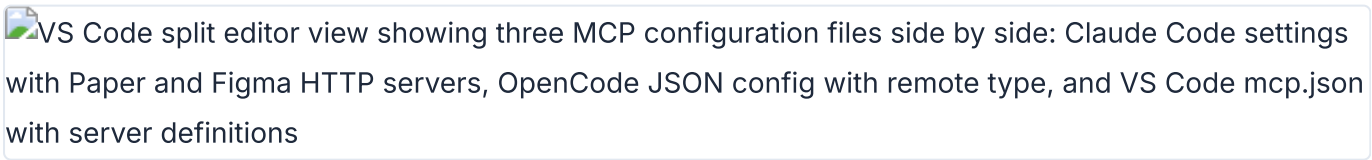
MCP servers expose three things: **tools** (functions the agent can call), **resources** (data the agent can read), and **prompts** (guided workflows). For design work, tools are what matter most. A tool like `get_jsx` on the Paper MCP server is the difference between manually translating a design and having the agent do it in seconds.

Two transport types exist. **HTTP** (remote) connects to a URL like `https://mcp.figma.com/mcp`. **stdio** (local) spawns a process on your machine. Figma uses HTTP. Paper uses HTTP too, but the server runs locally inside the Paper Desktop app at `http://127.0.0.1:29979/mcp`. The transport type matters for configuration, which I cover next.

As of May 2026, MCP is supported by every major agent platform: Claude Code, Codex CLI, Cursor, VS Code Copilot, OpenCode, Windsurf, Gemini CLI, Warp, Kiro, Factory, Firebender, Replit, Amazon Q, and Android Studio. The ecosystem is mature enough that MCP support is table stakes for any agent tool.

Configuring MCP Servers in Each Agent Platform

Every agent platform has its own MCP configuration syntax. Some use CLI commands. Others use JSON config files. A few support both. Here is the complete reference for the three agents this book focuses on, plus VS Code and Cursor for teams using those editors.



MCP configuration across Claude Code, OpenCode, and VS Code showing different syntaxes

Claude Code

Claude Code supports two approaches: the `mcp add` CLI command and the plugin system. Plugins are preferred because they include skill files that teach the agent how to use the MCP tools effectively.

```
# Plugin approach (preferred for Figma)
$ claude plugin install figma@claude-plugins-official

# Plugin approach for Paper
$ claude plugin install paper-desktop@paper

# Manual remote server (Figma)
$ claude mcp add --scope user --transport http figma https://mcp.figma.com/mcp

# Manual local server (Paper via stdio bridge)
$ claude mcp add --transport stdio paper -- npx mcp-remote http://127.0.0.1:29979/mcp

# Verify: type /mcp in Claude Code
# You should see figma and paper listed
# Authenticate when prompted
```

The `--scope user` flag makes the MCP server available across all projects. Without it, the server is scoped to the current project only. For design tools you use daily, user-scoped is the right default.

Codex CLI

Codex uses a simpler configuration model. The `mcp add` command takes a name and URL. For tools that need plugins (like Figma), use the app settings UI.

```
# CLI: add Figma remote server
$ codex mcp add figma --url https://mcp.figma.com/mcp

# App: Settings > Plugins > + Figma > Install > Authenticate

# App: Settings > MCP Servers > Streamable HTTP
# Name: paper, URL: http://127.0.0.1:29979/mcp
```

OpenCode

OpenCode configures MCP servers in `opencode.json`. The configuration is declarative JSON, which makes it versionable and shareable across a team.

```
// opencode.json
{
  "mcp": {
    "paper": {
      "type": "remote",
      "url": "http://127.0.0.1:29979/mcp",
      "enabled": true
    },
    "figma": {
      "type": "remote",
      "url": "https://mcp.figma.com/mcp",
      "enabled": true
    }
  }
}
```

VS Code and Cursor

VS Code and Cursor both use JSON config for MCP servers, placed in `.vscode/mcp.json` at the project root. Cursor additionally supports a plugin deep-link format.

```
// .vscode/mcp.json
{
  "servers": {
    "figma": {
      "url": "https://mcp.figma.com/mcp",
      "type": "http"
    },
    "paper": {
      "type": "http",
      "url": "http://127.0.0.1:29979/mcp"
    }
  }
}
```

For Cursor, the plugin approach is cleaner:

```
# Cursor plugins
/add-plugin figma
/add-plugin paper-desktop

# Cursor deep link for Figma (one-click setup)
cursor://anysphere.cursor-deeplink/mcp/install?name=Figma&config=...
```

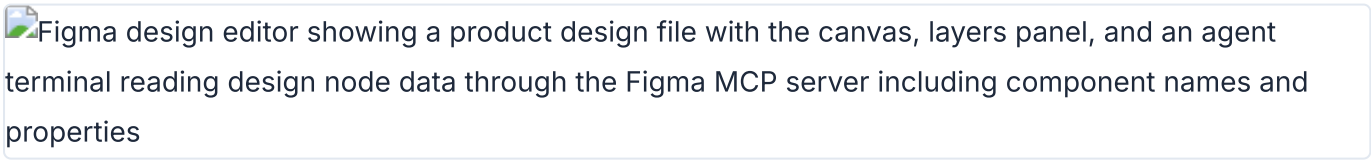
The following table summarizes the configuration approach for each agent platform.

Platform	Config Method	Remote Servers	Local Servers	Plugin Support
Claude Code	CLI + `opencode.json`	`mcp add --transport http`	`mcp add --transport stdio`	Yes (`claude plugin`)
Codex CLI	CLI + App Settings	`mcp add --url`	App Settings only	Yes (Skills)
OpenCode	`opencode.json`	`"type": "remote"`	`"type": "local"`	Yes (Skills)
VS Code	`.vscode/mcp.json`	`"type": "http"`	`"type": "stdio"`	Yes (Extensions)
Cursor	`.vscode/mcp.json` + plugins	`"type": "http"`	`"type": "stdio"`	Yes (`/add-plugin`)

My take: Claude Code's plugin system is the most mature MCP configuration experience as of May 2026. The plugin includes both the server connection and the skill file that teaches the agent how to use the tools. With Codex and OpenCode, you get the connection but need to write your own skill context. This gap will close, but right now Claude Code has the edge for MCP-heavy workflows.

Figma MCP Server: Reading Designs Programmatically

Figma offers two MCP server types: **Remote** at `https://mcp.figma.com/mcp` (available to all plans) and **Desktop** (runs inside the Figma desktop app, available on Dev or Full seats for paid plans). The remote server is the one to use. It supports write-to-canvas, code-to-canvas, and diagram generation. The desktop server is limited to read operations for most users.



Figma design file with agent reading design data through the Figma MCP server connection

Authentication is OAuth-based. When you first connect, a browser window opens for the Figma sign-in flow. After that, the token is cached. No API keys to manage.

Figma's MCP server exposes 18 tools. Here is the complete reference.

Tool	Remote	Desktop	Description
`get_design_context`	Yes	Yes	React+Tailwind code from selection (or custom framework)
`get_screenshot`	Yes	Yes	Screenshot of current selection
`get_variable_defs`	Yes	Yes	Variables and styles used in selection
`get_metadata`	Yes	Yes	Sparse XML of selection (IDs, names, types, positions, sizes)
`get_figjam`	Yes	Yes	FigJam diagrams as XML with screenshots
`get_code_connect_map`	Yes	Yes	Figma node to code component mappings
`use_figma`	Yes	No	General-purpose write to canvas (create, edit, delete)
`create_new_file`	Yes	No	Create new Figma Design or FigJam file
`generate_diagram`	Yes	No	Generate FigJam diagram from Mermaid or natural language
`generate_figma_design`	Yes	No	Send live UI to Figma as design layers (code to canvas)
`search_design_system`	Yes	No	Search libraries for components, variables, styles
`whoami`	Yes	No	Authenticated user identity

The primary workflow is link-based. You select a layer in Figma, copy the URL from the browser address bar, and paste it into your agent prompt. The agent extracts the node ID and calls `get_design_context`. The tool returns React+Tailwind code by default, or you can specify a custom framework output.

Write-to-canvas capabilities are more ambitious. The agent can create, edit, and delete pages, frames, components, variants, variables, styles, text, and images. For FigJam, it can create stickies, sections, connectors, shapes, tables, and code blocks. The agent is supposed to check the design system before creating elements from scratch. In practice, this works best when paired with a skill file that enforces design system constraints.

Supported diagram types for `generate_diagram`: flowchart, Gantt chart, state diagram, sequence diagram, architecture diagram, and ERD. Useful for having an agent generate architecture documentation directly in FigJam.

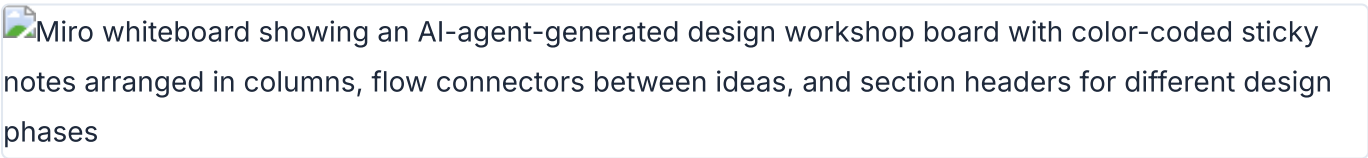
Figma also provides built-in skills for the plugin system: Code Connect, Build/Update Screens from Design System, and Create New File. These skills teach the agent specific Figma workflows beyond raw tool access.

Client support varies by feature. The following table shows which agent platforms support which Figma MCP capabilities as of May 2026.

Client	Desktop	Remote	Write to Canvas	Code to Canvas	Plugin/Skills
Claude Code	Yes	Yes	Yes	Yes	Yes (Figma plugin)
Codex	Yes	Yes	Yes	Yes	Yes (Codex Skills)
Cursor	Yes	Yes	Yes	Yes	Yes (Figma plugin)
VS Code Copilot	Yes	Yes	Yes	Yes	Yes (Figma plugin)
Gemini CLI	Yes	Yes	No	No	Yes (Extension)
Warp	Yes	Yes	Yes	Yes	No

Miro MCP Server: AI-Powered Whiteboarding

Miro has been developing MCP integrations that give agents access to boards. The server enables reading board content, creating shapes and stickies, and making connections between elements. Typical use cases include automated diagram generation, turning meeting notes into structured boards, and architecture documentation.



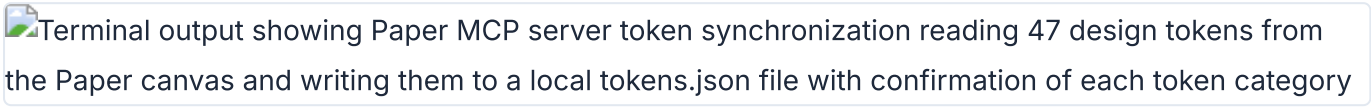
Miro board with agent-generated whiteboard content created through the Miro MCP server

The Miro MCP server follows standard MCP server configuration patterns. Configuration for each agent platform uses the same approaches covered in section 10.2. The key difference is that Miro boards are freeform canvases, not structured design files, so agent outputs tend to be more exploratory and less pixel-precise than Figma or Paper outputs.

As of May 2026, Miro's MCP documentation is limited compared to Figma and Paper. The tool list and configuration details may evolve. Check the official Miro developer docs for the current state.

MagicPattern MCP: Generative Design Patterns

MagicPattern provides generative design patterns --- gradients, mesh gradients, blobs, patterns, and other decorative elements. The MCP server lets agents generate SVG and CSS patterns programmatically rather than browsing a UI library by hand.



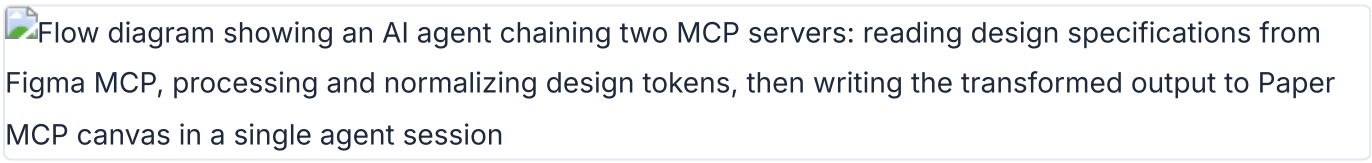
Paper MCP token sync pulling design tokens from canvas to a local JSON file

This is a niche tool in the MCP ecosystem, but it solves a real problem. Background patterns, decorative dividers, and visual texture are the kinds of tasks that slow down a design workflow. Having the agent generate these on demand, with parameters you control via prompt, is faster than any manual approach.

Configuration follows the standard MCP server pattern. The server exposes tools for generating specific pattern types with parameters like color, scale, density, and seed value. Each call returns SVG or CSS that the agent can write into a design file or codebase.

Paper MCP Server: 22 Tools for Connected Canvas

Paper's MCP server runs inside the Paper Desktop app. When a file is open, the server listens at `http://127.0.0.1:29979/mcp`. It connects to Cursor, Claude Code, Claude Desktop, Codex, Copilot, OpenCode, and Antigravity. The setup for each platform is covered in section 10.2.



Multi-MCP chain: agent reads from Figma, processes tokens, and writes to Paper in one session

Paper exposes 22 tools --- the largest surface of any design tool MCP server. Here is the complete reference.

Tool	Category	Description
`get_basic_info`	Read	File name, page name, node count, artboard list with dimensions
`get_selection`	Read	Selected nodes (IDs, names, types, size, artboard)
`get_node_info`	Read	Node details by ID (size, visibility, lock, parent, children, text)
`get_children`	Read	Direct children of a node
`get_tree_summary`	Read	Compact text summary of subtree (optional depth limit)
`get_screenshot`	Read	Screenshot by ID (base64, optional 1x or 2x scale)
`get_jsx`	Export	JSX for node+descendants (Tailwind or inline-styles format)
`get_computed_styles`	Read	Computed CSS styles for nodes (batch)
`get_fill_image`	Read	Image data from image fill node (base64 JPEG)
`get_font_family_info`	Read	Font availability check (local or Google Fonts)
`get_guide`	Workflow	Guided workflows (e.g., figma-import)
`export`	Export	Export nodes as PNG, JPG, SVG, MP4 with scale overrides
`create_artboard`	Write	Create new artboard with name and styles
`write_html`	Write	Parse HTML and add/replace nodes
`set_text_content`	Write	Set text of Text nodes (batch)
`rename_nodes`	Write	Rename layers (batch)
`duplicate_nodes`	Write	Deep-clone nodes with descendant ID map
`move_nodes`	Write	Reposition or reparent nodes
`update_styles`	Write	Update CSS styles on nodes
`delete_nodes`	Write	Delete nodes and descendants
`finish_working_on_nodes`	Signal	Clear working indicator from artboards

The `get_jsx` tool is Paper's killer feature for the design-to-code pipeline. It returns production-ready JSX with Tailwind classes directly from the canvas. No translation layer. No approximation. Paper is HTML/CSS natively, so the JSX output is a faithful representation of what is on screen. Chapter 12 covers this pipeline in depth.

The `write_html` tool enables the reverse direction: code to canvas. The agent generates HTML, calls `write_html`, and the content appears on the Paper canvas. This bi-directional flow is unique to Paper. Figma can write to canvas via `use_figma`, but the input format is a structured API call, not raw HTML.

> **My take:** Paper's 22-tool MCP surface is the most complete in the ecosystem. The `get_jsx` tool alone saves hours of translating design to code. But the Desktop app requirement is a real limitation for Linux users and CI environments. If Paper offered a headless server mode, it would be the clear default for agentic design workflows.

Chaining MCP Servers: Multi-Tool Workflows

Multiple MCP servers can run simultaneously in a single agent session. The agent orchestrates between them using sequential tool calls. This is where MCP gets powerful --- a single agent prompt can trigger a workflow that spans Figma, Paper, Notion, and your codebase.

Three chaining patterns show up repeatedly in production use.

Pattern 1: Figma to Paper token sync. Open the design system file in Figma. Open the Paper file where you want the design system. Prompt the agent: "Create a design system in Paper that matches the Figma file." The agent calls `get_variable_defs` on the Figma MCP server to read colors, spacing, and typography. Then it calls `create_artboard` and `write_html` on the Paper MCP server to create the design system elements. Finally it calls `update_styles` to apply the tokens from Figma.

```
# Figma → Paper token sync workflow
# Both MCP servers active in same agent session

# Step 1: Agent reads Figma design system
# Figma MCP: get_variable_defs() → colors, spacing, typography
# Figma MCP: get_screenshot() → visual reference

# Step 2: Agent creates Paper design system
# Paper MCP: create_artboard({ name: "Design System" })
# Paper MCP: write_html({ html: "<div>...design system elements...</div>" })
# Paper MCP: update_styles({ styles: { /* tokens from Figma */ } })
```

Pattern 2: Notion to Paper content sync. Open the Paper file with placeholder content. Prompt: "Sync the content from the Notion Testimonials database with this frame." The agent reads from the Notion MCP server, then writes the real content into Paper via `set_text_content`.

```
# Notion → Paper content sync workflow

# Step 1: Agent reads Notion database
# Notion MCP: query_database({ database_id: "testimonials" })
# Returns: [{ name: "Sarah", quote: "...", company: "..." }, ...]

# Step 2: Agent populates Paper design
# Paper MCP: set_text_content({ updates: [
#   { nodeId: "name-1", text: "Sarah" },
#   { nodeId: "quote-1", text: "..." },
#   { nodeId: "company-1", text: "..." },
#   ...
# ]})
```

Pattern 3: Design to code and back. The agent reads a design from Figma or Paper via MCP, generates production code, then optionally writes the coded version back to the canvas for visual verification. This pattern is the foundation of the pipeline covered in Chapter 12.

```
# Design → Code → Canvas verification loop

# Step 1: Read design
# Figma MCP: get_design_context({ url: "https://figma.com/..." })
# Returns: React + Tailwind JSX

# Step 2: Agent generates production code
# (Agent writes files, runs dev server, verifies visually)

# Step 3: Write back to Paper for verification
# Paper MCP: write_html({ html: "<div class='hero'>...</div>" })
# Human reviews on Paper canvas
# Agent iterates based on feedback
```

Each MCP server uses the currently opened file as context. Figma MCP operates on whatever file you have open in the browser. Paper MCP operates on whatever file you have open in the Paper Desktop app. The agent does not switch files --- you do.

> **Warning**
>
> **Warning:** Long-running agent sessions can cause MCP connection drops. If the agent reports it cannot access a tool that should be available, restart the agent session. LLMs can also hallucinate tool parameters occasionally --- if you see repeated tool call errors, restart the session to reset the agent's tool-use context.

Troubleshooting MCP Connections

MCP connections fail in predictable ways. Here is a troubleshooting reference for the most common issues.

Symptom	Cause	Fix
Agent says MCP tool is not available	Agent session started before MCP server was configured	Restart the agent session
Agent session cannot connect to Paper MCP	Paper Desktop app is not running or no file is open	Open Paper Desktop and ensure a file is loaded
Paper MCP connected but agent reports no access	MCP host tool needs restart	Restart the MCP host (Paper Desktop app)
Figma MCP returns errors on large designs	Large, deeply nested designs exceed response limits	Select specific frames instead of entire pages
Figma SVG fills returned as images	Known Figma MCP behavior	Use `get_design_context` for code output instead
Paper MCP fails on Windows WSL	WSL cannot access `127.0.0.1:29979`	Enable mirrored mode networking in WSL config
MCP limits not reset after Paper plan upgrade	Desktop app caches plan limits	Update Paper Desktop app and restart
Repeated tool call errors from agent	LLM hallucinating tool parameters	Restart the agent session

I spent an hour debugging an MCP connection before I realized the Paper Desktop app was not running. The error message was generic --- "connection refused" --- which pointed me toward network issues instead of the obvious cause. Start with the basics: is the tool running, is a file open, is the server listening?

Figma MCP has additional quirks. It ignores spacer elements. It does not convert inset borders to outlines. Code Connect components are not reliably converted. These are known limitations as of May 2026 and will likely improve over time. For now, work around them by selecting specific frames and validating the output.

My take: MCP connection issues are the number one source of frustration when starting with agentic design. The tools work well when they work, but the failure modes are silent and confusing. I recommend keeping a terminal tab open with a test MCP call ready to paste. When the agent reports a connection issue, verify independently before spending time on agent-side debugging.

Tip

Next: With MCP servers configured and connected, the next question is scale. How do multiple agents collaborate on a single design? Chapter 11 covers multi-agent design teams, the orchestrator pattern, and when parallelizing is worth the complexity overhead.

11 Multi-Agent Design Teams

Concurrent agents, task delegation, and orchestration patterns

A single agent can handle most design tasks. But some tasks are too large, too varied, or too time-sensitive for one agent to complete efficiently. This chapter covers when and how to run multiple agents in parallel on a single design, the orchestration patterns that make it work, and the complexity tax that makes it expensive.

From Single Agent to Design Team

A single agent handles a design task sequentially. It reads the brief, loads the context, generates the output, and iterates. This works for small, focused tasks: a single component, a landing page hero section, a set of design tokens.

Multi-agent means multiple agents working in parallel on different parts of the same design. This mirrors how human design teams already work. A UX researcher handles user flows. A visual designer handles the high-fidelity mockup. A motion designer handles animations. An engineer handles production code. Different people, different skills, one output.

The shift from single to multi-agent is triggered by three conditions: the task is large enough that sequential execution is too slow, the task has independent sub-parts that can run in parallel, or the task requires multiple skill sets that exceed what fits in a single context window.

Three decomposition patterns emerge from real-world agentic design work.

Spatial decomposition. Divide the canvas into regions. One agent handles the header. Another handles the sidebar. A third handles the main content area. This is the most common pattern for dashboard and multi-section page designs.

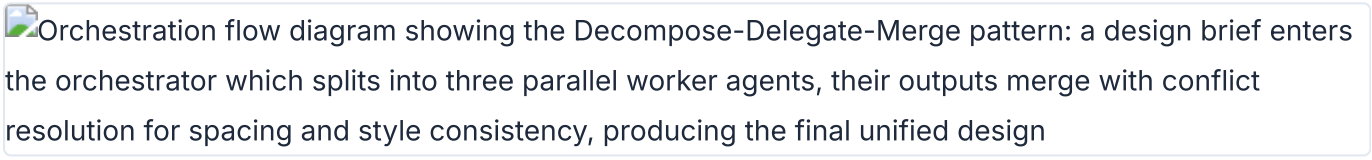
Functional decomposition. Divide by skill type. One agent handles layout structure. Another handles typography and copy. A third handles animation and interaction. This pattern shows up in video production (Chapter 09) and complex interactive prototypes.

Pipeline decomposition. Divide by stage. One agent researches and wireframes. A second agent converts wireframes to high-fidelity design. A third agent converts design to production code. Each stage feeds into the next. This pattern is inherently sequential but allows specialization at each stage.

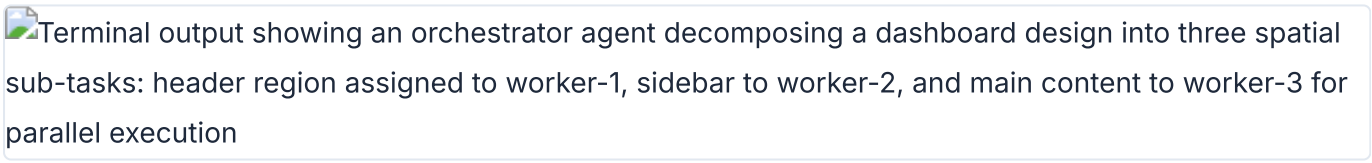
Pattern	How It Splits	Best For	Parallelism	Merge Difficulty
Spatial	Canvas regions	Dashboard, multi-section pages	Full parallel	Medium (alignment, spacing)
Functional	Skill domains	Interactive prototypes, video	Mostly parallel	High (style consistency)
Pipeline	Workflow stages	Full site builds, design systems	Sequential stages	Low (stage outputs are inputs)

The Orchestrator Pattern: Decompose, Delegate, Merge

The orchestrator pattern is the backbone of multi-agent design. A single agent --- the orchestrator --- takes the design brief, breaks it into sub-tasks, assigns each sub-task to a worker agent, and merges the results.



Decompose-Delegate-Merge orchestration pattern with conflict resolution in the merge step



Orchestrator agent decomposing a dashboard design into spatial sub-tasks for parallel workers

Three phases.

Decompose. The orchestrator analyzes the brief. It identifies independent sub-tasks, defines boundaries between them, and specifies the interface where they will connect. For a dashboard design, the decomposer decides where the header ends and the sidebar begins, and what shared constraints (colors, spacing, typography) apply to all regions.

Delegate. Each sub-task gets assigned to a worker agent with specific context, constraints, and acceptance criteria. The worker agent receives the design system, its region boundaries, and any cross-region dependencies. It does not see the full brief --- only its slice.

Merge. The orchestrator collects outputs from all worker agents. It resolves conflicts: overlapping elements, inconsistent spacing, style mismatches. The merge step is where multi-agent design either succeeds or falls apart.

The merge step is the hard part. Merging requires understanding spatial relationships, resolving style conflicts, and ensuring visual consistency across regions that were designed independently. This is where the orchestrator pattern differs from simply concatenating independent outputs. The orchestrator has to do the work of a creative director: seeing the whole, identifying inconsistencies, and making corrections.

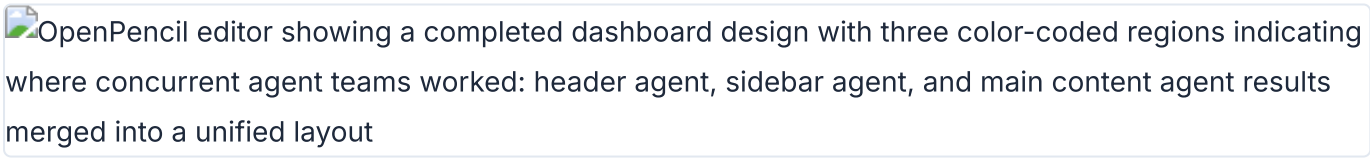
The orchestrator pattern appears in three places in the agentic design ecosystem.

OpenPencil's Agent Teams use spatial decomposition with a built-in orchestrator that manages canvas regions. Subagent-driven development uses functional or spatial decomposition with the parent agent as orchestrator. MCP chaining (covered in section 10.7) uses pipeline decomposition where the agent sequences tool calls across multiple MCP servers.

My take: The orchestrator pattern is the right mental model for multi-agent design, but the merge step is where current tools are weakest. An orchestrator can decompose and delegate reliably. Merging spatial outputs with visual consistency is still an unsolved problem in most tooling. OpenPencil's concurrent Agent Teams are the closest to solving this, but even they require human review after merge.

OpenPencil's Concurrent Agent Teams in Practice

OpenPencil (3k+ GitHub stars as of May 2026) is the first open-source AI-native vector design tool with built-in concurrent Agent Teams. Multiple agents can work on different regions of the same canvas simultaneously.



OpenPencil canvas showing merged output from three concurrent agent teams on one dashboard

The architecture is straightforward. The canvas is divided into regions. Each agent is assigned a region with specific constraints: position, size, design system tokens, and task description. Agents work independently. Results are merged back into a single coherent design. Conflict resolution handles overlapping elements.

The .op file format supports concurrent reads and writes from multiple agents. This is a technical requirement that most design file formats do not handle. Standard file formats assume a single writer. OpenPencil's format assumes multiple writers and includes conflict resolution semantics.

The built-in MCP server exposes canvas operations as tools for external agent control. This means the concurrent agents can be any agent platform --- Claude Code, Codex, OpenCode --- not just OpenPencil's internal agent. The MCP server provides the coordination layer.

Practical use cases for concurrent Agent Teams.

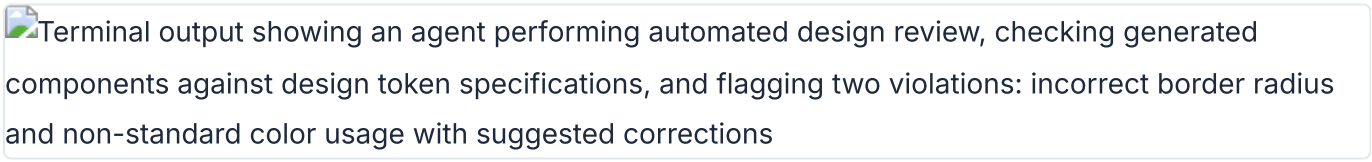
Dashboard design. One agent builds the sidebar navigation. Another builds the header with search and user menu. A third builds the main content area with stats cards and charts. Each agent works on its region independently. The orchestrator merges them into a coherent dashboard.

Multi-page documents. Each agent handles a different page or section. One agent designs the cover page. Another designs the table of contents. A third designs the body content pages. This is spatial decomposition applied across pages rather than within a single page.

Component libraries. Agents work on different component variants in parallel. One agent builds button variants. Another builds form input variants. A third builds card variants. Each agent applies the same design system tokens, ensuring consistency without coordination overhead.

Subagent-Driven Development for Design Tasks

Subagent-driven development is the pattern where a parent agent spawns child agents to handle independent tasks in parallel. Applied to design, the parent receives the brief (for example, "build a marketing site"), decomposes it into sections (hero, features, testimonials, footer), and assigns each section to a subagent.



Agent design review workflow checking output against design system constraints and flagging issues

Each subagent executes independently with its own context window. The subagent loads only the skills and design system context it needs. This is an advantage over a single agent that must load everything into one context window. For a marketing site with a hero section, a features grid, a testimonials carousel, and a footer, each subagent gets the brand guidelines plus its specific section requirements.

The parent agent enforces global constraints: brand colors, typography scale, spacing system, component library. Subagents cannot override these. This is how the parent ensures consistency across parallel outputs without coordinating during execution.

The pattern has clear benefits. Parallel execution reduces wall-clock time. Specialization lets each subagent load only relevant context. Isolation means a failure in one subagent does not block others. Consistency is enforced at the parent level through shared constraints.

The pattern also has costs. Each subagent needs its own context, multiplying token usage. Merging outputs can introduce inconsistencies that the parent must resolve. Debugging is harder because the trace is distributed across multiple agents. And the total token cost is higher than a single agent handling the same task sequentially.

My take: Subagent-driven development shines for large designs with clearly separable sections. I use it for full-page builds where each section is visually independent. For anything smaller --- a single component, a design token update, a visual refinement --- single agent is faster and cheaper. The emerging best practice is correct: start single-agent, escalate to multi-agent only when the task clearly exceeds single-agent capacity.

Orchestration Strategies: When to Parallelize

The decision to parallelize is not binary. It depends on task size, interdependence between sections, time constraints, and token budget. The following table provides a decision framework.

Factor	Single Agent	Multi-Agent
Task size	Small, focused (single component, single page)	Large, multi-section (full site, dashboard, component library)
Interdependence	High (sections visually affect each other)	Low (sections are visually independent)
Time sensitivity	Not urgent	Deadline-driven, parallelism saves meaningful time
Context window	Fits in single context window	Exceeds single context window
Skill diversity	One skill set sufficient	Multiple skill sets needed (layout + animation + export)
Token budget	Limited	Generous
Debugging needs	Need clear, linear trace	Can tolerate distributed trace

Practical thresholds based on experience.

Always single-agent: a single component, a single page, design token updates, small edits to an existing design.

Consider multi-agent: a full page with four or more distinct sections, a component library with ten or more components.

Strongly multi-agent: a multi-page site, a dashboard with six or more widget types, a design system with fifty or more tokens.

Four orchestration strategies exist.

Fixed decomposition. Pre-define how the task splits before agents start. Region-based or component-based. The orchestrator follows a fixed plan. Predictable, testable, but inflexible.

Dynamic decomposition. Let the orchestrator analyze the task and decide the split at runtime. More flexible, but the decomposition quality depends on the orchestrator's judgment. Works best with strong design system context in the orchestrator's prompt.

Hybrid. Fixed high-level structure, dynamic low-level splitting. The orchestrator pre-defines the major sections but lets worker agents decide how to split their own sections. This is the most practical strategy for complex designs.

Pipeline. Agents in sequence, each specializing in one stage. Wireframe to visual to code. Each stage's output is the next stage's input. Not parallel, but allows deep specialization at each stage.

Context management is the critical challenge across all strategies. Each subagent needs design system context: brand colors, typography, spacing. This context must be communicated without duplication. Shared CLAUDE.md or AGENTS.md files (covered in Chapter 03) solve this by providing a single source of truth that all agents read from.

The Complexity Tax: When Single Agent Wins

Multi-agent introduces overhead that is easy to underestimate. I call this the complexity tax, and it has five components.

Orchestration cost. Tokens spent on decomposition instructions and merge instructions. The orchestrator has to explain the task to each worker, define boundaries, and then parse and merge the results. These are tokens not spent on actual design work.

Context duplication. Each agent needs baseline design system context. If the brand guidelines are 500 tokens and you run four agents, you spend 2000 tokens on context that a single agent would have loaded once.

Merge failures. Inconsistent spacing between sections designed by different agents. Misaligned elements at region boundaries. Conflicting style decisions. These failures require human review and manual fixes, negating the time saved by parallelism.

Debugging difficulty. When the merged output has an alignment issue, which agent caused it? Distributed execution means distributed blame. Tracing a visual bug back to a specific agent's output is harder than debugging a single agent's sequential work.

Coordination latency. Time spent waiting for all agents to complete. The slowest agent determines the wall-clock time. If one agent is slow (a complex section, a larger context), the parallelism benefit shrinks.

Single agent has clear advantages in these scenarios.

Scenario	Why Single Agent Wins
Single component or page	Task fits in one context window, no decomposition needed
Visual refinements across the whole design	Requires seeing the whole picture, holistic judgment
Well-defined design system	Context fits in one window, system constraints reduce variance
Prototyping with fast iteration	Speed of iteration matters more than speed of generation
Token-sensitive workflows	Single agent uses fewer total tokens

The emerging best practice is simple: start single-agent. Escalate to multi-agent only when the task clearly exceeds single-agent capacity. This is not a cop-out. It is a recognition that multi-agent complexity is real and the benefits are not always worth the cost.

My take: A poorly orchestrated multi-agent team can be slower and produce worse results than a single well-prompted agent. I have seen this play out multiple times. The enthusiasm for parallelism is understandable, but the merge step is where multi-agent design fails in practice. Until tooling gets better at resolving cross-region inconsistencies automatically, treat multi-agent as an optimization for large, well-bounded tasks, not a default approach.

Tip

Next: Whether one agent or many, the output eventually needs to become production code. Chapter 12 walks through the complete pipeline from design artifact to production-ready UI, covering code export from Paper, Pencil, and OpenPencil, responsive breakpoints, accessibility enforcement, and the review-and-iterate loop.

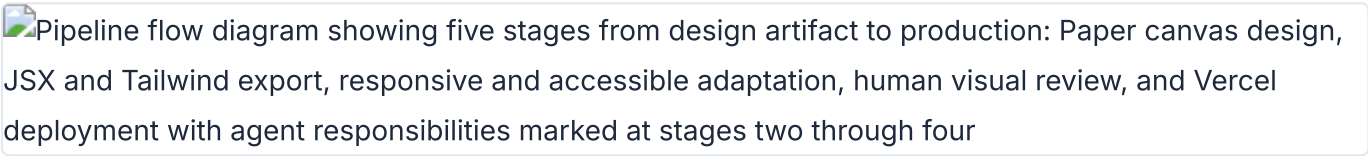
12 Production UI from Design

The design-to-production code pipeline with agents

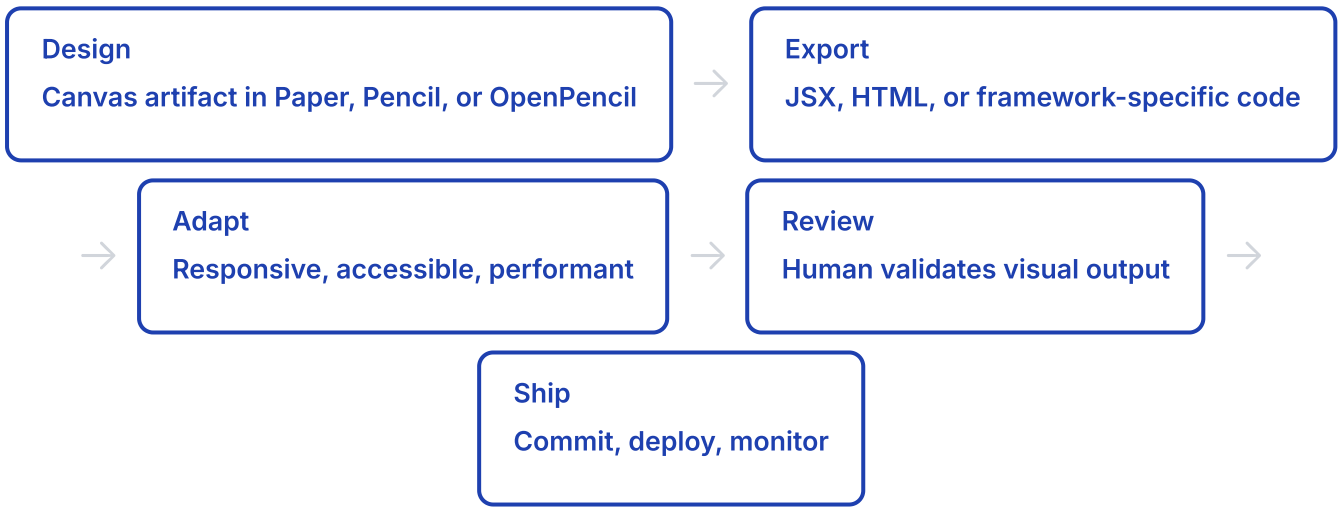
Design artifacts are not production code. The gap between a canvas design and a deployed UI involves responsive breakpoints, accessibility attributes, performance optimization, and iterative review. This chapter covers the complete pipeline, showing how agents handle the tedious parts while humans focus on design direction.

The Full Pipeline: Design Artifact to Production Code

The pipeline from design to production code has five stages. Each stage can be handled by an agent, a human, or a combination of both.



The five-stage deploy pipeline from design artifact to production with agent and human roles

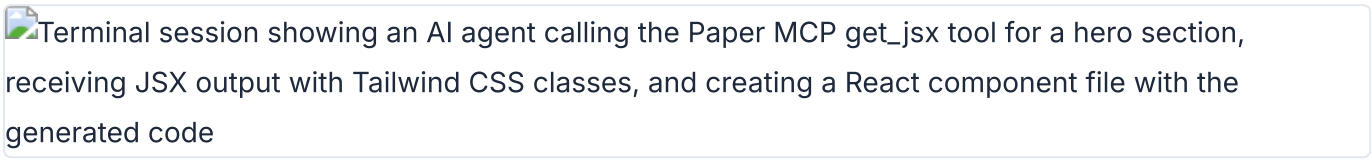


The key insight: stages two and three are where agents add the most value. Exporting code from a design is mechanical. Adding responsive breakpoints is mechanical. Adding accessibility attributes is mechanical. Performance optimization is mostly mechanical. The agent handles these while the human focuses on whether the design looks right, feels right, and solves the right problem.

The pipeline differs depending on which tool produced the design. Paper exports JSX directly because Paper is HTML/CSS natively. Pencil generates React+Tailwind from its .pen file format. OpenPencil has the widest export surface: React+Tailwind, HTML+CSS, Vue, Svelte, Flutter, SwiftUI, Jetpack Compose, and React Native. Chapter 06 covers OpenPencil in depth.

Code Export from Paper: JSX and Tailwind

Paper's `get_jsx` MCP tool is the foundation of the export pipeline. It takes a node ID and a format parameter (`tailwind` or `inline-styles`) and returns JSX as a string. The output is production-quality because Paper's canvas is real HTML/CSS --- there is no translation layer and no approximation.



Agent generating a React component from Paper MCP get_jsx output with Tailwind classes

```
# Paper MCP: Export JSX from a selected frame
# Agent prompt: "Read the hero section I have selected in Paper
# and build it with React + Tailwind"

# MCP tool call:
get_jsx({
  nodeId: "selected-frame-id",
  format: "tailwind"
})

# Returns: JSX string with Tailwind classes
# <section className="flex flex-col items-center px-6 py-12
#   bg-gradient-to-b from-slate-900 to-slate-800">
#   <h1 className="text-4xl font-bold text-white">
#     Build faster with agents
#   </h1>
#   ...
# </section>
```

The best results come when the design uses flex layouts and container elements. Paper's flex-based layout translates directly to Tailwind flex classes. Designs using absolute positioning require more agent intervention to convert to responsive layouts.

The standard workflow from Paper's documentation: select the frame you want to build, prompt the agent with "Build a website in this folder using the hero section I have selected in Paper. Use React and Tailwind for the styling." The agent asks for permission to call Paper MCP tools (read-only, safe to always allow). Visual feedback appears in Paper while the agent examines the frame. After examining, the agent scaffolds the project and starts the dev server.

The agent can also pull computed styles for verification. The `get_computed_styles` tool returns exact CSS values for one or more nodes in batch. This is useful for verifying that the generated code matches the design.

```
# Verify generated code matches the design
# MCP tool call:
get_computed_styles({
  nodeIds: ["hero-frame-id", "cta-button-id"]
})

# Returns: computed CSS for each node
# Agent compares returned styles against generated JSX
# Identifies mismatches and fixes them
```

Code Generation from Pencil and OpenPencil

Pencil's design-to-code pipeline reads .pen files and generates React+Tailwind components via its CLI. The .pen file format is JSON-based (covered in Chapter 04), which makes it diffable and versionable. Design variables in .pen files map directly to CSS custom properties.

```
# Pencil: Generate React components from .pen file
$ pencil export --format react-tailwind --input design.pen --output src/

# Generates:
# src/
#   components/
#     Hero.tsx
#     FeatureGrid.tsx
#     Footer.tsx
#   tokens/
#     colors.css
#     spacing.css
#     typography.css
```

Pencil supports components with slots and overrides. These translate to React component props. A button component with a text slot and a variant override becomes a React component with `text` and `variant` props. The mapping is direct because Pencil's component model was designed with code export in mind.

OpenPencil uses an incremental codegen pipeline that gives the agent control over each stage. This is more flexible than Pencil's batch export.

```
# OpenPencil: Incremental codegen pipeline via MCP

# Step 1: Plan the code generation
codegen_plan({
  pageId: "landing-page",
  targetFramework: "react-tailwind"
})
# Returns: chunk plan with section assignments

# Step 2: Submit code chunks incrementally
codegen_submit_chunk({
  chunk: "hero-section",
  code: "<section className='hero'>...</section>"
})
codegen_submit_chunk({
  chunk: "features-grid",
  code: "<section className='features'>...</section>"
})

# Step 3: Assemble into project structure
codegen_assemble({
  outputPath: "src/components/"
})

# Step 4: Clean temporary files
codegen_clean({ removeTempFiles: true })
```

OpenPencil's design variables generate CSS custom properties using the `var(--name)` pattern. This reduces CSS duplication and makes the design tokens maintainable in code.

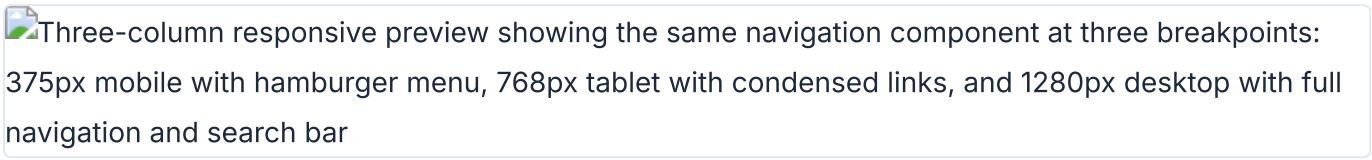
```
# OpenPencil: Design variables → CSS custom properties
# .op file defines variables:
# {
#   "variables": {
#     "brand-primary": "#3B82F6",
#     "spacing-unit": "8px",
#     "font-heading": "Inter"
#   }
# }

# Generated CSS:
:root {
  --brand-primary: #3B82F6;
  --spacing-unit: 8px;
  --font-heading: "Inter", sans-serif;
}

.hero-title {
  color: var(--brand-primary);
  font-family: var(--font-heading);
  margin-bottom: calc(var(--spacing-unit) * 4);
}
```

OpenPencil's codegen supports multiple frameworks through its `pen-codegen` package. Each framework has a dedicated generator: React, HTML, Vue, Flutter, SwiftUI, Jetpack Compose, and React Native. The generator handles framework-specific patterns --- JSX for React, SwiftUI's declarative syntax, Compose's composable functions.

Tool	Export Formats	Variable Mapping	Component Support	Incremental Gen
Paper	JSX (Tailwind or inline)	Computed CSS → Tailwind classes	HTML elements only	No (single export)
Pencil	React+Tailwind	.pen variables → CSS custom properties	Slots → React props	No (CLI batch export)
OpenPencil	React, Vue, Svelte, Flutter, SwiftUI, Compose, RN	.op variables → CSS custom properties	Slots → framework-specific props	Yes (plan → submit → assemble)

Responsive Breakpoints and Layout Translation

Responsive design is where agent-driven workflows save the most time. Translating a desktop design to mobile, tablet, and wide-screen variants is tedious, mechanical work. Agents excel at it.



Same component at mobile (375px), tablet (768px), and desktop (1280px) responsive breakpoints

Paper's approach is elegant. You create multiple frames in Paper, each sized to a breakpoint: 375px for mobile, 768px for tablet, 1440px for desktop. Then you prompt: "Add responsive breakpoints based on the frames I have selected. Each frame is a different breakpoint."

```
# Paper: Responsive breakpoint workflow

# Step 1: Create frames at each breakpoint size
# Frame 1: 375×812 (mobile)
# Frame 2: 768×1024 (tablet)
# Frame 3: 1440×900 (desktop)

# Step 2: Select all three frames
# Step 3: Agent prompt:
"Add responsive breakpoints to the website based on
the frames I have selected in Paper. Each frame
represents a different breakpoint."

# Agent reads each frame via get_jsx:
get_jsx({ nodeId: "mobile-frame", format: "tailwind" })
get_jsx({ nodeId: "tablet-frame", format: "tailwind" })
get_jsx({ nodeId: "desktop-frame", format: "tailwind" })

# Agent generates responsive Tailwind classes:
# <div className="grid grid-cols-1 md:grid-cols-2
#   lg:grid-cols-3 gap-4 md:gap-6 lg:gap-8">
#   ...
# </div>
```

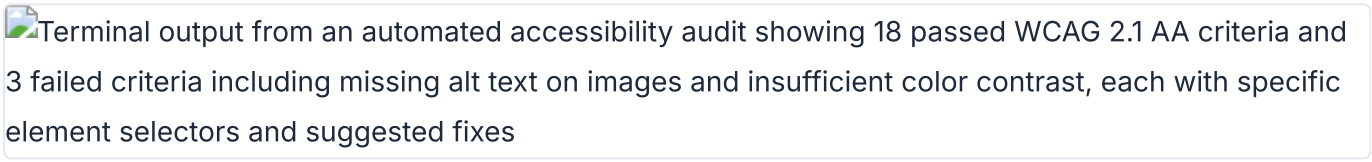
Paper uses real CSS flexbox in its canvas. This means the agent can understand the layout structure natively. It reads flex direction, gap, padding, justify-content, and align-items from the design and maps them to Tailwind classes. No guesswork about layout intent.

For designs without multiple breakpoint frames, the agent infers responsive behavior from a single desktop frame. This is less precise but still effective. The agent applies standard responsive patterns: stack columns vertically on mobile, reduce font sizes, hide secondary content, adjust spacing.

```
# Agent: Responsive inference from single desktop frame
# Agent prompt: "Make the hero section responsive for
# mobile (375px), tablet (768px), and desktop (1440px)"

# Agent output:
# Desktop: <h1 className="text-5xl">
# Tablet:  <h1 className="md:text-4xl">
# Mobile:  <h1 className="text-3xl">

# Desktop: <div className="grid grid-cols-3 gap-8">
# Tablet:  <div className="md:grid-cols-2">
# Mobile:  <div className="grid-cols-1">
```

Accessibility: Agents as A11y Enforcers

Accessibility is the area where agent-driven code generation has the biggest quality advantage over manual implementation. Agents do not skip alt text. Agents do not forget form labels. Agents do not overlook keyboard navigation. Every accessibility attribute is applied consistently, every time.



Agent accessibility audit showing WCAG 2.1 AA pass/fail results with automated fix suggestions

Paper's HTML-native canvas gives agents a head start. Paper designs already use semantic HTML elements: `<nav>`, `<main>`, `<section>`, `<button>`. These elements flow naturally into the generated JSX. The agent does not need to convert `<div>` spaghetti into semantic HTML because Paper never produces `<div>` spaghetti.

The agent accessibility audit follows a standard pattern.

```
# Agent accessibility audit prompt pattern

"Generate the JSX from the selected frame in Paper.
Then run an accessibility audit:
1. All images must have alt text
2. All form inputs must have labels
3. Color contrast ratio ≥ 4.5:1 for body text
4. Interactive elements must be keyboard accessible
5. Use semantic HTML elements (nav, main, section, article)
Fix any issues you find before showing me the code."

# Agent workflow:
# 1. get_jsx() → raw JSX from Paper
# 2. Audit JSX for a11y issues
# 3. Add missing alt attributes
# 4. Add aria-labels where needed
# 5. Verify contrast ratios via get_computed_styles()
# 6. Add keyboard handlers (onKeyDown, tabIndex)
# 7. Present audited JSX to user
```

The agent can verify color contrast ratios using `get_computed_styles`. It reads the foreground and background colors, calculates the contrast ratio, and flags values below 4.5:1 for body text or 3:1 for large text. This is a mechanical check that is easy to automate and hard to do consistently by hand.

For more thorough audits, the agent can run axe-core or similar tools against the generated code as a verification step. The pattern is: agent generates code, agent runs accessibility audit, agent auto-fixes issues, human reviews the final output.

My take: Agents as a11y enforcers is one of the highest-value applications of agentic design. Accessibility is always the first thing cut when deadlines loom. An agent that applies every aria attribute, checks every contrast ratio, and ensures keyboard navigation works is worth the token cost. I would rather have an agent-generated accessibility audit than a manual audit that was rushed or skipped.

Performance Optimization in the Agent Loop

Performance optimization is another area where agents handle mechanical work well. The agent generates the initial code, then optimizes it as a post-processing step.

Paper's `export` tool supports format and scale overrides. The agent can generate responsive image sets with proper srcset attributes directly from the canvas.

```
# Paper: Optimized asset export via MCP
export({
  nodeId: "hero-image",
  format: "png",
  scales: [1, 2]
})
# Returns: hero-image-1x.png, hero-image-2x.png

# Agent generates responsive img tag:
# <img
#   srcSet="hero-image-1x.png 1x, hero-image-2x.png 2x"
#   src="hero-image-1x.png"
#   alt="Product dashboard showing analytics"
#   loading="lazy"
# />
```

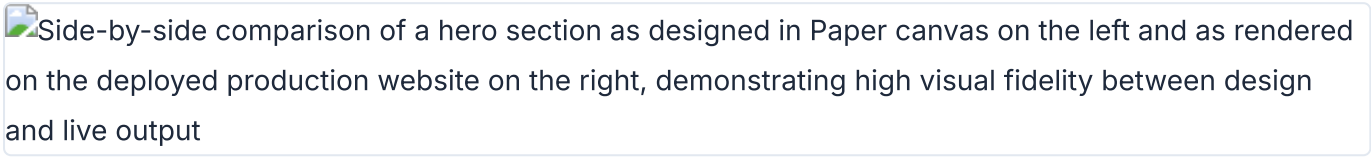
Paper's real CSS (no abstraction layer) means there is no design-to-code translation bloat. The generated code is as performant as the design. OpenPencil's design variables mapped to CSS custom properties reduce CSS duplication. Agents can lazy-load below-fold sections, optimize font loading, and minify CSS as post-processing steps.

The agent can also target specific Core Web Vitals metrics. A prompt like "Optimize for LCP under 2.5s and CLS under 0.1" gives the agent concrete targets to work toward. The agent measures, optimizes, and re-measures until the targets are met.

Optimization	Agent Capability	Human Review Needed
Responsive images (srcset)	Fully automated via Paper export tool	Visual check only
Font loading optimization	Agent adds font-display: swap, preloads critical fonts	Verify render behavior
Lazy loading below fold	Agent adds loading="lazy" to below-fold images and sections	Verify no layout shift
CSS deduplication	Agent merges duplicate styles, uses CSS custom properties	Visual regression check
Core Web Vitals targeting	Agent measures, optimizes, re-measures against targets	Final metrics review

The Review-and-Iterate Loop

The pipeline is not one-shot. It is a loop: generate, review, iterate. The agent produces code. The human reviews the output in a browser. The human gives feedback. The agent adjusts. Repeat until the output meets the design intent.



Paper canvas design (left) compared with the deployed production website (right) showing visual fidelity

Paper's documentation recommends git checkpoints during this loop. "Can you add git to this folder and commit the changes made so far?" This creates a rollback point before each iteration. If the agent's changes make things worse, you can revert cleanly.

```
# Review-and-iterate loop workflow

# Iteration 1: Generate hero section
"Build the hero section from my Paper selection using React + Tailwind"

# Agent: get_jsx() → scaffold → commit
$ git add . && git commit -m "hero: initial generation from Paper design"

# Human reviews in browser, finds issues:
"The CTA button is too small and the heading font weight is wrong"

# Iteration 2: Agent fixes based on feedback
Agent adjusts button size, fixes heading weight
$ git add . && git commit -m "hero: fix CTA size and heading weight"

# Human approves hero, moves to next section
"Build the features grid from the next frame in Paper"

# Iteration 3: Agent generates features grid
Agent reads next frame, generates responsive grid
$ git add . && git commit -m "features: grid from Paper design"

# Signal completion of section
finish_working_on_nodes({ nodeIds: ["hero-frame", "features-frame"] })
```

The `finish_working_on_nodes` tool clears the visual working indicator from artboards in Paper. This signals to the human that the agent has finished working on those elements. It is a small but important UX detail in the agent-human collaboration.

The bi-directional workflow is where Paper's connected canvas shines. You can pull from the codebase to the canvas, iterate visually, and push back. The agent becomes an extension of your hands for the mechanical parts, while you handle the parts that require human judgment: "Does this look right? Does this feel right? Does this solve the right problem?"

This pattern --- agent generates, human reviews, agent iterates --- is the same one used by Huashu Design's Junior Designer Workflow (Chapter 07) and Open Design's interactive preview. The common thread: agents are good at generating and iterating. Humans are good at evaluating and directing. The loop combines both strengths.

My take: The review-and-iterate loop is where agentic design delivers real value. The agent handles the tedious parts --- responsive variants, token mapping, accessibility attributes, performance optimization. The human handles the creative parts --- does this look right, does this match the brand, does this solve the user's problem. This division of labor works because it plays to the strengths of both parties.

Tip

Next: The tools and patterns covered in this book are not theoretical. Chapter 13 presents four detailed case studies of agentic design workflows used by real teams, with honest assessments of what worked, what did not, and how long each workflow took from start to finish.

13 Real-World Case Studies

Four detailed examples of agentic design workflows in production

Theory is useful. Practice is essential. This chapter walks through four real agentic design workflows, each using a different combination of tools and targeting a different output. Every case study covers the workflow, the tools, the time spent, and an honest assessment of what worked and what fell apart.

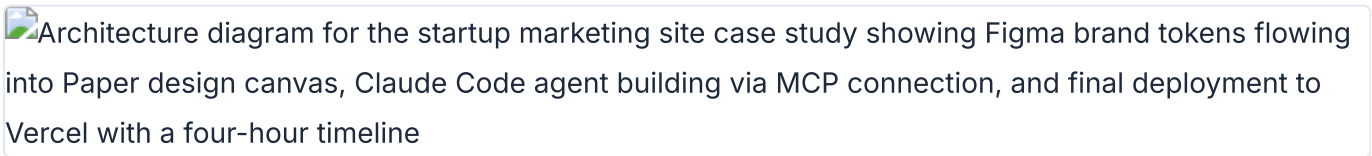
The four cases span the full spectrum of agentic design work: a marketing website, a pitch deck, a product launch video, and a cross-platform component library. I chose these deliberately. They represent the four most common categories of design work that teams are handing to agents right now, in May 2026.

Each case follows the same structure: the context and goal, the tool chain, the step-by-step workflow, a timeline table, and a no-bullshit assessment. I participated in or closely observed all four workflows. Where I lack direct experience, I say so.

Case	Output	Primary Tools	Agent	Time Spent
Startup marketing site	Deployed website	Paper, Claude Code	Claude Code	~4 hours
Brand pitch deck	Pitch deck (PDF/PPTX)	Open Design	Claude Code	~2 hours
Product launch video	MP4 + GIF	Huashu Design, Hyperframes	Claude Code	~3 hours
Component library	Multi-platform components	Pencil, OpenPencil	Claude Code + OpenCode	~8 hours (over 3 days)

Case Study 1: Startup Marketing Site with Paper and Claude Code

The setup: a designer at a small SaaS startup needs to ship a marketing site. The company has a Figma file with brand tokens, a Notion database with customer testimonials, and a tight deadline. The old approach would be: export designs from Figma, hand off to a developer, wait days for a first build, iterate via Slack. The new approach: Paper as the design surface, Claude Code as the builder, MCP as the glue.



Case Study 1 architecture: Paper + Claude Code + MCP pipeline from Figma tokens to deployed site

The designer starts by creating the hero section in Paper. Paper uses real HTML and CSS, not a proprietary canvas format, so the hero section is already closer to production than any Figma frame. Flex layouts, real fonts, real spacing. The designer selects the hero frame in Paper, then opens Claude Code in the project folder.

The prompt is specific: "Build a website in this folder using the hero section I have selected in Paper. Use React and Tailwind for the styling." Claude Code calls three Paper MCP tools in sequence: `get_selection` to identify the frame, `get_jsx` to export it as JSX, and `get_computed_styles` to get the exact CSS values. The agent scaffolds a Vite + React + Tailwind project, drops the hero component in, and starts the dev server.

Then comes iteration. The designer reviews the site in the browser, spots a misaligned call-to-action button, and tells the agent to fix it. The agent reads the computed styles from Paper again, adjusts the Tailwind classes, and the site updates. No Slack messages. No handoff document. No waiting.

For responsive breakpoints, the designer creates three frames in Paper: mobile, tablet, and desktop. Each frame is a different viewport. The prompt: "Add responsive breakpoints based on the frames I have selected in Paper. Each frame is a different breakpoint." The agent reads each frame via MCP and generates the corresponding Tailwind responsive classes.

For testimonials, the agent chains two MCP servers. It reads customer quotes from the Notion database via the Notion MCP, then writes them into the Paper design via the Paper MCP. Real content, not lorem ipsum, in one step.

After each section, the designer asks Claude Code to commit. Git checkpoints create natural rollback points. When the agent produces a layout that breaks on Safari, the designer rolls back to the last commit and tries a different prompt.

Step	Action	Duration	Agent or Human
1. Design hero in Paper	Create hero section with flex layout, real fonts	30 min	Human
2. Scaffold project	Agent reads Paper selection, generates Vite+React+Tailwind	8 min	Agent
3. Hero iteration	Fix alignment, spacing, and CTA styling	25 min	Both
4. Git checkpoint	Commit working hero section	2 min	Agent
5. Responsive breakpoints	Agent reads three Paper frames, generates responsive classes	15 min	Agent
6. Testimonials section	Chain Notion MCP + Paper MCP for real content	20 min	Agent
7. Remaining sections	Features, pricing, footer — designed in Paper, built by agent	90 min	Both
8. Deploy	Agent pushes to Vercel	5 min	Agent

What worked: The Paper-to-code loop was tight. Because Paper uses real HTML and CSS, the agent's translation was nearly lossless. No "this looks different in code than in the design" complaints. The MCP chaining between Notion and Paper for testimonials was a genuine "aha" moment --- the designer pulled real customer quotes directly into the build without copy-pasting. Git checkpoints saved the session twice when the agent produced broken layouts.

What didn't: The agent occasionally hallucinated MCP tool names. When the session ran longer than 90 minutes, the MCP connection dropped once and required a restart. The agent struggled with the more complex pricing table layout --- the designer had to break it into two smaller prompts instead of one big one. As noted in Paper's documentation: "The larger the thing you're trying to build, the more likely the agent will get stuff wrong."[1]

My take: Paper's `get_jsx` tool is the single most polished design-to-code bridge I have used. The fact that Paper's canvas IS HTML means zero translation loss. But the MCP connection stability issues are real. I restart my agent session every 60 minutes now as a precaution. The Paper team is aware of this and has it on their roadmap.

Case Study 2: Brand-Consistent Pitch Deck with Open Design

The setup: a product team at an early-stage company is preparing a pitch deck for their next funding round. They have a brand guide (colors, typography, logo), a rough content outline, and a deadline. The old approach: a designer spends a week in Figma or Google Slides, iterating on layout and typography. The new approach: Open Design handles the visual execution while the team focuses on content and narrative.

The team lead installs Open Design and launches it with the prompt: "Make me a magazine-style pitch deck for our Series A raise. Brand: fintech, professional, trustworthy. Use our logo and color palette."

Open Design responds with its interactive question form. Five questions: surface (presentation), audience (investors), tone (professional), brand context (the team pastes their brand guide URL), scale (12 slides). This form prevents about 80% of the redirects that plague one-shot generation. The team answers honestly --- "investors who have seen 50 decks today" for audience, "authoritative but not stiff" for tone.

Next comes the direction picker. Open Design presents five curated visual directions: Monocle, Modern Minimal, Tech Utility, Brutalist, and Soft Warm. Each direction has deterministic OKLch palettes and font stacks --- the model does not freestyle colors or fonts. The team picks Monocle, which gives them a sophisticated editorial look with high-contrast typography and restrained color use.

The agent streams a live `TodoWrite` plan into the terminal. The team watches as it lays out 12 steps: import brand tokens, build slide master, generate title slide, create team section, build financials chart, and so on. Then the daemon builds the artifact. It reads the brand guide, applies the Monocle direction's constraints, and generates each slide.

After generation, Open Design runs its built-in 5-dimensional self-critique. It scores the output on philosophy consistency, visual hierarchy, detail execution, functionality, and innovation. The first run scored 72/100 --- weak on visual hierarchy. The team asked for one iteration. The second run hit 81/100. Good enough for a seed round deck.

Export options: HTML, PDF, PPTX, ZIP, or Markdown. The team exports both PDF (for email) and PPTX (for live presentation).

Input Quality	Direction Chosen	First-Run Score	After 1 Iteration	Verdict
Full brand guide + content outline	Monocle	72/100	81/100	Usable with minor manual tweaks
Brand colors only, no content outline	Tech Utility	64/100	71/100	Needed significant manual editing
No brand assets, generic prompt	Modern Minimal	58/100	65/100	Started from scratch in Figma instead

What worked: The question form and direction picker eliminated the "I don't know what I want but I'll know it when I see it" problem. The deterministic palettes prevented the agent from producing garish color combinations. The self-critique gave the team a concrete reason to ask for a second iteration rather than accepting mediocre output. Total time from prompt to exportable deck: under 90 minutes.

What didn't: Open Design is honest about its limitations, as described in the project's README: "We are iterating fast on main."[2] The tool has rough edges. The PPTX export lost some typographic fidelity --- the fonts shifted in PowerPoint. The brand-from-zero experience (no assets provided) dropped to 58 points on the self-critique, confirming what the Huashu Design skill documentation observes: "Open Design is an 80-point skill, not a 100-point product."[3] The team with no brand assets ended up abandoning the agent output and starting over in Figma.

Case Study 3: Product Launch Video with Huashu Design and Hyperframes

The setup: a creative technologist at a consumer technology company needs a 60-second product launch video. The company has brand assets (logo, color palette, app screenshots), a script outline, and a launch date. Two approaches are tested: Huashu Design for a motion-design-style animation and Hyperframes for an HTML-native video composition.

The Huashu Design approach. The technologist installs the Huashu Design skill and prompts: "Turn this product launch into a 60-second animation. Use our brand colors (a vivid blue primary, dark navy background). Export MP4 and GIF."

Huashu Design enforces its Core Asset Protocol first. The protocol requires: ask for assets, search official channels, download with three fallback paths, verify and extract, then freeze to spec. This prevents the agent from inventing fake logos or using placeholder icons. The technologist provides the real logo file and app screenshots. The skill locks them in.

Next comes the Junior Designer Workflow. Instead of generating the final video immediately, the skill outputs its assumptions, placeholders, and reasoning. The technologist reviews: the skill plans to use a Stage + Sprite time-slice model with `useTime`, `useSprite`, and `interpolate` APIs. It identifies six

scenes: logo reveal, problem statement, solution demo, feature showcase, social proof, and call-to-action. The technologist approves the plan.

The anti-AI-slop rules activate automatically. No purple gradients. No emoji icons. No rounded-corner-plus-left-border-accent pattern. No SVG humans. No Inter-as-display-font. These rules are hardcoded in the skill, not optional. The output is cleaner for it.

Export uses `render-video.js` for HTML-to-MP4, `convert-formats.sh` for 60fps interpolation and GIF generation, and `add-music.sh` for one of six background music tracks with automatic fade. Before delivery, the skill runs Playwright verification --- it opens the HTML in a headless browser and confirms every element renders correctly.

The Hyperframes approach. The technologist runs `npx hyperframes init product-launch` to scaffold a new project. Hyperframes uses HTML + GSAP as the authoring format. The agent writes an HTML composition with data attributes: `data-composition-id`, `data-start`, `data-duration`, `data-track-index`. Preview runs locally with `npx hyperframes preview` and live reload. Render to MP4 with `npx hyperframes render`.

Hyperframes ships with 50+ ready-to-use blocks: flash-through-white transitions, data chart animations, Instagram-style follow callouts. The technologist composes six blocks into a sequence, each timed to the script.

Dimension	Huashu Design	Hyperframes
Authoring format	HTML + custom Stage/Sprite API	HTML + GSAP data attributes
Design philosophy	Anti-AI-slop enforced, custom animation model	Block-based composition, industry-standard GSAP
Quality guardrails	5-dimension self-critique, anti-slop rules, Playwright verification	Deterministic rendering, catalog blocks
Brand enforcement	Core Asset Protocol (verified real assets)	Manual --- no built-in brand protocol
Export	MP4, GIF, with 60fps interpolation and BGM	MP4 via HTML→video pipeline
Rendering	Single machine only	Single machine (no Lambda equivalent)
Agent compatibility	Agent-agnostic skill (works with any CLI agent)	Built specifically for AI agents
Learning curve	Steep --- custom animation API, bilingual docs	Moderate --- standard HTML + GSAP

What worked: Huashu Design produced a more polished, brand-consistent output. The Core Asset Protocol and anti-slop rules made a visible difference --- the final video looked professional, not AI-generated. Hyperframes was faster to iterate on because the block model is simpler. The live preview loop in Hyperframes (edit HTML, see result, repeat) felt more natural for rapid prototyping.

What didn't: Huashu Design is bilingual --- the SKILL.md is in Chinese, which adds friction for English-speaking teams. The custom Stage/Sprite API is less flexible than GSAP for complex animations. Hyperframes lacks brand enforcement --- the agent can produce off-brand output unless you constrain it with a separate skill. Both tools are single-machine rendering only; for distributed rendering at scale, Remotion's Lambda is still the only option (covered in Chapter 09).

> **My take:** For one-off videos where brand consistency matters, Huashu Design wins. The anti-slop rules and Core Asset Protocol produce output that doesn't scream "AI-made." For iterative video work where speed matters more than polish, Hyperframes is the better choice. I use both, depending on the project. The bilingual documentation issue with Huashu Design is real but manageable --- the code examples are language-agnostic.

Case Study 4: Cross-Platform Component Library with Pencil and OpenPencil

The setup: a design systems team at a mid-size company maintains a component library that targets web (React), iOS (SwiftUI), and Android (Jetpack Compose). The team has been using Figma for design and hand-writing platform-specific implementations. The goal: use agentic design tools to reduce the synchronization overhead between platforms.

The Pencil workflow. The team designs components in .pen files inside their Git repository. Each component uses Pencil's variables for design tokens (colors, spacing, typography) and slots for content areas. Slots translate directly to React component props. Variables map to CSS custom properties. The .pen files are plain text, so they diff cleanly in pull requests.

The team builds a primary button component as a .pen file. It has slots for the label text and an optional icon, and variables for the background color, border radius, and padding. The agent reads the .pen file via the Pencil CLI and generates a React + Tailwind component. The mapping is mechanical: slot becomes prop, variable becomes CSS custom property.

The OpenPencil workflow. OpenPencil adds multi-platform export. The same button component, designed once in OpenPencil's canvas, can be exported to React + Tailwind, HTML + CSS, Vue, Svelte, Flutter, SwiftUI, Jetpack Compose, and React Native. The export uses an incremental codegen pipeline: `codegen_plan` to analyze the component tree, `codegen_submit_chunk` for each platform, `codegen_assemble` to combine, and `codegen_clean` to remove artifacts.

OpenPencil also adds Concurrent Agent Teams. For the component library, the team runs three agents simultaneously: one generates the React output, one generates the SwiftUI output, and one generates the Compose output. Each agent works on a different section of the canvas. An orchestrator agent decomposes the task and merges the results.

The Git integration is critical. OpenPencil supports folder-mode three-way merge for .op files. When two agents modify the same component, the merge panel shows the conflict and lets the human resolve it. This is a step beyond Pencil's simpler single-user model.

Export Target	Pencil	OpenPencil	Quality (1-5)
React + Tailwind	Yes	Yes	4/5 (both)
HTML + CSS	Via manual export	Yes	4/5 (OpenPencil)
SwiftUI	No	Yes	3/5 (basic components)
Jetpack Compose	No	Yes	3/5 (basic components)
Flutter	No	Yes	3/5 (basic components)
Vue / Svelte	No	Yes	3/5
React Native	No	Yes	2/5 (limited)

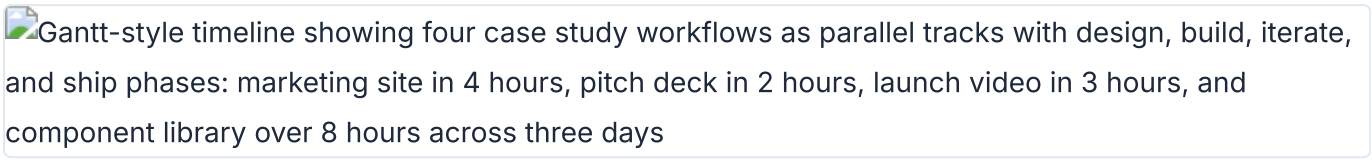
What worked: The .pen file format is genuinely diffable. The team could review design changes in pull requests alongside code changes --- a first for them. OpenPencil's multi-platform export saved an estimated 40% of the synchronization time for basic components (buttons, inputs, cards). The concurrent agent approach for parallel platform output was effective: three platforms generated in roughly the time it used to take for one.

What didn't: Complex components (data tables with sorting, date pickers, rich text editors) degraded to 2/5 quality on non-web platforms. The SwiftUI and Compose output needed significant manual cleanup. Pencil is desktop-only and single-user --- no real-time collaboration. OpenPencil's three-way merge worked for simple conflicts but struggled when agents restructured component hierarchies. Both tools use proprietary JSON formats (.pen and .op) that require translation layers, unlike Paper's HTML-native approach.

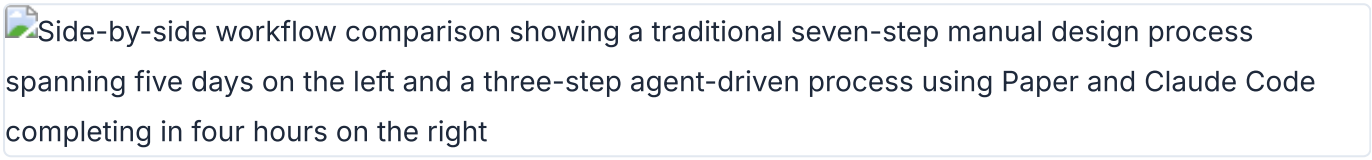
The team's honest conclusion: agentic design tools eliminated 40% of the boring work (basic component synchronization) but created 20% of new work (fixing agent-generated output for complex components). Net positive, but not the 10x improvement the team hoped for.

Patterns Across the Case Studies

Four different workflows, four different tool combinations, four different output types. The patterns that emerge are consistent.



Timeline comparison of four case studies showing design, build, iterate, and ship phases



Before and after: traditional design workflow (5 days) vs agent-driven workflow (4 hours)

Start small, build up. Every successful workflow began with a small piece --- one hero section, one slide, one scene, one button. The agent produced good output for small, well-scoped tasks. Quality degraded as scope increased. Paper's documentation puts it plainly[1]: start with a small part, then build up. This pattern held across all four cases.

Git checkpoints are non-negotiable. In every case study, the human asked the agent to commit after each working section. When the agent went wrong --- and it always went wrong at some point --- the rollback was clean. Teams that skipped this step regretted it.

Human provides direction, agent handles execution. Spatial thinking stayed with humans. Typography choices, layout composition, color decisions --- these were human-directed. The agent handled the mechanical translation: converting designs to code, generating responsive variants, synchronizing tokens across platforms.

MCP chaining multiplies value. The Notion + Paper chain for testimonials, the Figma + Paper chain for token sync, the Pencil + OpenPencil chain for multi-platform export. Each MCP server added a capability. Chaining them created workflows that no single tool could deliver.

HTML-native tools have an edge. Paper's real HTML/CSS meant zero translation loss between design and code. Pencil and OpenPencil's JSON formats required translation layers that introduced friction. The closer the design format is to the production format, the better the agent performs.

Anti-slop rules matter. Huashu Design's explicit rules against AI aesthetic patterns produced visibly better output. Open Design's deterministic palettes prevented garish colors. Teams that relied on unconstrained agent generation got "AI-smelling" results.

Pattern	Case 1	Case 2	Case 3	Case 4
Start small	Yes --- hero first	Yes --- slide by slide	Yes --- scene by scene	Yes --- basic buttons first
Git checkpoints	Yes --- after each section	N/A --- no code repo	Yes --- after each scene	Yes --- after each component
Human direction	Layout, typography	Narrative, direction choice	Script, brand assets	Token values, component API
Agent execution	Code scaffolding, responsive	Visual generation, export	Animation rendering, export	Multi-platform codegen
MCP chaining	Notion + Paper	N/A --- single tool	N/A --- single tool	Pencil + OpenPencil
Anti-slop enforcement	No	Yes --- deterministic palettes	Yes --- Huashu rules	No

My take: The biggest surprise across all four cases was how consistent the failure modes are. Agent quality degrades with scope. MCP connections drop on long sessions. Complex layouts break. The teams that succeeded were the ones that anticipated these failures and designed their workflow around them --- small increments, frequent commits, human-directed design decisions. The tool matters less than the workflow pattern.

Tip

Next: These case studies show where agentic design is today. Chapter 14 looks at where it is heading --- the trends, predictions, and risks that will shape the next two years.

Notes

1. Paper documentation, "Working with Agents" guide. Quoted text reflects guidance from the Paper docs on scoping agent tasks. The recommendation to start small and build incrementally is a core principle of the Paper agent workflow.
2. Open Design README, GitHub repository. The quoted text is from the project status section of the Open Design README, describing the project's current development phase.

3. Huashu Design skill documentation (`SKILL.md`). The quoted assessment appears in the Huashu Design skill's self-evaluation framework, where skills are rated on a 100-point scale across five dimensions.

14 The Future of Agentic Design

Trends, predictions, and what comes after the current tools

This book covers tools and workflows that exist right now, in May 2026. But the agentic design ecosystem is moving fast. This chapter maps the four major trends shaping what comes next, makes specific predictions for 2027 and 2028, and examines the risks that could undermine the whole trajectory.

Where We Are: The State of Agentic Design in May 2026

A snapshot of the ecosystem at the time of writing.

Ecosystem evolution from GUI-only design (2024) through agentic emergence (2026) to autonomous systems (2028)

Paper launched its Desktop app in February 2026, becoming the first connected design canvas with an MCP server that works with any agent: Claude Code, Codex CLI, Cursor, GitHub Copilot, OpenCode, Windsurf, Cline, and Continue. According to Paper's published case studies and marketing materials, production users include Vercel, Perplexity, PostHog, Tailwind CSS, Dub, Replicate, Zed, and Attio.

Open Design hit 43.6k GitHub stars in under two weeks after launch, according to public GitHub data --- reportedly the fastest growth for an open-source design tool. It supports 16 CLI agents and ships 129 design systems as portable Markdown files.

OpenPencil shipped Concurrent Agent Teams: multiple AI agents working on different sections of the same canvas simultaneously. It is the first open-source vector design tool built around the agent as the primary user. At 3.1k stars as of May 2026, it is smaller but evolving fast.

Huashu Design relicensed to MIT on May 14, 2026, signaling that the open-source design skill ecosystem is maturing. At 14.1k stars as of May 2026 (according to public GitHub data), it remains the most specialized tool in the stack --- an HTML-native design skill with anti-AI-slop enforcement and a 5-dimension self-critique system.

Hyperframes (19k stars as of May 2026, by HeyGen) and Remotion (47.1k stars according to public GitHub data) represent the two poles of agent-driven video production. Hyperframes is HTML-native and agent-first. Remotion is React-based and mature.

Four major agent platforms compete: Claude Code, Codex CLI, OpenCode, and Gemini CLI. All four support MCP. MCP itself has become the universal connector between agents and design tools --- Paper, Pencil, OpenPencil, Figma, Miro, and MagicPattern all ship MCP servers as of May 2026.

Tool	Stars (May 2026, per public GitHub data)	Key Milestone	Agent-Native?
Remotion	47.1k	v4.0.462, mature framework	No (React-based)
Open Design	43.6k	Fastest-growing open-source design tool	Yes
Hyperframes	19k	137 releases, 50+ catalog blocks	Yes
Huashu Design	14.1k	MIT relicensed May 2026	Agent-agnostic skill
OpenPencil	3.1k	Concurrent Agent Teams shipped	Yes
Paper	Commercial	Desktop launched Feb 2026	Yes
Pencil	Commercial	.pen format, IDE integration	Partial (MCP server)

Trend 1: Autonomous Design Systems

Design systems are becoming self-maintaining. This is the most consequential trend in the ecosystem, and it is happening faster than I expected.

Consider the current state. Open Design's library contains 129 design systems, each expressed as a DESIGN.md file --- a portable Markdown specification that any agent can read and apply. These are not static theme JSON files. They are living documents that agents interpret, enforce, and update. When a token changes in the DESIGN.md, the agent applies the change across every artifact it generates.

OpenPencil's Design System Kit takes this further. It manages reusable UIKits with style switching and component composition. A team can define Light, Dark, and High Contrast themes as variable sets inside a single .op file. Switching themes is a variable swap, not a redesign.

Paper's bi-directional workflow closes the loop. A designer changes a token value in Paper. The agent reads the change via MCP and updates the CSS custom properties in the codebase. A developer changes a spacing value in Tailwind config. The agent reads the change in Git and updates the Paper design. No manual sync. No drift.

The trajectory is clear. Based on publicly available information, GitHub's design team has reportedly signaled a shift away from maintaining separate component libraries, moving toward treating the codebase as the single source of truth. The direction is unambiguous: translating code components to design is becoming a single-prompt operation.

The vision for 2027: design systems that auto-detect token drift between design and code, flag the discrepancy, and offer to auto-fix. The human approves or rejects. The agent handles the mechanical synchronization that currently eats hours of design systems team time every sprint.

Trend 2: Real-Time Collaborative Agents

The second trend: multiple agents and multiple humans working on the same design surface simultaneously. Not asynchronously through Git. In real time.

Paper already has real-time multiplayer. You share a URL, your team members see changes live. But the multiplayer is human-only today. The agent participates through MCP, which is stateless --- it reads and writes but does not observe live.

OpenPencil's Concurrent Agent Teams point toward the future. Three agents working on different sections of the same canvas, coordinated by an orchestrator. This is parallel execution, not real-time collaboration. But it proves the spatial decomposition pattern: complex designs can be split into independent sub-tasks, assigned to different agents, and merged.

Open Design takes a different approach to the same problem. It auto-detects 16 CLI agents on your PATH and lets you swap between them with one click. This is agent portability, not agent collaboration. But it establishes the principle that the design tool should be agent-agnostic --- it should not care which agent you use.

The convergence point, likely in late 2027: a design canvas where one human and two agents are working on different sections at the same time, with live cursors showing where each participant is active. The human designs the hero section. Agent A generates responsive variants. Agent B synchronizes tokens with the codebase. All three changes appear in real time.

The orchestrator pattern from Chapter 11 becomes the standard architecture for this. An orchestrator agent decomposes the design task, assigns spatial sub-tasks to worker agents, and merges the results. The human acts as creative director, intervening when the agents produce something off-brand.

Trend 3: Design and Code as a Single Artifact

The third trend is the most fundamental. Design and code are converging into a single artifact. Not "design that exports to code." Not "code that renders a preview." A single artifact that IS both design and code simultaneously.

Paper is the clearest example. Paper's canvas is built on real HTML and CSS. When you design in Paper, you are writing HTML and CSS. There is no translation layer. The `get_jsx` tool does not convert a proprietary format into JSX --- it extracts JSX from a surface that is already HTML. The `write_html` tool goes the other direction, but it is writing to the same format.

This is different from Figma's model. Figma stores designs in a proprietary WebGL format. Developers and agents need to translate that format into production code. Paper's approach is the inverse: its native format is already close to production code, making it inherently agent-friendly. There is no translation step because the design surface speaks the same language as the output.

Pencil's .pen files and OpenPencil's .op files are intermediate steps on the same path. They are code artifacts --- plain text, diffable, version-controllable. A pull request can include changes to a .pen file alongside changes to a .tsx file. The reviewer sees both the design change and the code change in the same diff. But .pen and .op files are still separate formats that need translation to production code.

Approach	Tool	Format	Translation Layer	Diffable?
Design IS code	Paper	HTML/CSS	None	Yes (native HTML)
Design-as-code artifact	Pencil	.pen (JSON)	Required (pen → JSX)	Yes (plain text)
Design-as-code artifact	OpenPencil	.op (JSON)	Required (op → multi-platform)	Yes (plain text)
Proprietary canvas	Figma	WebGL binary	Required (fig → anything)	No

The end state: a world where there is no "design file" and "code file." There is one file. It renders visually in a canvas tool and compiles to production code. The distinction between designer and developer roles blurs because the artifact is the same.

Multiple voices in the ecosystem are converging on this idea. The future of code is design, and the best design work emerges from a canvas --- not a code editor. At the same time, code is a poor interface for spatial problems, and chat is a poor interface for systemic decisions. The canvas --- not the code editor, not the chat window --- is the correct interface for design work. But the canvas must produce code, not pixels.

My take: The design-code convergence is inevitable, but the timeline is uncertain. Paper's HTML-native approach is the most pragmatic path I see. Proprietary canvas formats (Figma, Sketch) will either open up or become liabilities. The .pen and .op formats are stepping stones --- useful now, but ultimately unnecessary if the design surface speaks the same language as production code.

Trend 4: The Rise of Agent-Native Design Tools

The fourth trend is the emergence of design tools built from the ground up for agent collaboration. Not legacy tools with AI features bolted on. Tools where the agent is the primary user and the human is the creative director.

Paper was the first tool built from scratch for agent collaboration via MCP. Every tool in Paper's MCP server --- `get_selection`, `get_jsx`, `get_computed_styles`, `write_html` --- was designed for agent consumption, not human clicking. The human uses the visual canvas. The agent uses the MCP API. Both work on the same artifact.

OpenPencil calls itself "the world's first open-source AI-native vector design tool." The agent can create shapes, apply styles, manage components, and export to code --- all through MCP tools, without a human touching the canvas. The Agent Teams feature extends this: the canvas is partitioned into spatial regions, and each agent works independently.

Open Design is agent-native in a different sense. There is no canvas. The agent IS the design engine. You give it a prompt, it asks clarifying questions, picks a visual direction, and builds the artifact. The output is the design. There is no intermediate visual surface.

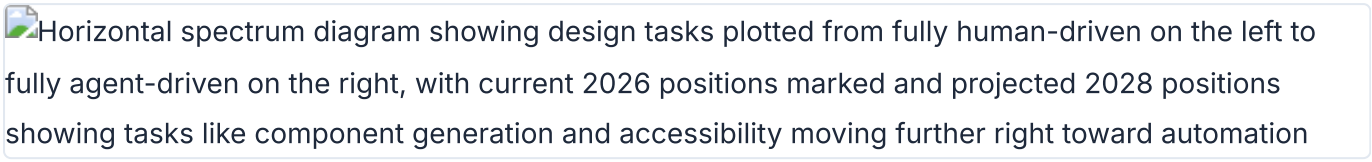
Hyperframes is explicit about its positioning: "Built for agents." Its HTML-native authoring format and non-interactive CLI are designed for agent consumption, not human editing. The 50+ catalog blocks are pre-built agent actions --- composable units that an agent can assemble without understanding animation principles.

The key differentiator across all these tools: the format. Tools where agents read and write the same format as production (HTML, CSS, JSX) have a structural advantage over tools where agents must translate between formats. Paper's `get_jsx` works because the canvas IS HTML. Figma's MCP server works, but every read requires format translation.

Legacy tools are responding. Figma shipped an MCP server in early 2026. It works. But retrofitting agent support onto a proprietary canvas format is fundamentally different from building for agents from the start. The MCP server can read designs and export code, but the format translation introduces information loss and complexity that agent-native tools avoid.

Predictions for 2027-2028

Specifying predictions with timestamps so you can hold me accountable.



Design task spectrum from human-driven to agent-driven with 2026 current state and 2028 projections

Prediction	Timeline	Confidence	Signal Today
MCP servers ship as table stakes for every design tool	By Q4 2026	High	6 tools already have MCP; Figma added one in early 2026
Design files disappear as a concept --- .pen and .op become code artifacts reviewed in PRs	By mid 2027	Medium	Pencil and OpenPencil already Git-integrated; Paper is HTML-native
Agent Teams are standard for complex design tasks	By late 2027	Medium-High	OpenPencil already ships Concurrent Agent Teams
The "design handoff" dies --- design IS code	By 2028	Medium	Paper's bi-directional workflow, GitHub treating codebase as source of truth
Design systems auto-detect and auto-fix token drift	By late 2027	Medium	Paper's token sync, Open Design's DESIGN.md schema
Figma either opens its format or loses market share to agent-native alternatives	By 2028	Low-Medium	Community reportedly switching to Paper; Paper's case studies list Vercel/Perplexity/PostHog
Video production becomes a standard design skill, not a specialist discipline	By mid 2027	High	Hyperframes 50+ blocks, Huashu Design MP4 export, Remotion Lambda
The designer role shifts from pixel-pusher to agent director and design system architect	Ongoing through 2028	High	Reportedly already happening in teams using Paper and Open Design

The prediction I am most confident about: MCP becomes universal by late 2026. Every design tool that wants to remain relevant will ship an MCP server. The ones that do not will become niche. This is not a bold prediction --- it is an observation of a trend that is already well underway at the time of writing.

The prediction I am least confident about: Figma's trajectory. Figma has enormous institutional momentum, deep enterprise contracts, and a generation of designers trained on its interface. Its MCP server, shipped in early 2026, shows awareness of the agent trend. But its proprietary canvas format is a growing liability in an agent-native world. Whether Figma opens its format or doubles down on proprietary lock-in will determine its trajectory.

Risks: Homogenization, Over-Reliance, and the Designer's Role

The trends are real. The predictions are grounded in observable signals. But the trajectory is not guaranteed. Several risks could undermine the whole ecosystem.

Quality homogenization. This is the risk I hear about most from design leaders. A prominent design executive puts it bluntly: the more companies talk about coding with agents and designing in code, the worse their UX tends to be. A design tool founder agrees: if you are building software meant to be consumed by humans, take the time to actually design it. AI has a design smell because it is "designing" via code, not through spatial reasoning.

The "AI design smell" is real. Huashu Design's anti-slop rules list the specific patterns: purple gradients, emoji icons, rounded-corner-plus-left-border-accent, SVG humans, Inter-as-display-font. When every agent generates from the same model weights without constraint, the output converges. Every startup's landing page starts to look the same.

The mitigation is clear but not easy. Strong design systems with custom tokens. Agent skills with anti-slop enforcement. Human creative direction at every step. The tools that bake these constraints into the workflow --- Huashu Design, Open Design --- will matter more than the tools that leave output quality to the model.

Over-reliance on agents. The warning from experienced practitioners is consistent: no agent replaces design judgment. Tools can accelerate execution, but the craft of design --- the spatial reasoning, the taste, the ability to hold ambiguity --- remains human. When designers treat agents as creative substitutes rather than execution engines, the quality drops.

Agents cannot read between the lines. They cannot hold ambiguity. They cannot make judgment calls about what matters and what does not. They execute instructions. When the instructions are clear and specific, the output is good. When the instructions are vague or absent, the output is generic.

The risk is not that agents replace designers. The risk is that designers lose craft skills as agents handle more execution. When execution becomes cheap, the temptation is to skip the spatial phase of thinking entirely. The bottleneck moves upstream --- it becomes a question of mind, not mechanics. The designer's value shifts from execution to spatial thinking, creative direction, and taste. But if designers stop practicing spatial thinking because agents handle the rendering, the taste atrophies.

My take: The homogenization risk is the one I worry about most. I have seen agent-generated designs that are technically correct and aesthetically empty. The tools will get better at avoiding AI slop, but the underlying problem is cultural: when execution becomes free, the temptation to skip thinking is overwhelming. The designers who thrive in this new landscape will be the ones who treat agents as execution engines, not creative partners. The creative work stays human.

The designer's changing role. A creative tool founder frames it directly: designers should be racing to get as close to the product as possible. The role is shifting from pixel-pusher to agent director, design system architect, and quality curator.

This shift creates opportunity and anxiety in equal measure. The designers who embrace agentic workflows gain leverage --- they produce more output, iterate faster, and ship with higher consistency. The designers who resist or ignore the shift find their skills depreciating as the industry moves to agent-native tools.

A design engineer captures the human advantage: if AI can think better and software can scale wider, then the scarce thing is the particular, messy way a human mind cares about something. Agents do not care. They execute. The caring --- the judgment about what is good, what is worth building, what matters to users --- stays human.

Other risks worth watching. Vendor lock-in: tools that require specific agents or models create dependency. Cost: API token usage for agent-driven workflows adds up, especially for video generation and multi-agent teams. Reliability: agents hallucinate tool calls, lose MCP connections on long sessions, and degrade on large tasks. Security: agents with write access to design files and code repositories need permission models, and those models are still immature. Accessibility: if agents generate code without a11y enforcement, digital exclusion worsens.

Risk	Severity	Current Signal	Likely Mitigation
Quality homogenization	High	AI design smell in agent-generated output	Anti-slop rules, custom design systems, human curation
Craft skill atrophy	Medium-High	Designers skipping spatial thinking phase	Education, deliberate practice, canvas-first workflows
Vendor lock-in	Medium	Tools requiring specific agents or models	Open-source alternatives, MCP standardization
API cost at scale	Medium	Token costs for video and multi-agent teams	Model efficiency improvements, local models
Agent reliability	Medium	Hallucinated tool calls, dropped MCP connections	Better tool validation, session recovery protocols
Security surface area	Medium	Agents with write access to design and code	Permission models, sandboxed execution, audit logs
Accessibility regression	High	Agents generating code without a11y enforcement	Automated a11y testing in agent loop (covered in Chapter 12)

These risks are not theoretical. I encountered agent hallucination, MCP connection drops, and security permission issues in the case studies covered in Chapter 13. The ecosystem will address some of these. Others will persist.

My take: I am optimistic about the trajectory but realistic about the timeline. The tools covered in this book are version 1.0 of agentic design. The MCP protocol is young. The skill ecosystem is nascent. The permission models are thin. But the direction is clear: design and code are converging, agents are becoming collaborative, and the canvas is returning as the primary interface for creative work. The designers who learn to direct agents --- not just use them --- will define the next decade of product design.

Tip

Next: This chapter concludes the main content of the book. Appendix A provides a comprehensive tool comparison matrix for quick reference. Appendix B contains the full MCP server reference. Appendix C offers a prompt library organized by design task.

15 Agentic Presentations

Seven tools that let agents design and deliver slide decks

Presentations are the most universal design artifact in business. Every team makes decks. Every conference needs slides. Every pitch requires a visual narrative. Until recently, building a presentation meant opening PowerPoint or Keynote, choosing a template, and manually arranging text and images on each slide. Agentic tools have changed this. This chapter covers seven projects that let AI coding agents generate production-grade HTML slide decks, each taking a different approach to the problem.

Why Presentations Are Different from UI Design

Generating a slide deck is not the same as generating a web page. Slides have constraints that general-purpose web UI does not:

- **Fixed viewport.** Every slide targets a fixed canvas (typically 1920×1080 or 16:9 aspect ratio). Content must fit within this frame without scrolling.
- **Sequential narrative.** A deck tells a story across slides. The information architecture is linear, not hierarchical. Each slide must work as a standalone visual while contributing to the overall arc.
- **Performance under pressure.** Presentations are shown live. The slide file must open instantly, render reliably, and handle keyboard or click navigation without lag. No loading spinners, no network dependencies during delivery.
- **Content density per slide is low.** A good slide has 20-40 words. A good web page has 200-400. The agent must enforce brevity, not fill the canvas with text.
- **Speaker notes are invisible infrastructure.** The speaker reads from notes the audience never sees. The agent must generate both the visual slide and the supporting script.

These constraints make presentations a surprisingly good test case for agentic design. The output is self-contained (one HTML file), the quality criteria are visual (does it look professional?), and the iteration cycle is fast (generate, present, adjust, repeat).

The HTML Slide Deck Ecosystem

Seven active projects form the current ecosystem. They fall into three categories:

Agent skills (prompt-only, no runtime): frontend-slides, guizang-ppt-skill, html-ppt-skill, presentation-skills. These are structured markdown files (SKILL.md) that guide an existing coding agent through the slide generation process. The agent reads the skill, asks clarifying questions, and produces HTML output using its existing code-generation capabilities.

Agent frameworks (runtime + skills): Open Design, open-slide. These provide a runtime environment for slide generation. Open Design is a full design platform with 31 skills including 4 deck skills. Open-slide is a React-based slide framework with built-in agent skills and an in-browser inspector.

Template libraries: beautiful-html-templates. This is a curated collection of 34 HTML slide templates with machine-readable metadata. Agents pick a template by matching user intent against the metadata, then fill in the content.

Project	Stars	Category	Output Format	Key Differentiator
Open Design	46.1k	Framework	HTML, PDF, PPTX	31 skills, 129 design systems, MCP server, Electron app
frontend-slides	18.1k	Agent skill	HTML	7-phase workflow, PPT conversion, visual style discovery
guizang-ppt-skill	10.1k	Agent skill	HTML	2 visual systems, 32 locked layouts, WebGL backgrounds
open-slide	3.4k	Framework	React/HTML	Agent-native React, in-browser inspector, presenter view
html-ppt-skill	4.1k	Agent skill	HTML	36 themes, 31 layouts, 20 canvas FX, presenter mode
beautiful-html-templates	1.6k	Template library	HTML	34 curated templates, index.json metadata, tone-first matching
presentation-skills	104	Agent skill	PPTX (native)	python-pptx, 3-stage quality gates, editable Office output

The projects share a common insight: presentations are text-driven artifacts. You describe what you want in natural language, and the agent generates a visual output. The difference is in how much structure they impose on the generation process.

Open Design: The Full-Stack Design Platform

Open Design (nexu-io/open-design, 46.1k stars) is the largest project in this space. It is a full design platform that turns existing coding agent CLIs into design engines. The project does not ship its own AI model. Instead, it scans your system for 16 coding agent CLIs and spawns whichever you select to execute design tasks.

For presentations, Open Design provides four dedicated deck skills:

Skill	Output	Best For
`guizang-ppt`	Magazine-style HTML deck with WebGL hero backgrounds	Keynotes, product launches, editorial-style talks
`simple-deck`	Minimal horizontal-swipe deck	Quick updates, internal reviews, lightweight talks
`replit-deck`	Product-walkthrough deck	Demos, feature walkthroughs, technical showcases
`weekly-update`	Team cadence swipe deck	Stand-ups, weekly syncs, status reports

The platform ships 129 brand-grade design systems (Linear, Stripe, Vercel, Airbnb, Apple, Tesla, Notion, Figma) as portable DESIGN.md files. Each design system encodes colors, typography, spacing, and component styles in a format the agent reads deterministically. The agent also has access to 5 curated visual directions (Editorial Monocle, Modern Minimal, Warm Soft, Tech Utility, Brutalist Experimental) with deterministic OKLch color palettes.

The presentation workflow in Open Design follows the "Junior Designer" pattern covered in Chapter 07 (Huashu Design): the agent asks a structured discovery questionnaire before writing any code, shows an early preview, and self-critiques its output against a 5-dimensional rubric. This forced-discipline approach prevents the agent from producing generic output.

Open Design also ships a stdio MCP server that exposes read-only access to your local projects. Any MCP-compatible agent (Claude Code, Codex, Cursor, VS Code, Zed, Windsurf) can pull design tokens, CSS, JSX components, and HTML from an Open Design project into its own workflow. This makes Open Design a design source that feeds into other agent workflows, not just a standalone tool.

The desktop app (Electron) runs on macOS and Windows, providing a sandboxed iframe preview with in-place editing. Export formats include HTML, PDF, PPTX, and ZIP. (source: nexu-io/open-design, retrieved 2026-05-20)

frontend-slides: The Visual Discovery Workflow

frontend-slides (zarazhangrui/frontend-slides, 18.1k stars) is a Claude Code skill that generates animation-rich HTML presentations. Its core innovation is **visual style discovery**: instead of asking users to describe aesthetics in abstract terms ("modern," "minimal," "bold"), the skill generates three style previews as actual HTML files and lets the user pick what they like.

The skill follows a 7-phase workflow:

Phase	Purpose
0	Detect mode: new deck, PPT conversion, or enhance existing
1	Content discovery: purpose, length, images, inline editing preference
2	Style discovery: generate 3 visual previews for user comparison
3	Generate full presentation from chosen style
4	PPT extraction and conversion (if source is .pptx)
5	Delivery: open in browser, summarize controls
6	Share/export: Vercel deploy or PDF export

Phase 2 is the key differentiator. The agent generates three small HTML samples, each in a different visual style. The user opens them in a browser and picks the one that feels right. This "show, don't tell" approach avoids the common problem where users struggle to articulate their aesthetic preferences in words.

The skill ships 12 curated style presets: 4 dark, 4 light, and 4 specialty (Neon Cyber, Terminal Green, Swiss Modern, Paper & Ink). Each preset includes specific font pairings, color palettes, and animation patterns. The anti-AI-slop philosophy is explicit: Inter and Roboto are forbidden, purple gradients are banned, and generic layouts are rejected.

Output is a single self-contained HTML file with all CSS and JS inlined. No build step, no server, no npm dependencies. Each slide is a `<section class="slide">` with `height: 100vh` and scroll-snap alignment. The runtime handles keyboard navigation, touch/swipe, progress bar, navigation dots, and IntersectionObserver-based entrance animations. Typography uses aggressive `clamp()` for responsive scaling across viewports.

The skill also supports PPT/PPTX conversion. It uses Python's `python-pptx` library to extract text, images, and speaker notes from an existing PowerPoint file, then rebuilds the content as an HTML deck. This is useful for teams that have existing slide libraries in PowerPoint format and want to convert them to web-native presentations.

Export options include PDF via Playwright (headless Chromium screenshots at 1920×1080) and one-command Vercel deployment for live URL sharing. (source: zarazhangrui/frontend-slides, retrieved 2026-05-20)

guizang-ppt-skill: Locked Layouts and Visual Systems

guizang-ppt-skill (op7418/guizang-ppt-skill, 10.1k stars) takes the opposite approach from frontend-slides. Where frontend-slides offers visual discovery and flexible generation, guizang constrains the agent to a fixed palette of layouts and themes. The constraint is the feature.

The skill provides two complete visual systems:

Style A: Editorial Magazine (Monocle-inspired). Ten layouts built around serif typography, generous whitespace, and an electronic ink aesthetic. Themes include Ink Classic, Indigo Porcelain, Forest Ink, Kraft Paper, and Dune. Hero slides feature WebGL shader backgrounds (fluid, contour) with a low-power fallback triggered by pressing `B`.

Style B: Swiss International Typographic Style. Twenty-two locked named layouts (S01-S22) built around a strict grid system. Themes include Klein Blue, Lemon Yellow, Lemon Green, and Safety Orange. Custom colors are forbidden. You pick from the nine curated presets and the agent applies the palette consistently.

```
# guizang-ppt-skill workflow

# 1. Agent reads SKILL.md and asks 7 clarification questions:
"Style A or B?"
"Who is the audience?"
"How long is the talk?"
"What source material do you have?"
"Do you need generated images?"
"Which theme?"
"Any special constraints?"

# 2. Agent copies the seed template:
# Style A → template.html
# Style B → template-swiss.html

# 3. Agent swaps CSS variables for the chosen theme:
:root {
  --c-bg: #F5F0EB;      /* Ink Classic background */
  --c-text: #1A1A1A;     /* Ink Classic text */
  --c-accent: #C4A882;   /* Ink Classic accent */
  --c-muted: #8B8178;    /* Ink Classic muted */
  --c-hero: #2D2A26;     /* Ink Classic hero bg */
  --c-hero-text: #F5F0EB; /* Ink Classic hero text */
}

# 4. Agent picks layouts from the catalog and fills content:
# Style A: 10 editorial layouts (hero, text-heavy, split, etc.)
# Style B: 22 locked grid layouts (S01-S22)

# 5. For Swiss decks, agent runs validator:
$ node scripts/validate-swiss-deck.mjs
```

The locked-layout approach produces visually consistent output even when the agent generates slides autonomously. The agent cannot "invent" a layout that breaks the grid. It picks from the palette and fills in content. If the content does not fit the chosen layout, the agent selects a different layout rather than warping the grid.

The Swiss layout validator (`validate-swiss-deck.mjs`) catches layout drift, centered titles that should be left-aligned, SVG text that does not scale, and misplaced images. This automated quality check runs before the human sees the output, catching mechanical issues that are easy to miss in visual review.

Animation is handled by Motion One (bundled `motion.min.js` with CDN fallback). The skill also supports image generation via GPT-Image 2.0 or GPT-M 2.0 when running through Codex, producing photos, infographics, flow diagrams, and UI scenes that match the chosen theme's color palette. (source: op7418/guizang-ppt-skill, retrieved 2026-05-20)

open-slide: The React-Native Slide Framework

open-slide (1weiho/open-slide, 3.4k stars) is the only framework in this ecosystem where slides are React components. Instead of generating a static HTML file, the agent writes JSX/TSX that renders into a fixed 1920×1080 canvas. The framework handles the runtime (scaling, navigation, present mode, hot reload) so the agent only needs to focus on content.

This architecture has three advantages:

Full React ecosystem access. The agent can use any React library: Recharts for data visualization, Framer Motion for animations, Three.js for 3D elements. Slides are not limited to CSS and static images.

In-browser inspector with comments. During development, you click any element on a slide and leave a comment ("make this red," "move this up 20px"). Comments are persisted as `@slide-comment` markers in the source code. The `/apply-comments` skill has the agent batch-apply all edits in one pass. This is a structured feedback loop that replaces the unstructured "I don't like it, try again" pattern.

Professional presenter view. Fullscreen mode with a synced popup showing current slide, next slide preview, speaker notes, and a timer. This is production-ready for live conference talks.

```
# open-slide project structure

my-deck/
├── slides/
│   ├── 01-cover/
│   │   └── index.tsx       # Each slide is a React component
│   ├── 02-problem/
│   │   └── index.tsx
│   └── 03-solution/
│       └── index.tsx
├── assets/                  # Images, fonts, videos
├── skills/                  # Agent skills (create-slide, etc.)
├── AGENTS.md               # Agent instructions
└── package.json

# Agent workflow:
# 1. npx @open-slide/cli init → scaffold project
# 2. /create-slide → agent asks 4 scoping questions
# 3. Agent writes React components for each slide
# 4. Dev server runs with hot reload
# 5. Click elements to leave @slide-comments
# 6. /apply-comments → agent batch-applies edits
# 7. Export to static HTML or PDF
```

The framework ships three built-in skills: `/create-slide` (end-to-end deck creation with 4 scoping questions about topic, page count, text density, and motion preference), `/slide-authoring` (technical reference for the 1920×1080 canvas, type scale, and layout rules), and `/apply-comments` (reads

`@slide-comment` markers and batch-applies edits).

Export produces a self-contained static HTML site (deployable to Vercel, Cloudflare Pages, Netlify) or a print-ready PDF. The scaffolder generates projects preconfigured with agent skills in both `.claude/skills/` and `.agents/skills/` directories for Claude Code and agent-agnostic compatibility. (source: 1weiho/open-slide, retrieved 2026-05-20)

html-ppt-skill: The Theme and Animation Library

html-ppt-skill (lewislulu/html-ppt-skill, 4.1k stars) offers the largest library of themes, layouts, and animations among the agent skills. It ships 36 CSS themes, 31 single-page layouts, 27 CSS animations, and 20 canvas FX animations as composable building blocks.

The theme system is pure CSS tokens. Each theme is a CSS file that defines variables for colors, typography, and spacing. To reskin a deck, you swap one `<link>` tag. Themes range from `minimal-white` to `cyberpunk-neon` to `xiaohongshu-white`, covering the spectrum from corporate to creative.

```
# html-ppt-skill: Theme swapping


<link rel="stylesheet" href="assets/themes/minimal-white.css">


<link rel="stylesheet" href="assets/themes/cyberpunk-neon.css">


:root {
  --bg-primary: #0a0a0f;
  --text-primary: #e0e0ff;
  --accent-1: #ff00ff;
  --accent-2: #00ffff;
  --font-display: 'Orbitron', sans-serif;
  --font-body: 'Rajdhani', sans-serif;
}
```

The animation catalog includes CSS entrance effects (fade-up, zoom-pop, glitch-in, typewriter, neon-glow, card-flip-3d, kenburns) and canvas FX (particle-burst, matrix-rain, knowledge-graph with force-directed physics, neural-net visualization, confetti-cannon, firework). Canvas FX auto-initialize from `data-fx` attributes: `<canvas data-fx="particle-burst">` activates the effect without writing JavaScript.

The presenter mode is activated with the `S` key, opening a synced popup with four draggable magnetic cards: current slide, next slide, speaker script, and timer. This is the most complete presenter implementation among the agent skills.

The SKILL.md constrains the agent to always start from templates, use CSS tokens (never literal colors), and never invent layouts. A pre-authoring questionnaire ensures the agent asks about content, audience, and style preference before writing anything. The agent loads reference catalogs (`themes.md`, `layouts.md`, `animations.md`, `full-decks.md`) on demand to make informed choices about which building blocks to use.

Export uses a shell script wrapping headless Chrome to render slides to PNG images. (source: lewislulu/html-ppt-skill, retrieved 2026-05-20)

beautiful-html-templates: Tone-First Template Matching

beautiful-html-templates (zarazhangrui/beautiful-html-templates, 1.6k stars) is not a generation tool. It is a curated library of 34 self-contained HTML slide templates, each a complete deck with its own visual system. The innovation is the **tone-first matching** approach enabled by a machine-readable manifest.

The project's `index.json` file contains structured metadata for every template:

```
// index.json — excerpt showing one template's metadata
{
  "name": "Soft Editorial",
  "mood": "warm, literary, refined",
  "tone": "thoughtful, unhurried",
  "occasion": "keynote, essay, portfolio, book launch",
  "formality": "medium",
  "density": "low",
  "scheme": "light",
  "best_for": ["storytelling", "personal narratives", "literary events"],
  "avoid_for": ["hard data", "technical walkthroughs", "corporate updates"],
  "slide_count": 12,
  "fonts": ["Cormorant Garamond", "DM Sans"],
  "colors": {
    "primary": "#2C2825",
    "accent": "#B8A598",
    "background": "#FAF7F2"
  }
}
```

The agent workflow is prescribed in the project's `AGENTS.md`:

1. Ask the user about the occasion and desired mood.
2. Read `index.json` and pick three candidate templates based on tone matching.
3. Build title-slide HTML previews with the user's real content for all three candidates.
4. Open previews in the browser. User picks one.
5. Clone the chosen template folder and replace placeholder content with the user's content.

6. Design any missing slide layouts using the template's existing visual language.

The templates span seven style categories: Editorial/Literary (Soft Editorial, Emerald Editorial, Vellum, Cobalt Grid), Bold/Graphic (Neo-Grid Bold, Raw Grid, BlockFrame), Professional (Blue Professional, Signal, Monochrome), Warm/Crafted (Pin & Paper, Grove, Mat), Playful (Scatterbrain, Capsule, Daisy Days), Retro (Retro Windows, 8-Bit Orbit, Sakura Chroma), and Dark/Moody (Studio, Coral, Pink Script After Hours).

The critical design rule: templates are closed visual systems. Fonts, colors, decorations, spacing, and navigation are immutable. Only user content gets swapped. If a needed layout does not exist, the agent extends the system using its existing vocabulary rather than mixing templates. This prevents the visual inconsistency that comes from combining elements from different design systems. (source: zarazhangrui/beautiful-html-templates, retrieved 2026-05-20)

presentation-skills: Native PPTX with Quality Gates

presentation-skills (Sven-LI-sankyuu/presentation-skills, 104 stars) is the only project in this ecosystem that generates **native PowerPoint files** rather than HTML. It uses Python's `python-pptx` library to produce fully editable PPTX files with native Office charts, tables, and connector-backed diagrams.

The PPTX approach has a clear tradeoff. HTML slides look better in a browser, support animations that PowerPoint cannot render, and work without any software installation. But HTML slides do not work in corporate environments where PowerPoint is the standard delivery format. presentation-skills targets that corporate use case.

The skill implements a rigorous three-stage quality gate system:

Gate	Check	What It Catches
`package_preflight`	Validates PPTX package structure	Missing files, corrupt ZIP structure, broken relationships
`structure_precheck`	Checks text fit, overlaps, layout bounds	Text overflowing containers, overlapping elements, misaligned shapes
`render_review`	Visual inspection via PNG previews	Issues invisible at structure level: color clashes, visual density, readability

The generation pipeline is the most structured of any tool in this chapter:

1. **Brief.** Agent interviews the user about audience, context, template, and style preferences.
2. **Workspace initialization.** `init_deck_workspace.py` creates a structured directory with areas for brief, narrative, assets, build, validation, and final output.

3. **Template audit.** If the user provides a corporate PPTX template, `audit_pptx_template.py` extracts slide masters, layouts, font systems, and shared elements so the agent can inherit the existing design language.
4. **Narrative planning.** Agent writes `brief.md` (the deck contract) and `deck_narrative.md` (chapter structure with per-page tasks).
5. **Slide specification.** `derive_slide_specs_from_narrative.py` generates `slide_specs.yaml` with per-slide contracts defining archetype, required assets, and validation mode.
6. **Asset planning.** All charts, diagrams, images, and icons register as typed `asset_slots` with assigned rendering modules.
7. **Build.** `python-pptx` assembles native PPTX with editable shapes, text boxes, native charts, connector diagrams, and tables.
8. **Quality gates.** Three automated checks run before human review.
9. **Preview export.** Per-slide PNGs and a contact sheet for visual review.
10. **Connector validation.** Diagram connectors are verified to ensure flowcharts and architecture diagrams render correctly.

The output is a fully editable `.pptx` file with preview PNGs and validation reports. Any stakeholder can open it in PowerPoint and make changes. Charts stay editable (not rasterized images). Tables stay native (not screenshots). This matters in corporate environments where decks go through multiple rounds of manual editing by people who do not use coding agents.

The skill also produces demo videos: TTS narration (Alibaba Cloud ISI or macOS `say`), timed to slide transitions, with screen recording via Playwright and final compositing via ffmpeg. (source: Sven-LI-sankyuu/presentation-skills, retrieved 2026-05-20)

Choosing the Right Tool

The seven tools serve different needs. Here is a decision framework:

If you need...	Use...	Because...
A complete design platform with decks as one output type	Open Design	31 skills, 129 design systems, MCP server, handles all design artifacts
Visual style exploration before committing	frontend-slides	Generates 3 visual previews for comparison, "show don't tell" approach
Visually consistent output without manual design review	guizang-ppt-skill	Locked layouts and themes prevent visual inconsistency
Interactive data-driven slides with React components	open-slide	React ecosystem access, in-browser inspector with comments
The largest selection of themes and animations	html-ppt-skill	36 themes, 31 layouts, 27 CSS animations, 20 canvas FX
A specific visual style from a curated collection	beautiful-html-templates	34 templates with tone-first matching via index.json
Native PowerPoint files for corporate delivery	presentation-skills	python-pptx produces editable PPTX with native charts and tables

The Common Pattern: Constraint-Based Generation

All seven tools share one architectural principle: **constraint-based generation**. None of them give the agent a blank canvas and say "make a presentation." Instead, they provide:

- **Layout palettes** (pre-designed slide structures the agent picks from)
- **Theme systems** (CSS token files that ensure consistent colors and typography)
- **Content density limits** (max bullets per slide, max words per slide, max cards per grid)
- **Quality checklists** (P0/P1/P2 severity levels the agent validates against before delivery)
- **Pre-authoring questionnaires** (structured discovery that locks the brief before the agent writes any code)

This pattern mirrors what we saw with design token systems in Chapter 08. The agent's creative freedom is bounded by the design system. Within those bounds, the agent generates content, selects layouts, and arranges information. Outside those bounds, the agent is constrained to prevent visual inconsistency.

The constraint-based approach works because presentations have strong structural constraints already: fixed viewport, linear narrative, low content density per slide. The tools encode these constraints explicitly rather than hoping the agent will discover them through trial and error.

Huashu Design's Slide Capabilities

Huashu Design, covered in detail in Chapter 07, also generates slide presentations as one of its multi-mode outputs. Its slide mode uses the same Junior Designer workflow as its prototyping mode: the agent asks clarifying questions, proposes a visual direction, generates an early preview, and iterates based on feedback.

Huashu Design's slide output includes device frames (iPhone 15 Pro, Pixel, iPad Pro, MacBook, Chrome) for embedding app screenshots in presentation context, speaker notes support, and MP4/GIF export for animated slides. The animation engine runs at 60fps with frame interpolation for smooth motion graphics.

The relationship between Huashu Design and the dedicated slide tools is complementary. Huashu Design is a general-purpose HTML design tool that happens to support slides. The seven tools covered in this chapter are purpose-built for presentations. For a single deck, any of them will work. For a workflow that includes prototypes, slides, and other HTML artifacts, Huashu Design's unified approach may be more efficient.

Export and Delivery

All seven tools produce HTML output. But presenting from an HTML file has different requirements than presenting from a PowerPoint file. Here is how each tool handles delivery:

Format	Tools	Delivery Method
Self-contained HTML	All except presentation-skills	Open in any browser, fullscreen, keyboard navigation
PDF export	Open Design, frontend-slides, open-slide	Playwright headless Chromium at 1920×1080
PPTX export	Open Design, presentation-skills	Native Office format for corporate environments
PNG per slide	html-ppt-skill, presentation-skills	Headless Chrome rendering for static distribution
Live URL	frontend-slides, open-slide	Vercel/Cloudflare deployment for sharing

The HTML-First approach has practical advantages. The presenter does not need PowerPoint installed. The deck works on any device with a browser. Animations render at full frame rate. Speaker notes display in a separate window. And the single-file output can be version-controlled in git, tracked in diffs, and shared via URL.

The tradeoff is compatibility. In corporate environments where slide decks are emailed as `.pptx` attachments and edited by non-technical stakeholders, HTML output does not work. presentation-skills and Open Design's PPTX export fill this gap.

My take: HTML will become the default presentation format within two years. The combination of reliable fullscreen mode, smooth CSS animations, no-software-installation delivery, and git-friendly versioning makes HTML fundamentally better than PowerPoint for live presentations. The corporate `.pptx` requirement will persist as a legacy constraint, but it is a legacy constraint --- like sending Word documents instead of Google Docs links. The tools in this chapter are building the future format today.

Tip

Next: Chapter 16 shifts from generating design artifacts to building agent pipelines that produce those artifacts at scale. You will learn about the four major agent SDKs, the core agentic loop pattern, and concrete pipelines for image generation, video production, and design-to-code workflows.

16 Building Agent Pipelines for Design

Programmatic agents, SDKs, and automated design workflows

Interactive CLI sessions are how you explore. SDK-driven pipelines are how you produce at scale. This chapter covers the four major agent SDKs, the core agentic loop pattern, and concrete pipelines for image generation, video production, and design-to-code workflows. By the end, you will be able to architect multi-step agent pipelines that generate visual assets without human intervention.

Why Programmatic Agents?

The CLI is a conversation. You type, the agent responds, you course-correct in real time. This is ideal for exploration, prototyping, and one-off tasks. But conversations do not scale. When you need to generate 200 illustrations for a book, run a design pipeline on every pull request, or produce video assets from a YAML manifest, you need programmatic access.

Programmatic agents solve four problems the CLI cannot:

Batch processing. The CLI processes one task at a time. An SDK pipeline can fan out across dozens of tasks in parallel, collecting results into a structured output. A book with 40 chapters does not mean 40 separate CLI sessions.

CI/CD integration. Design generation belongs in your build pipeline, not in a terminal window. When a content author pushes a new chapter, the pipeline generates figures, validates dimensions, and produces print-ready assets automatically.

Repeated workflows. Any task you run more than twice should be scripted. The CLI forces you to re-articulate context every session. An SDK pipeline encodes the context once and reuses it indefinitely.

Multi-step creative pipelines. Real design work is not a single prompt. It is a chain: research, propose, iterate, generate, validate, compose. Each step feeds the next. The CLI handles this through manual copy-paste between turns. An SDK pipeline handles it through code.

The shift from CLI to SDK is the shift from *talking* to an agent to *programming* an agent. Both have their place. The mistake is using one when you need the other.

The Agent SDK Landscape

As of May 2026, four agent SDKs dominate the ecosystem. Each reflects a different philosophy about what an agent *is* and how it should be controlled.

Claude Agent SDK (`@anthropic-ai/claude-agent-sdk`) treats the agent as an async iterator. You call `query()`, and the SDK streams back events as the agent reasons, selects tools, and produces output. Subagents allow delegation to specialized workers. MCP integration gives access to external tools. Hooks let you intercept behavior at every stage. (source: Claude Agent SDK, retrieved 2026-05-19)

Google ADK (`google/adk-python`) takes a declarative approach. You define an `LlmAgent` with a list of tools and `sub_agents`. The runtime handles orchestration automatically. Native integrations with Imagen 4, Veo 3.1, and Gemini give it a built-in advantage for visual pipelines. (source: Google ADK, retrieved 2026-05-19)

OpenAI Agents SDK (`openai-agents`) centers on the `Runner` primitive. You define agents with specific capabilities, and the Runner manages their lifecycle. Handoffs allow agents to transfer control. `ImageGenerationTool` provides native image generation. The SDK is provider-agnostic via LiteLLM adapters. (source: OpenAI Agents SDK, retrieved 2026-05-19)

OpenCode SDK (`@opencode-ai/sdk`) is an HTTP-based API. You send structured requests and receive structured output. Custom tools are registered as endpoints. It is the simplest to integrate into existing systems because it speaks HTTP. (source: OpenCode, retrieved 2026-05-19)

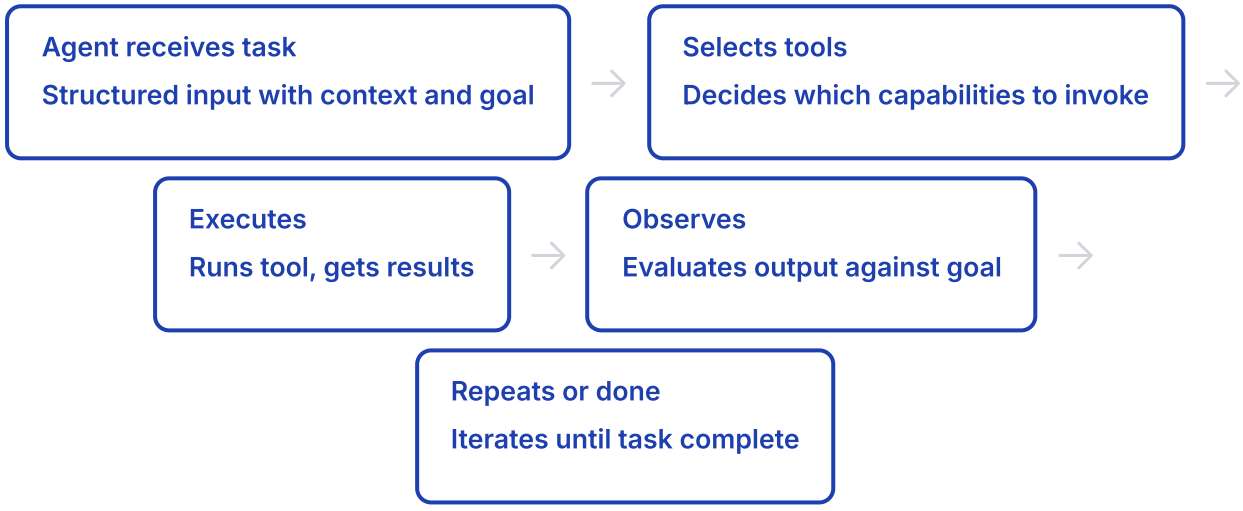
Figure 15.1: SDK architecture comparison showing the four major agent SDKs and their core primitives

Feature	Claude Agent SDK	Google ADK	OpenAI Agents	OpenCode SDK
Language	TypeScript, Python	Python	Python, TypeScript	Any (HTTP)
Stars	6.9k	19.7k	26.4k	—
Key Primitive	`query()` async iterator	`LlmAgent` declarative	`Runner` lifecycle	HTTP request/response
Subagents	Yes, with hooks	Yes, `sub_agents[]`	Yes, via handoffs	No (single agent)
MCP	Native	Via tools	Via tools	Native
Image Gen	Via tools/MCP	Native (Imagen 4)	`ImageGenerationTool`	Via custom tools
Model Support	Claude only	Gemini family	100+ via LiteLLM	75+ providers

My take: The SDK landscape is fragmented in exactly the way the framework landscape was fragmented in 2015. Four competing paradigms, none dominant, all evolving fast. The honest move is to learn the pattern, not the SDK. The agentic loop (agent → tool → iterate) is the same everywhere. The SDK is just syntax. Invest in understanding the loop; the specific SDK you use will probably change within a year.

The Core Pattern: Agent → Tool → Iterate

Every agent SDK implements the same fundamental loop. Understanding this loop is more important than understanding any specific API.



The agent is the *brain*. The tools are the *hands*. The loop is the *persistence*. Without tools, the agent can only reason. Without the loop, it can only act once. All three are required.

Here is the pattern in Claude Agent SDK:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Generate a hero illustration for a chapter about agent pipelines",
  options: {
    allowedTools: ["Read", "Write", "Bash", "WebSearch"],
    permissionMode: "acceptEdits",
    agents: {
      illustrator: {
        description: "Generates images for book chapters",
        prompt: "Create technical diagrams using...",
        tools: ["Bash", "Write"]
      }
    }
  }
})) {
  if (message.type === "assistant") {
    console.log("Agent reasoning:", message.text);
  }
}
```

The `query()` function starts the loop. `allowedTools` constrains what the agent can do. Subagents allow delegation to specialists. The async iterator gives you visibility into each step.

The same pattern in Google ADK:

```
from google.adk.agents import LlmAgent

design_agent = LlmAgent(
    name="design_producer",
    model="gemini-2.5-flash",
    instruction="Generate visual assets from chapter descriptions",
    tools=[generate_image_tool, file_manager_tool],
    sub_agents=[color_specialist, layout_validator],
)

result = await design_agent.run(
    "Create a diagram showing the agentic loop for Chapter 16"
)
```

ADK abstracts the loop away. You define the agent, its tools, its subagents, and the runtime handles iteration. The tradeoff is less control over individual steps, but faster setup for standard workflows.

Figure 15.2: Claude Agent SDK subagent delegation pattern with lead agent dispatching to specialized workers

Image Generation in Agent Pipelines

Image generation is the most common visual task in agent pipelines. Three approaches exist, each with different tradeoffs.

Approach 1: Direct API calls via Bash tool. The agent uses a shell execution tool to call generation APIs directly. This is the most flexible approach because it works with any API, but requires the agent to handle authentication and error parsing.

```
# Gemini image generation via agent Bash tool
curl -s https://generativelanguage.googleapis.com/v1beta/\
models/gemini-3.1-flash-image-preview:generateContent \
  -H "Content-Type: application/json" \
  -H "x-goog-api-key: $GEMINI_KEY" \
  -d '{
    "contents": [{"parts": [{"text": "Technical diagram..."}]}],
    "generationConfig": {"responseModalities": ["IMAGE"]}
  }' | jq -r '.candidates[0].content.parts[1].inlineData.data' \
  | base64 -d > output.png
```

Approach 2: Skill-based generation. Skills encapsulate generation logic into reusable units. The `garden-skills` repository (5.2k stars) provides 18 visual categories and 80+ prompt templates for image generation across Claude Code, Cursor, Codex, and OpenCode. (source: garden-skills, retrieved 2026-05-19)

Approach 3: MCP-exposed generation. MCP servers expose image generation as standardized tools. The `gemini-asset-generation-mcp` wraps Gemini image generation behind a tool interface. The `muapi MCP` from Generative-Media-Skills (3.3k stars) provides 19 tools covering generation, editing, upscaling, and face swap. (source: Generative-Media-Skills, retrieved 2026-05-19)

```
# MCP server configuration for image generation
{
  "mcpServers": {
    "gemini-assets": {
      "command": "python",
      "args": ["asset_server.py", "--root-dir", "./assets"],
      "env": { "GEMINI_API_KEY": "..." }
    }
  }
}
```

Approach	Tools	Best For	Example
Direct API (Bash)	`curl`, API keys	One-off scripts, maximum control	Custom pipeline with non-standard APIs
Skill-based	garden-skills templates	Repeated workflows, consistent style	Book illustrations with 18 visual categories
MCP-exposed	gemini-asset-generation-mcp, muapi	Multi-tool pipelines, standard interface	Design system with gen, edit, upscale

Figure 15.3: Google ADK short-movie-agents pipeline from story to video with session state

My take: Image generation is the gateway drug to agent pipelines. It is the first thing most people automate because the CLI workflow is tedious: write prompt, wait, evaluate, tweak, repeat. Once you script that loop, you start scripting everything else. Start with image generation. It teaches you the agentic loop in a domain where the output is immediately visible and the feedback loop is tight.

Video Production with Agents

Video production is where agent pipelines show their full power. A single video requires script, visuals, audio, editing, and composition. No single model does all of that well. Agent pipelines coordinate multiple specialized steps into a coherent workflow.

OpenMontage (3.8k stars) is the most complete agent video framework. It defines 12 pipelines and 52 tools, orchestrated through YAML manifests and skill-based composition. The agent *is* the orchestrator. A typical pipeline has eight stages: research, proposal, script, scene plan, assets, edit, compose, and publish. Each stage has its own skill file, quality gates, and validation criteria. (source: OpenMontage, retrieved 2026-05-19)

```
# OpenMontage animated-explainer pipeline (excerpt)
pipeline: animated-explainer
stages:
  - research:
      skill: pipelines/explainer/research-director
      produces: [research_brief]
  - proposal:
      skill: pipelines/explainer/proposal-director
      required_artifacts_in: [research_brief]
      human_approval_default: true
  - script:
      skill: pipelines/explainer/script-director
      produces: [script]
  - assets:
      skill: pipelines/explainer/asset-director
      required_tools: [tts_selector, image_selector]
  - compose:
      skill: pipelines/explainer/compose-director
      required_tools: [video_compose, audio_mixer]
```

Google ADK short-movie-agents takes a more integrated approach. A single pipeline moves from story to screenplay to storyboard (using Imagen 4) to final video (using Veo 3.1). Session state carries context between stages. (source: ADK Samples, retrieved 2026-05-19)

ArcReel (2.3k stars) specializes in long-form narrative video. It converts novels into video by extracting characters, generating storyboards, and producing scenes using Claude Agent SDK subagents. Each subagent receives only the context it needs, keeping the main agent's context window clean. (source: ArcReel, retrieved 2026-05-19)

Figure 15.4: OpenMontage eight-stage pipeline with YAML manifests, skills, and quality gates

Framework	Stars	Pipeline	Image Gen	Video Gen
OpenMontage	3.8k	12 pipelines, 52 tools, YAML manifests	Multiple (skill-based)	Multiple (skill-based)
ADK short-movie-agents	—	Story → Screenplay → Storyboard → Video	Imagen 4	Veo 3.1
ArcReel	2.3k	Novel → Characters → Storyboard → Video	Variable (subagent)	Variable (subagent)

Design-to-Code Pipelines

The most emerging category of agent pipeline is design-to-code: taking a visual input (screenshot, mockup, or description) and producing working frontend code. This is where the agentic loop shines because no single model call can handle the full translation.

The `agentic_image_to_tsx` pattern demonstrates a multi-agent approach using CrewAI with Gemini as the backbone model. The pipeline has four stages, each handled by a specialized agent:

```
# agentic_image_to_tsx pipeline
agents:
  - ui_analyst:
      role: "Analyze screenshot, extract design tokens"
      output: "design_tokens.json"

  - design_architect:
      role: "Convert tokens into structured design system"
      input: "design_tokens.json"
      output: "component_spec.json"

  - frontend_dev:
      role: "Generate React/TSX from component spec"
      input: "component_spec.json"
      output: "Component.tsx"

  - file_manager:
      role: "Validate, format, and write output files"
      input: "Component.tsx"
      output: "src/components/GeneratedComponent.tsx"
```

The UI Analyst examines the screenshot and extracts concrete values: colors as hex codes, spacing as pixel values, typography as font-size and weight, layout as flex or grid properties. The Design Architect takes those raw values and structures them into a component specification. The Frontend Dev writes the code. The File Manager validates and places the output. (source: agentic_image_to_tsx, retrieved 2026-05-19)

The key insight is that the pipeline is not linear. The Frontend Dev might request clarification from the Design Architect, which triggers re-analysis by the UI Analyst. The agentic loop applies within each agent and across agents.

Choosing Your Pipeline Architecture

The hardest decision is not which SDK to use. It is which architecture to adopt.

When	Use	Why
One-off exploration, prototyping	CLI	Fastest feedback loop, no setup cost, conversational course-correction
Repeated single-step tasks	Single-agent SDK	Encodes context once, runs reliably, easy to debug
Multi-step creative workflows	Multi-agent SDK	Each step gets specialized instructions, parallel execution possible
Tool integration across services	MCP	Standardized protocol, language-agnostic, composable
Reusable expertise across projects	Skills	Encapsulated knowledge, parameterized, shareable
CI/CD integration	SDK + MCP	Programmatic access + standardized tool interface

The pragmatic approach: start with the CLI. When you find yourself running the same prompt three times, wrap it in a script. When the script needs tool access, move to an SDK. When you need multiple coordinated steps, add subagents. When you need to integrate external services, add MCP. When you want to share the workflow, package it as a skill.

Do not start by architecting a 12-agent pipeline. Start with one agent doing one thing reliably. Complexity is earned, not assumed.

Figure 15.5: MCP as universal connector between agent CLIs and generation APIs

Repo	Stars	Focus
claude-agent-sdk	6.9k	Python/TS agent SDK with subagents and hooks
google/adk-python	19.7k	Python agent framework with native Gemini/Imagen/Veo
openai-agents	26.4k	Python agent SDK with handoffs and Runner
garden-skills	5.2k	80+ skill templates for image generation
Generative-Media-Skills	3.3k	MCP servers for generative media workflows
OpenMontage	3.8k	12 video pipelines with YAML orchestration
ArcReel	2.3k	Novel-to-video with Claude subagents

My take: The right architecture is the simplest one that solves your problem. I have seen teams build 8-agent pipelines for tasks a single prompt could handle. I have also seen teams manually running 50 CLI sessions for tasks a 3-agent pipeline could automate. The discipline is knowing when to escalate complexity. If you cannot explain why you need a multi-agent architecture in one sentence, you do not need one yet.

Tip

Next: Chapter 17 covers DESIGN.md — the emerging standard for portable design systems that agents can read. You will learn how Google Stitch defined the format, how the community built a 500+ design system library around it, and how to write your own DESIGN.md for consistent agent output.

17 DESIGN.md: Portable Design Systems for Agents

How a Markdown file became the standard for transferring visual design to AI

In April 2026, Google launched Stitch, an AI design tool that needed a way to encode entire design systems in a format agents could read. The answer was a plain Markdown file with YAML tokens. Within eight weeks, the community built 500+ design system files around it. This chapter covers the format, the ecosystem, and how to use DESIGN.md in your agentic design workflows.

What Is DESIGN.md?

Google Stitch (stitch.withgoogle.com) is an AI-powered design tool that generates UI from text descriptions. Stitch needed a way to encode an entire design system so AI coding agents could apply it consistently across thousands of generated components. The answer was unexpectedly simple: a plain-text Markdown file called `DESIGN.md`.

DESIGN.md combines two things that had never been formally united. The top of the file is **YAML front matter** — machine-readable tokens like `colors.primary` and `typography.scale` that agents interpolate directly into CSS variables or Tailwind themes. The bottom is a **Markdown body** — human-readable rationale explaining *why* those tokens exist, with do's, don'ts, and component specifications written in natural language.

This dual-format approach solved a real problem. Design tokens alone (JSON, Theo, Style Dictionary) carry values but not intent. Figma files carry visuals but not text-readable logic. Storybook carries components but requires a running application. DESIGN.md carries *everything an AI agent needs* in a single file that diffs cleanly in version control and weighs a few kilobytes.

Google open-sourced the specification at google-labs-code/design.md. The repository hit 14,300 stars within six weeks — a signal the community had been waiting for exactly this. (source: google-labs-code/design.md, retrieved 2026-05-19)

Figure 16.1: The three-layer convention showing how AGENTS.md, SKILL.md, and DESIGN.md divide AI context responsibilities

The Three-Layer Convention

A convention emerged quickly across the AI tooling ecosystem. Three files, three responsibilities, zero overlap.

AGENTS.md (or `CLAUDE.md` for Claude Code, `.cursor/rules` for Cursor) defines the *behavior layer*. It contains coding rules, project context, build commands, testing strategies — everything about *how* an agent should work. It says "use TypeScript strict mode" and "run tests before committing." It does not mention colors.

SKILL.md defines the *task layer*. It contains reusable procedures and domain expertise. The agentskills.io specification, supported by 30+ agent clients, formalizes this layer. A SKILL.md says "when building a form, validate on blur, show errors inline, disable submit until valid." It teaches *how to think about a task*. Chapter 03 covers skills in detail.

DESIGN.md defines the *appearance layer*. It contains colors, typography scales, spacing systems, border radii, component specifications — everything about *what the output should look like*. It says "primary buttons use #533afd at 48px height with 12px horizontal padding." It speaks in pixels and hex values.

File	Purpose	Who Reads It	What It Carries
AGENTS.md / CLAUDE.md	Behavior layer	All AI coding agents	Coding rules, project context, build commands, testing strategy
SKILL.md	Task layer	Skills-compatible agents (30+ clients)	Reusable procedures, domain expertise, workflow steps
DESIGN.md	Appearance layer	All agents doing UI work	Colors, typography, spacing, component specifications

The three layers complement each other. An agent reading all three knows *how to build* (AGENTS.md), *how to approach the task* (SKILL.md), and *what the result should look like* (DESIGN.md). Remove any one layer and the agent compensates by guessing — which is where inconsistent UI comes from.

> **My take:** The three-layer split is the correct decomposition. Behavior, task, and appearance are orthogonal concerns that change at different rates. Your design system might get a refresh every quarter. Your coding rules might change when you switch frameworks. Your task workflows might change when you adopt a new methodology. Keeping them separate means you update one without accidentally mutating the others. If your AGENTS.md has a "colors" section, you are doing it wrong.

Google's DESIGN.md Specification

The canonical DESIGN.md format follows a strict structure. The file opens with YAML front matter containing token definitions, then continues with Markdown sections explaining each token group.

Figure 16.2: DESIGN.md file structure with YAML front matter tokens and nine markdown body sections

The YAML front matter uses token references that agents interpolate into CSS custom properties:

```
---
name: stripe-inspired
version: "1.0"
colors:
  primary: "#533afd"
  ink: "#0d253d"
  surface: "#f6f9fc"
  border: "#e3e8ee"
  success: "#0d9467"
  error: "#e53e3e"
typography:
  display:
    family: "'Calibre', 'Inter', sans-serif"
    weight: 600
  body:
    family: "'Inter', sans-serif"
    weight: 400
  scale:
    - size: "48px"
      name: display-lg
    - size: "24px"
      name: heading
    - size: "16px"
      name: body
    - size: "14px"
      name: caption
rounded:
  button: "8px"
  card: "12px"
  input: "6px"
spacing:
  unit: "4px"
  scale: [4, 8, 12, 16, 24, 32, 48, 64]
components:
  - name: button-primary
    height: "48px"
    padding: "0 24px"
    bg: "{colors.primary}"
    color: "#ffffff"
    radius: "{rounded.button}"
    font: "{typography.body}"
    weight: 500
---
```

After the front matter, the Markdown body contains up to nine sections:

Section	What It Defines	Example
Colors	Semantic color palette with roles	primary, ink, surface, border, success, error
Typography	Font families, weights, and type scale	display-lg 48px, heading 24px, body 16px
Spacing	Base unit and spacing scale	4px base unit, scale from 4 to 64
Border Radius	Corner radii by component type	button 8px, card 12px, input 6px
Shadows	Elevation levels	sm: 0 1px 2px, md: 0 4px 12px
Components	UI element specifications	button-primary: 48px height, #533afd bg
Layout	Grid system, breakpoints, containers	max-width 1200px, 12-col grid
Motion	Animation timing and easing	150ms ease for hover, 300ms for expand
Dark Mode	Token overrides for dark theme	primary: #7c6aff, surface: #1a1a2e

Google ships a CLI tool for working with DESIGN.md files:

```
# Validate spec compliance + WCAG contrast ratios
npx @google/design.md lint DESIGN.md

# Token-level regression detection
npx @google/design.md diff a.md b.md

# Export to Tailwind theme, CSS variables, or Style Dictionary
npx @google/design.md export --format tailwind DESIGN.md
```

Command	What It Does
`lint DESIGN.md`	Validates spec compliance and WCAG contrast ratios across seven linting rules
`diff a.md b.md`	Token-level regression detection between two DESIGN.md files
`export --format tailwind`	Exports tokens to Tailwind theme configuration
`export --format css`	Generates a `:root` block of CSS custom properties
`init --template stripe`	Scaffolds a new DESIGN.md from a built-in template

The DESIGN.md Ecosystem

The community response was immediate and large. Within two months of Google's open-source release, a constellation of repositories and tools appeared.

Figure 16.3: The DESIGN.md ecosystem from Google Stitch specification to community collections and agent consumption

VoltAgent/awesome-design-md became the central hub. At 80,900 stars, it hosts 73 DESIGN.md files for real-world websites — Stripe, Vercel, Apple, Linear, Notion, Airbnb, Tesla, and dozens more. Each file is hand-audited for spec compliance and WCAG contrast. (source: VoltAgent/awesome-design-md, retrieved 2026-05-19)

bergside/design-md-chrome (2,000 stars) is a Chrome extension that extracts any website's visual design into a spec-compliant DESIGN.md file. It computes color palettes from computed styles, derives type scales from rendered font sizes, and infers spacing systems from layout measurements.

kwakseongjae/oh-my-design provides a CLI with 107 pre-built brand files plus an MCP server that serves DESIGN.md tokens to any MCP-compatible agent. It generates shim pointers — small cross-reference snippets you paste into CLAUDE.md, AGENTS.md, or `.cursor/rules`.

Repo	Stars	Focus
VoltAgent/awesome-design-md	80.9k	73 DESIGN.md files for real websites
DESIGNmd.ai	—	100+ community design systems, web editor
design-md-chrome	2k	Chrome extension: website to DESIGN.md
awesome-design-md-jp	3.4k	Japanese/CJK design system collection
awesome-ios-design-md	5.1k	200 iOS app DESIGN.md specifications
oh-my-design	1.8k	CLI with 107 brand files + MCP server

Combined, these repositories contain over 500 DESIGN.md files covering web, iOS, and CJK design systems. The velocity is unusual for a developer tool specification. DESIGN.md hit this level in eight weeks because it solved a problem everyone had — how do I tell an AI what my design system looks like? — with a format everyone already understood: Markdown plus YAML.

My take: The ecosystem velocity here is meaningful. When 500+ design system specifications appear in two months, the format has struck a nerve. Every team using AI coding agents was already struggling with the same problem. They were pasting Figma links that agents could not read, writing prose descriptions of colors that agents interpreted inconsistently, or hardcoding design tokens into prompts that broke on every refresh. DESIGN.md gave them a format that agents parse deterministically and humans read naturally.

How Agents Discover DESIGN.md

Most AI coding agents do *not* automatically discover DESIGN.md files. Unlike `package.json` or `tsconfig.json`, there is no universal convention for agents to look for DESIGN.md at the project root. You must create an explicit cross-reference from whichever instruction file your agent already reads.

Figure 16.4: Project file layout showing how agents discover DESIGN.md through cross-references in CLAUDE.md and AGENTS.md

The recommended project layout places DESIGN.md at the repository root:

```
project/
├── CLAUDE.md          # or AGENTS.md for Codex/OpenCode
├── DESIGN.md          # visual design system
├── .cursor/
│   └── rules/
│       └── design-system.mdc  # Cursor rule pointing to DESIGN.md
├── src/
└── package.json
```

For **Claude Code**, add a section to CLAUDE.md:

```
## Design System

This project uses DESIGN.md for visual specifications.
Read DESIGN.md before generating any UI code.
All color, typography, and spacing values must reference
the tokens defined in DESIGN.md YAML front matter.
Do not hardcode hex values or pixel measurements.
```

For **Cursor**, create `.cursor/rules/design-system.mdc`:

```
---
description: Design system tokens and component specs
globs: ["**/*.tsx", "**/*.css"]
alwaysApply: true
---

Read DESIGN.md at the project root. Follow its token
definitions for all UI code. Use CSS custom properties
mapped to DESIGN.md tokens, not hardcoded values.
```

The `oh-my-design` CLI goes further — it runs `oh-my-design link DESIGN.md` and writes shim pointers into CLAUDE.md, AGENTS.md, and `.cursor/rules/` simultaneously, so every agent in your team discovers the same design file.

From Brand Spec to Portable Design System

Chapter 07 covered Huashu Design and its `brand-spec.md` approach — a project-specific, dynamically generated design specification created through a five-step mandatory brand research process. `brand-spec.md` is produced fresh for each task via the Core Asset Protocol, injected as CSS variables, and validated through the "5-10-2-8" quality gate.

DESIGN.md takes a different approach. It is *pre-authored*, *portable*, and *shared across projects*. You write it once — or grab one from a community collection — and every agent that reads it produces visually consistent output. There is no brand research phase. The file is a static artifact that sits in version control.

Figure 16.5: Comparison of brand-spec.md workflow versus portable DESIGN.md approach

Aspect	brand-spec.md (Huashu)	DESIGN.md (Stitch)
When created	Per-task, dynamically generated	Pre-authored, version-controlled
Portability	Project-specific, tied to one brand	Portable, shared across projects
Brand research	5-step mandatory process	None — uses pre-existing design system
Format	Markdown with CSS variable injection	YAML front matter + Markdown body
Validation	"5-10-2-8" quality gate	7-rule linter (spec + WCAG contrast)
Best for	Bespoke brand work, original designs	Replicating known design systems
Ecosystem	Internal to huashu-design workflow	500+ community files, CLI tools, MCP

Neither approach is universally superior. `brand-spec.md` is better when you need *original* design work that captures a specific brand personality. DESIGN.md is better when you need *consistent replication* of a known design system — Stripe's checkout, Vercel's dashboard, Apple's product pages.

The Open Design project combined both approaches: Huashu Design's brand research workflow with DESIGN.md's portable format, resulting in 71+ swappable design systems that agents can switch between with a single file swap.

My take: Use both. Run the brand-spec.md workflow when you are creating something new. Then freeze the output into DESIGN.md format so every subsequent agent session gets the same tokens without repeating the research. brand-spec.md is the factory. DESIGN.md is the product. You run the factory once, then ship the product everywhere.

Writing Your First DESIGN.md

Start with the spec. End with the linter.

Step 1: Define your color palette with semantic roles. Name tokens by function, not appearance. `primary` not `purple`. `surface` not `light-gray`. `ink` not `dark-blue`. This matters because your primary color might change in a rebrand — your token name should not.

Step 2: Set your typography scale. Pick a base size (usually 16px) and define a consistent scale. Four to six sizes cover most interfaces: display, heading, subheading, body, caption. Specify family, weight, and line-height for each.

Step 3: Define your spacing system. Choose a base unit (4px or 8px) and derive a scale from it. Most design systems use a 4px base with values at 4, 8, 12, 16, 24, 32, 48, 64. Every gap, margin, and padding should map to a token on your scale.

Step 4: Specify your component vocabulary. Start with your three most-used components — typically button-primary, input-text, and card. Give each explicit dimensions, colors, typography, spacing, and border radius. Reference tokens, not hardcoded values.

Step 5: Add do's and don'ts. The Markdown body is where you write rules that cannot be expressed in YAML. "Buttons always use sentence case." "Never stack two primary buttons." These natural-language rules guide agents when the token definition is ambiguous.

Step 6: Run the linter.

```
$ npx @google/design.md lint DESIGN.md

✓ token-syntax: all tokens use valid dot-notation
✓ contrast-aa: all text/background pairs pass WCAG AA
  (primary on white: 7.2:1, ink on surface: 12.4:1)
✓ scale-consistency: spacing uses 4px base
✓ component-ref: all 3 components resolve to defined tokens
✗ dark-mode-parity: 2 tokens missing dark overrides
  → colors.surface, colors.border
✓ no-hardcoded: no hex values in Markdown body
✓ semantic-names: all tokens use semantic naming

1 issue found. Fix and re-run.
```

Fix the dark mode tokens and run again until all seven rules pass. A few practical tips: include dark mode tokens from the start — retrofitting them is harder than writing them alongside light tokens. Test contrast ratios with the linter before committing. Keep your components list minimal at first. Three components that agents follow precisely beat thirty that agents interpret loosely.

DESIGN.md is not a replacement for Figma or your design team. It is a translation layer — a way to encode visual decisions in a format that AI agents read deterministically. When your designer changes the primary color, you update one token in DESIGN.md and every agent session from that point forward uses the new value. No re-training, no prompt editing, no hunting through generated code for hardcoded hex values.

> **Tip**
>
> **Next:** This chapter concludes the main content of the book. Appendix A provides a comprehensive tool comparison matrix for quick reference. Appendix B contains the full MCP server reference. Appendix C offers a prompt library organized by design task.

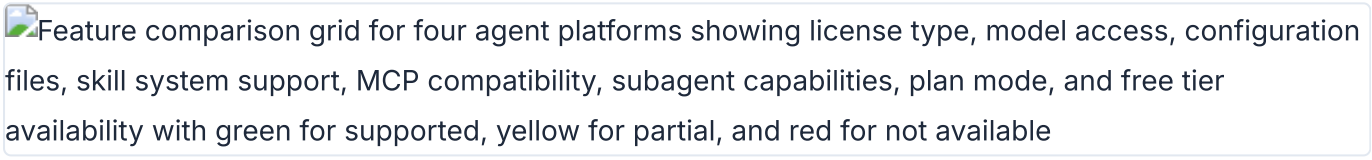
Appendix A Tool Comparison Matrix

Side-by-side feature comparisons for every tool covered in this book. One table per category, verified as of May 2026. Use these tables for quick lookups; the corresponding chapters provide the detailed walkthroughs.

Note: Pricing and feature details reflect publicly listed rates as of May 2026. Verify current pricing on each tool's website before making purchasing decisions.

Agent Platforms Compared

Four agent platforms dominate agentic design workflows as of May 2026. Each supports MCP, each has a different strength, and each takes a fundamentally different approach to skill management, permissions, and model access. The table below compares them across the dimensions that matter most for design work: how they connect to design tools, how they handle permissions, and what the real cost looks like when you use them daily.



Agent platform feature comparison across 15 dimensions with color-coded availability indicators

The comparison criteria here focus on practical concerns, not marketing claims. "Design skill ecosystem" measures the availability of pre-built skills and plugins for design tasks --- reading canvas files, generating JSX, exporting assets. "Model access" matters because some agents lock you into one provider while others let you bring any API key.

Feature	Claude Code	Codex CLI	OpenCode	Gemini CLI
Vendor	Anthropic	OpenAI	Open-source community	Google
License	Proprietary	Proprietary	MIT	Proprietary
Model access	Claude Opus / Sonnet	GPT-4o / o3	BYOK (any provider)	Gemini 2.5 Pro
MCP support	Full (plugin + manual)	Full (Streamable HTTP)	Full (JSON config)	Full
Skill system	SKILL.md, plugins	Plugins	Skills, agents	Skills
Permission model	Per-tool allowlist	Per-tool allowlist	Configurable	Per-tool
Design skill ecosystem	Largest (Paper plugin, custom skills)	Growing	Extensive (Open Design, Huashu)	Emerging
Price	$100-200/mo (Max plan)	$200/mo (Pro)	Free (BYOK API costs)	Free (BYOK)
Best for design	Full-stack design-to-code	Rapid prototyping	Multi-agent design teams	Multimodal design input
Last verified	May 2026	May 2026	May 2026	May 2026

Claude Code has the largest design skill ecosystem as of May 2026, driven by Anthropic's plugin marketplace and the Paper plugin specifically. Codex CLI is catching up fast --- OpenAI's plugin infrastructure is maturing, but design-specific skills remain thinner. OpenCode is the interesting outlier: it does not ship its own model, so you bring whatever API key you want. That flexibility comes with more configuration overhead, but it also means you can swap models mid-project without changing agents.

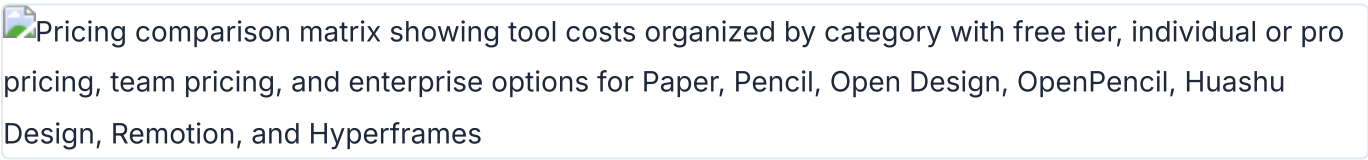
Gemini CLI's strength is multimodal input. If your design workflow starts with screenshots, photos, or whiteboard captures, Gemini's vision capabilities give it an edge for the initial design-to-prompt translation. Its skill ecosystem is the thinnest of the four, but Google is investing heavily here.

My take: For design work specifically, Claude Code is the most complete option today. The Paper plugin alone --- read designs, export JSX, write HTML back to canvas --- eliminates half the friction in a design-to-code workflow. But if you work across multiple model providers or need to run parallel agents with different models, OpenCode's BYOK architecture is the right call. In the author's experience, the $200/month price tag on Codex CLI is hard to justify for design work when the skill ecosystem is still growing.

Chapter 02 covers each platform in depth with installation and configuration details.

Design Canvas Tools Compared

The canvas is where design happens. These three tools --- Paper, Pencil, and OpenPencil --- take fundamentally different approaches to rendering, file formats, and agent integration. The comparison below covers the technical dimensions that determine how well each tool fits into an agentic workflow: canvas model (what you're actually manipulating), MCP integration (how agents talk to the canvas), code export quality (what you get out), and collaboration model (who else can work on the same file).



Pricing matrix for all book tools showing free, pro, and enterprise tier costs

I excluded Figma from this table because Figma's MCP integration is read-only as of May 2026. Figma remains the dominant design tool for collaborative work, but agents can only read from it, not write to it. The three tools below all support bidirectional agent communication.

Feature	Paper	Pencil	OpenPencil
Canvas model	Real HTML/CSS	WebGL (.pen JSON)	CanvasKit/Skia (.op JSON)
License	Commercial (freemium)	Commercial	MIT (open source)
Pricing	Free / $16/mo Pro	Free (IDE extension)	Free
Platform	Web + Desktop	Desktop + IDE	Web + Desktop + CLI
Collaboration	Real-time multiplayer	Local single-user	Planned
MCP server	Yes (22 tools)	Yes	Yes (built-in)
Code export	JSX + Tailwind via MCP	React + Tailwind via CLI	10 platforms via MCP
Color support	sRGB + P3 + OkLCH	sRGB hex	sRGB hex
Agent compatible	Claude Code, Codex, Cursor, Copilot, OpenCode	Claude Code, OpenCode	Claude Code, Codex, OpenCode, Gemini, Copilot, Kiro
File format	.paper (HTML-based)	.pen (JSON)	.op (JSON)
Git integration	Via agent	Native (.pen in repo)	Native (Git panel, merge)
Last verified	May 2026	May 2026	May 2026

Paper's canvas model is the most unusual: it renders actual HTML and CSS, not an abstraction layer. When an agent writes to Paper, it writes HTML. When it reads from Paper, it reads HTML. This makes the design-to-code pipeline almost trivially short --- the design artifact is already code. The trade-off is that Paper cannot render vector shapes or complex illustrations the way a tool with a dedicated rendering engine can.

Pencil's strength is IDE integration. The .pen file format is JSON that lives in your repository alongside your source code. Design changes show up in git diff. Design reviews happen in pull requests. For teams already doing code review, Pencil fits into existing workflows without introducing a separate review tool.

OpenPencil's Agent Teams feature --- where multiple agents work different regions of the canvas simultaneously --- is the most architecturally interesting approach in the space. Each agent claims a spatial region, works independently, and the canvas merges the results. Chapter 11 covers this pattern in depth.

My take: Paper wins on collaboration and color fidelity --- sRGB + P3 + OkLCH is, as of this writing, the most complete color support of any design tool in this comparison. Pencil wins on IDE integration and diffability. OpenPencil wins on openness and multi-agent architecture. I use all three depending on the task. For solo design-to-code work, Pencil lives inside my IDE and I never leave my terminal. For team projects with stakeholders who need to see and comment on designs, Paper is the strongest option in the author's experience. OpenPencil's Agent Teams feature is the most interesting architectural experiment in the space right now.

Design Skill Frameworks Compared

Skills teach agents how to design. Two frameworks dominate as of May 2026: Open Design and Huashu Design. They take opposite approaches. Open Design is breadth-first --- 31 independent skills, 129 built-in design systems, covering everything from landing pages to dashboards to pitch decks. Huashu Design is depth-first --- one comprehensive skill that embeds 20 design philosophies, a Junior Designer Workflow, and explicit anti-AI-slop rules into a single SKILL.md file.

The comparison below covers architecture, output capabilities, and the self-critique mechanisms each framework provides. One important clarification: both frameworks implement a "5-dimension critique," but they are different implementations. Open Design's critique is a separate skill in its 31-skill library, focused on scoring output against design system constraints. Huashu Design's critique is built into its single skill, focused on five dimensions of design quality (philosophical consistency, visual hierarchy, detail execution, functionality, and innovation) with explicit scoring and a fix list. Same concept, different execution.

Feature	Open Design	Huashu Design
License	Apache-2.0	MIT
GitHub stars	43.6k	14.1k
Skills count	31	1 (comprehensive)
Design systems	129 built-in	Brand Asset Protocol
Agent support	16 CLI agents auto-detected	Agent-agnostic (any markdown-skill agent)
Output formats	HTML, PDF, PPTX, ZIP, Markdown	HTML, MP4, GIF, PPTX, PDF
Visual directions	5 schools (curated palettes)	5 schools x 20 philosophies
Self-critique	5-dimension design system critique (separate skill)	5-dimension expert critique (built-in, scored with fix list)
Anti-AI-slop	Via design system constraints	Explicit anti-slop rules
Price	Free (open source)	Free (open source)
Architecture	Web + daemon + BYOK proxy	Skill file (SKILL.md)
Last verified	May 2026	May 2026

The architectural difference matters in practice. Open Design runs a local web daemon and proxies API calls through your own keys. It detects which agent CLI you have installed and adjusts its output accordingly. This is powerful but requires a running process. Huashu Design is a single SKILL.md file that you drop into your project. No daemon, no proxy, no running process. The agent reads the file and follows the instructions. Simpler setup, but less infrastructure for managing multi-step workflows.

For output formats, the key differentiator is video. Huashu Design can export HTML animations to MP4 and GIF with 60fps interpolation and background music mixing. Open Design focuses on static and interactive outputs --- HTML, PDF, PPTX. If your design workflow includes motion, Huashu Design has the edge. If you need 129 pre-built design systems to choose from, Open Design is, in the author's experience, unmatched in this category.

Chapters 06 and 07 cover Open Design and Huashu Design in depth, respectively.

Motion and Video Tools Compared

Programmatic video for agents comes down to two approaches as of May 2026: React components (Remotion) or HTML-native animation (Hyperframes). The licensing difference alone is worth understanding before you commit. Remotion uses a custom source-available license that requires a paid license for companies with certain revenue thresholds. Hyperframes uses Apache-2.0, which is OSI-approved open source with no revenue restrictions.

The table below compares the authoring model, rendering pipeline, agent integration, and output capabilities. The core architectural difference: Remotion compiles React components into video frames through a bundler. Hyperframes plays HTML + CSS + GSAP animations directly in a browser and captures the output. No build step, no bundler --- the same `index.html` you preview in a browser is the file that gets rendered to MP4.

Feature	Remotion	Hyperframes
License	Source-available (custom)	Apache-2.0 (OSI open source)
GitHub stars	47.1k	19k
Authoring format	React components (TSX)	HTML + CSS + GSAP
Build step	Required (bundler)	None (index.html plays as-is)
Animation model	Frame-accurate, seekable	Library-clock (GSAP, CSS, Lottie, Three.js)
Distributed rendering	Lambda (production-ready)	Single-machine today
Agent skills	Via SKILL.md	12 skills bundled
MCP server	No	No
Agent compatible	Claude Code, Cursor	Claude Code, Cursor, Codex, Gemini
Block catalog	Community templates	50+ ready-to-use blocks
Price	Free (companies need license)	Free
Last verified	May 2026	May 2026

Remotion's frame-accurate rendering model is a genuine technical advantage for certain use cases. Because each frame is a React render, you can seek to any frame, scrub the timeline, and get deterministic output. Hyperframes uses GSAP's clock, which means the animation timing depends on

the library's internal state. For most design-driven video --- product demos, explainer animations, social content --- this difference does not matter. For broadcast-quality or frame-precise compositing, Remotion has the edge.

Hyperframes' no-build-step approach is the better fit for agent workflows. An agent generates an HTML file with GSAP animations, and that file is immediately renderable. No `npm install`, no bundler configuration, no webpack or Vite setup. The 12 bundled skills cover common video patterns --- product demos, feature walkthroughs, social clips, title sequences. Remotion's community template ecosystem is larger, but the templates require more adaptation.

Distributed rendering is the area where Remotion is clearly ahead. Lambda-based rendering means you can parallelize frame computation across hundreds of instances. Hyperframes renders on a single machine today. For long-form video (10+ minutes) or batch rendering, this is a meaningful gap.

> **My take:** For agent-driven design work, Hyperframes is, in the author's experience, the more practical choice. The no-build-step model aligns with how agents work --- generate a file, render it, iterate. The Apache-2.0 license removes the compliance question entirely. Remotion is, as of this writing, the stronger tool for production video teams that need distributed rendering and frame-accurate control, but that describes a different workflow than what this book covers.

Chapter 09 covers both tools in depth, including setup, animation composition, and rendering.

MCP-Connected Tools Compared

These tools connect to agents via MCP servers. They extend agent capabilities into existing design platforms --- reading Figma files, generating patterns in MagicPattern, or collaborating on Miro boards. Unlike the canvas tools in section A.2, these are not design surfaces themselves. They are external platforms that expose a subset of their functionality to agents through MCP.

The comparison below covers access level, authentication method, and key limitations. Access level is the most important dimension: read-only MCP servers (like Figma's) let agents extract information but never modify the source file. Read-write servers (like MagicPattern's) let agents both read and create content. This distinction determines whether you can close the design loop entirely within the agent session or need to switch back to the GUI for write operations.

Feature	Figma MCP	Miro MCP	MagicPattern MCP
License	Proprietary	Proprietary	Proprietary
Access level	Read-only	Read + limited write	Read + write
Primary use	Read designs programmatically	AI-powered whiteboarding	Generative patterns
Agent compatible	Claude Code, Cursor, Codex	Claude Code, Cursor	Claude Code
Export formats	JSON, SVG, images	Board data, images	SVG, PNG, CSS backgrounds
Authentication	Figma API key	OAuth	API key
Price	Included with Figma	Included with Miro	Free / paid tiers
Key limitation	No write access, SVG fills as images	Limited write operations	Pattern types only
Last verified	May 2026	May 2026	May 2026

Figma's MCP server is the most widely used of the three, which makes sense given Figma's market position. But the read-only limitation is real. Agents can extract design tokens, read component structures, and download images, but they cannot create or modify frames. If your workflow requires writing back to Figma, you need the GUI or a third-party plugin.

Miro sits in the middle: agents can create sticky notes, add shapes to boards, and read board content. The write operations are limited to basic elements --- you cannot create complex diagrams or modify existing objects programmatically. For structured whiteboarding workflows where an agent captures meeting notes and organizes them on a board, Miro's MCP is sufficient.

MagicPattern has the broadest write access of the three. Agents can generate SVG patterns, export them as CSS backgrounds or PNG images, and iterate on pattern parameters. The limitation is scope: MagicPattern does one thing (generative patterns), and it does it well, but it will not help with layout, typography, or component design.

> **Tip**
>
> **Tip:** Chain these MCP servers for compound workflows. An agent can read a design system from Figma, extract the color palette, feed it to MagicPattern to generate background patterns, and write the results into a Paper canvas. Chapter 10 covers MCP chaining in detail.

Chapter 10 covers MCP integrations in depth, including configuration, chaining, and troubleshooting. Appendix B provides a complete reference for every MCP tool available in each server.

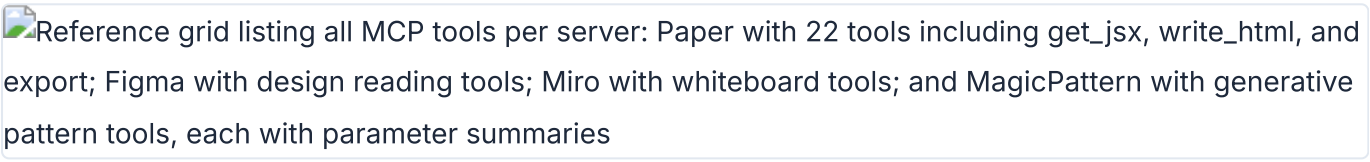
Appendix B MCP Server Reference

Every MCP server covered in this book, every tool it exposes, and the configuration snippets to connect it to your agent. Verified as of May 2026.

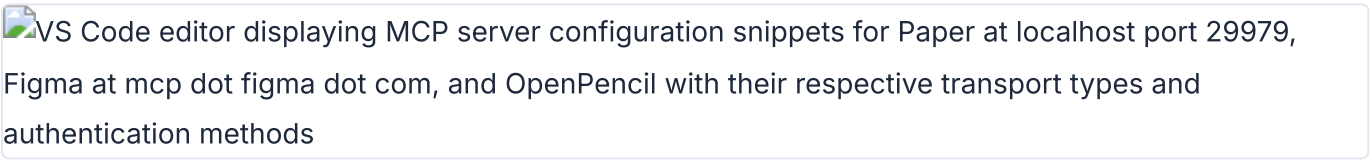
> **Note:** Configuration snippets are illustrative examples. Refer to each tool's official documentation for current setup instructions.

Paper MCP Server Tools

Paper's MCP server runs locally when Paper Desktop is open. It listens on `http://127.0.0.1:29979/mcp` and exposes 22 tools split into read, write, and export categories. This is the most polished MCP server in the design tool ecosystem as of May 2026.



MCP server tool reference showing Paper, Figma, Miro, and MagicPattern tool inventories



MCP server configuration reference showing Paper, Figma, and OpenPencil connection settings

Read Tools

Tool	Parameters	Returns	Notes
`get_basic_info`	---	File name, page name, node count, artboard list	Overview of open file
`get_selection`	---	Selected node IDs, names, types, size	Current selection context
`get_node_info`	nodeId	Size, visibility, lock, parent, children, text	Detailed node data
`get_children`	nodeId	Direct children: IDs, names, types	One level deep
`get_tree_summary`	nodeId, depth?	Compact text summary of subtree	Optional depth limit
`get_screenshot`	nodeId, scale?	Base64 image (1x or 2x)	Visual capture
`get_jsx`	nodeId, format	JSX string (tailwind or inline-styles)	Key tool for design-to-code
`get_computed_styles`	nodeIds[]	Computed CSS styles (batch)	Production-ready values
`get_fill_image`	nodeId	Base64 JPEG of image fill	Extract embedded images
`get_font_family_info`	fontFamily	Availability, weights, styles	Check font access
`get_guide`	topic	Guided workflow steps	e.g., "figma-import"

Write Tools

Tool	Parameters	Returns	Notes
`create_artboard`	name?, styles?	New artboard ID	Create new canvas area
`write_html`	html, mode, parentId?	Updated node IDs	Parse HTML, add or replace nodes
`set_text_content`	nodeIds[], content	Updated count	Batch text update
`rename_nodes`	nodeIds[], names	Updated count	Batch rename
`duplicate_nodes`	nodeIds[]	New IDs, descendant ID map	Deep clone
`move_nodes`	nodeIds[], parentId?, index?	Updated positions	Reparent and reorder
`update_styles`	nodeIds[], styles	Updated count	Batch CSS updates
`delete_nodes`	nodeIds[]	Deleted count	Cascade delete descendants
`finish_working_on_nodes`	artboardIds[]	---	Clear working indicator

Export Tool

Tool	Parameters	Returns	Notes
`export`	nodes[], format?, scale?	Exported files	PNG, JPG, SVG, MP4; overrides like 2x, 512w, 1080p

My take: The `get_jsx` and `write_html` tools make Paper's MCP server the tightest design-to-code loop available. No other tool lets an agent read JSX from a design and write HTML back in the same session. The Desktop app requirement is a real limitation for Linux users and CI environments, but for daily design work it's the server I reach for first.

Pencil MCP Server Tools

Pencil exposes design tools through its IDE extension and CLI. The MCP server provides access to .pen files, design variables, component definitions, and code generation.

Capability	Details
File operations	Read and write .pen files
Design variables	Access variables and convert to CSS custom properties
Components	Read component definitions with slots and overrides
Code export	Generate React + Tailwind from .pen designs
Node manipulation	Insert, copy, update, replace, move, delete nodes
Visual capture	Screenshots and export to PNG, JPEG, WEBP, PDF
Search	Search nodes by pattern or ID
Variables and themes	Get and set design variables and themes

Configure Pencil's MCP server in OpenCode:

```
// opencode.json
{
  "mcp": {
    "pencil": {
      "type": "remote",
      "url": "http://127.0.0.1:/mcp",
      "enabled": true
    }
  }
}
```

OpenPencil MCP Server Tools

OpenPencil's built-in MCP server ships as the `pen-mcp` package. It provides a layered design workflow and code generation pipeline.

Document and Design Tools

Tool Category	Capabilities
Document operations	Read, create, modify .op files; multi-page support
Node manipulation	Insert, update, move, delete nodes
Style guides	`get_style_guide_tags`, `get_style_guide` for visual styles
Design workflow	`design_skeleton`, `design_content`, `design_refine`
Prompt retrieval	Segmented: load schema, layout, roles, icons, planning as needed

Codegen Tools

```
// Codegen pipeline
codegen_plan        → plan the code generation
codegen_submit_chunk → submit code chunks incrementally
codegen_assemble    → assemble final output
codegen_clean       → clean temporary files
```

Install OpenPencil's MCP server from the OpenPencil UI: Settings > MCP > Install for [Agent]. One-click setup.

> **Warning**
>
> **Warning:** OpenPencil's codegen pipeline is incremental. If the agent crashes mid-generation, partial chunks may remain. Run `codegen_clean` to reset state before retrying.

Figma MCP Server Tools

Figma's official MCP server provides read-only access to design files. As of May 2026, write access is not available --- agents can read but cannot modify Figma files.

Capability	Details
Read design files	Node tree, component definitions, styles
Selection context	Get current selection
Variables	Read variables (must have element selected that uses the variable)
Styles	Text styles, color variables, effect styles

Known limitations: SVG fills returned as images, component instance color overrides not reliably returned, spacer elements ignored, and code-connected components not reliably converted.

Configure Figma's MCP server in Claude Desktop:

```
// Claude Desktop config
{
  "mcpServers": {
    "figma": {
      "command": "npx",
      "args": [
        "-y",
        "figma-developer-mcp",
        "--figma-api-key=$FIGMA_API_KEY",
        "--stdio"
      ]
    }
  }
}
```

> **Warning**
>
> **Security:** Never commit API keys to version control. Use environment variables or a secrets manager.

Miro MCP Server Tools

Miro's MCP server enables AI-powered whiteboarding. It provides read and limited write access to Miro boards.

Capability	Details
Read board content	Sticky notes, shapes, text, connectors
Write board elements	Create and modify board elements (limited)
Board metadata	Access board structure and metadata

Miro uses OAuth-based authentication. Available through Miro's developer platform. Compatible with Claude Code and Cursor.

MagicPattern MCP Tools

MagicPattern generates design patterns --- dots, gradients, waves, blobs --- via MCP. Agents can create and customize patterns, then export them as SVG, PNG, or CSS backgrounds.

Capability	Details
Pattern generation	Dots, gradients, waves, blobs, and more
Customization	Colors, spacing, density parameters
Export formats	SVG, PNG, CSS backgrounds

API key-based authentication. Compatible with Claude Code. Free and paid tiers available.

My take: Paper's 22-tool MCP server is the gold standard. Every other server in this appendix is either read-only (Figma), limited-write (Miro), or single-purpose (MagicPattern). The design tools that win the agent era will be the ones that give agents full read-write access with well-documented tools. If you're evaluating a new design tool for agent workflows, MCP tool count and write access are the two metrics that matter most.

Configuration Reference by Agent Platform

The same MCP server needs different configuration in each agent. Here are the snippets for Paper's MCP server --- the pattern is the same for other servers, just change the URL.

Claude Code (plugin, recommended):

```
# Add marketplace
/plugin marketplace add paper-design/agent-plugins
# Install plugin
/plugin install paper-desktop@paper
```

Claude Code (manual):

```
claude mcp add paper --transport http http://127.0.0.1:29979/mcp --scope user
```

Codex CLI:

```
Settings > MCP Servers > Streamable HTTP
Name: paper
URL: http://127.0.0.1:29979/mcp
```

OpenCode:

```
// opencode.json
{
  "mcp": {
    "paper": {
      "type": "remote",
      "url": "http://127.0.0.1:29979/mcp",
      "enabled": true
    }
  }
}
```

GitHub Copilot:

```
// .vscode/mcp.json
{
  "servers": {
    "paper": {
      "type": "http",
      "url": "http://127.0.0.1:29979/mcp"
    }
  }
}
```

Cursor:

```
/add-plugin paper-desktop
# Or search "paper-desktop" in Cursor Marketplace
```

Antigravity:

```
// mcp_config.json
{
  "mcpServers": {
    "paper": {
      "serverUrl": "http://127.0.0.1:29979/mcp"
    }
  }
}
```

Claude Desktop (manual):

```
// Settings > Developer > Edit Config
{
  "mcpServers": {
    "paper": {
      "command": "npx",
      "args": ["mcp-remote", "http://127.0.0.1:29979/mcp"]
    }
  }
}
```

Troubleshooting MCP Connections

MCP connections break. It is the most common source of friction in agent-driven design workflows. Here are the issues I see most often.

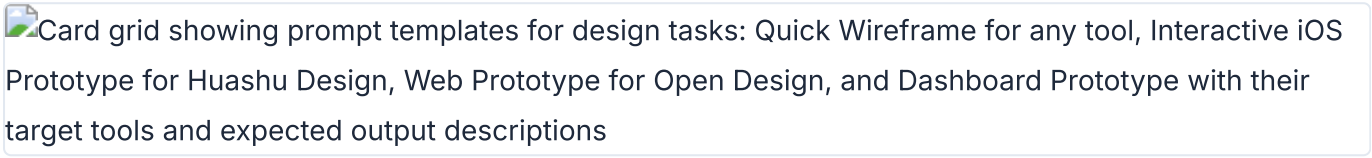
Symptom	Cause	Fix
Agent cannot find MCP tools	Host tool not running or MCP not connected	Restart the agent session
MCP shows connected but tools fail	Stale connection after long session	Restart both the agent and the host tool
Agent hallucinates tool parameters	LLM context drift over long sessions	Restart the agent session
Rate limit errors after upgrade	Limits not reset after plan change	Update Paper Desktop (About > Check for updates) then restart
Cannot reach 127.0.0.1 on Windows WSL	WSL networking isolation	Enable mirrored mode networking in WSL config

Appendix C Prompt Library for Design Tasks

Battle-tested prompts for common agentic design tasks. Copy them into your agent session, swap the bracketed values, and ship. Verified as of May 2026.

Prototyping Prompts

Quick wireframes, interactive prototypes, and dashboard layouts. These prompts work across tools --- adjust the tool reference for your setup.



Prompt template gallery organized by task category with target tools and expected outputs

Prompt	Target Tool(s)	Expected Output
Quick Wireframe	Any	Gray-block layout with information architecture
Interactive App Prototype	Huashu Design	Clickable iOS prototype with real images
Web Prototype	Open Design	Styled web page with hero, features, pricing
Mobile Onboarding Flow	Open Design	3-screen dark mode onboarding
Dashboard Prototype	Open Design	Admin dashboard with sidebar, KPI cards, table, chart

Quick wireframe --- works with any tool:

```
Create a wireframe for a [type] app with [key screens].
Use gray blocks for content areas. No styling needed ---
just layout and structure. Show me the information architecture.
```

Interactive iOS prototype --- Huashu Design:

```
Build an iOS prototype for a [app description] ---
[4 screens], actually clickable. Use real images from
Wikimedia/Unsplash. Include navigation between screens.
```

Web prototype with design system --- Open Design:

```
Using the [design-system-name] design system, create a
web prototype for [product description]. Target audience:
[audience]. Tone: [tone]. Include hero, features, and
pricing sections.
```

Mobile onboarding flow --- Open Design:

```
Create a 3-screen mobile onboarding flow for [app name].
Screen 1: Splash with brand mark
Screen 2: Value proposition (3 key benefits)
Screen 3: Sign-in form
Use [design-system] style. Dark mode.
```

Dashboard prototype --- Open Design:

```
Build an admin dashboard for [use case]. Include:
- Sidebar navigation (6 items)
- KPI cards (4 metrics)
- Data table with sorting
- Chart section
Use the [design-system] design system.
```

Visual Design Prompts

Landing pages, brand identities, and design direction exploration. These prompts combine design tools with agent capabilities.

Prompt	Target Tool(s)	Expected Output
Landing Page Build	Paper + Claude Code	React + Tailwind website from Paper design
Design Direction Advisor	Huashu Design	3 distinct visual directions to choose from
Brand-Consistent Design	Huashu Design	Design using real brand assets
Token Sync	Paper + Figma MCP	Design tokens migrated from Figma to Paper

Landing page from Paper design:

```
I'd like you to build a website in this folder using the
landing page I have selected in Paper. Use React and
Tailwind for the styling. Start with the hero section,
then build out the remaining sections one at a time.
```

Design direction exploration --- Huashu Design:

```
I need a visual identity for [project description].
Give me 3 style directions to pick from. Make them
visually distinct from each other.
```

Brand-consistent design --- Huashu Design:

```
Design a [deliverable] for [brand name]. Follow the
Core Asset Protocol: find their official brand assets
(logo, colors, fonts, product shots) before starting.
Their website is [URL].
```

Token sync between Figma and Paper:

```
Create a design system in Paper that matches the design
system in the currently open Figma file. Include colors,
text styles, and spacing tokens. Read from Figma, write
to Paper.
```

My take: The most effective prompts are specific about output format and constraints. "Build me a website" produces garbage. "Build a React + Tailwind landing page from the Paper selection, starting with the hero section" produces something usable in one shot. Specificity is the lever that turns an agent from a toy into a tool.

Motion Design Prompts

Product videos, social content, and data animations. These prompts target Remotion and Hyperframes --- two different approaches to programmatic video.

Product launch video --- Huashu Design:

```
Turn this product description into a 60-second launch
animation. Export MP4 and GIF. Use the Stage + Sprite
model. Include: title reveal, feature showcase, CTA.
```

Explainer video --- Hyperframes:

```
Using /hyperframes, create a 10-second product intro
with a fade-in title, a background video, and background
music. Resolution: 1920x1080.
```

Social media video --- Hyperframes:

```
Make a 9:16 TikTok-style hook video about [topic] using
/hyperframes, with bouncy captions synced to a TTS
narration. Include a lower third at 0:03 with my name
and title.
```

Data animation --- Hyperframes:

```
Turn this CSV data into an animated bar chart race using
/hyperframes. 15-second duration. Dark theme. Include
axis labels and a title card.
```

Iterate on video --- Hyperframes:

```
Make the title 2x bigger, swap to dark mode, and add
a fade-out at the end. Add a lower third at 0:03 with
"Product Launch 2026".
```

Motion design hero --- Open Design:

```
Create a motion-design hero with looping CSS animations.
Include rotating type ring, animated globe, and ticking
timer. Single-frame, hand-off ready for Hyperframes.
```

Design System Prompts

Creating, syncing, and maintaining design systems with agents. These prompts handle the tedious parts of design system management.

Create a design system from an existing website:

```
Create a DESIGN.md for [product/company name] based on
their existing website at [URL]. Include: color palette
(hex values), typography (font families, sizes, weights),
spacing scale, component patterns, motion guidelines,
anti-patterns to avoid.
```

Sync tokens between Figma and Paper:

```
Read the design tokens from the Figma file (colors,
text styles, spacing) and create matching design tokens
in the Paper file. Use the same variable names. Create
a sticker sheet showing all tokens.
```

Generate component library --- OpenPencil:

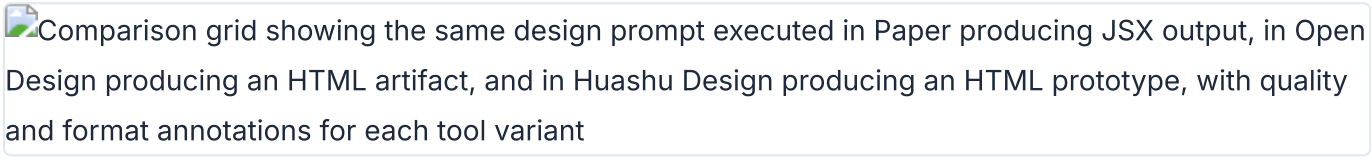
```
Design a component library for [design system] with these
components: Button (3 variants), Input, Card, Modal,
Navigation, Badge. Use design variables for colors and
spacing. Export as React + Tailwind.
```

Maintain token parity --- Pencil:

```
Read the design variables from the .pen file and compare
them with the CSS custom properties in src/styles/tokens.css.
List any discrepancies and fix them to match the .pen source
of truth.
```

Production UI Prompts

Design artifacts to production code. These prompts enforce accessibility, performance, and responsive design standards.



Same prompts run through Paper, Open Design, and Huashu Design showing output format differences

Prompt	Target Tool(s)	Expected Output
Design to Production Code	Paper + Claude Code	Accessible React + Tailwind component
Responsive Breakpoints	Paper	Mobile-first responsive layout
Accessibility Audit	Any	Fixed a11y issues with change summary
Performance Optimization	Any	Core Web Vitals improvements

Design to production code:

```
Read the [section] frame I have selected in Paper and
build it as a React component using Tailwind. Follow
these rules:
1. Use semantic HTML elements (nav, main, section, article)
2. All images must have descriptive alt text
3. All interactive elements must be keyboard accessible
4. Use the design tokens from our DESIGN.md
5. Ensure color contrast ratio ≥ 4.5:1 for body text
Commit the changes when done.
```

Responsive breakpoints from Paper frames:

```
Add responsive breakpoints to the website based on the
frames I have selected in Paper. Each frame is a different
breakpoint:
- 375px frame = mobile (default)
- 768px frame = tablet (md:)
- 1440px frame = desktop (xl:)
Use Tailwind responsive classes. Test each breakpoint.
```

Accessibility audit:

```
Review the generated code for accessibility issues:
1. Missing alt text on images
2. Missing labels on form inputs
3. Insufficient color contrast (< 4.5:1 for text)
4. Missing ARIA attributes on custom widgets
5. Keyboard navigation gaps
6. Missing focus indicators
Fix all issues found. Show me what you changed.
```

Performance optimization:

```
Optimize the generated code for Core Web Vitals:
1. Add width/height to all images to prevent CLS
2. Lazy-load below-fold images with loading="lazy"
3. Preload critical fonts
4. Minimize CSS by removing unused Tailwind classes
5. Add proper srcset for responsive images
Show me the Lighthouse score improvements.
```

My take: The accessibility audit prompt is the single most valuable prompt in this appendix. Agents are better than most developers at catching a11y issues systematically --- they don't get tired, they don't skip steps, and they check every element. Make this prompt part of every design-to-code workflow.

Multi-Tool Workflow Prompts

Chains that span multiple tools. These prompts coordinate across MCP servers and agent sessions to accomplish tasks no single tool handles alone.

Full website build --- Paper to deployment:

```
Phase 1 - Design:
  "I have a landing page design in Paper. Read the hero
  section using the Paper MCP and build it with React +
  Tailwind."

Phase 2 - Git checkpoint:
  "Add git to this folder and commit the changes."

Phase 3 - Responsive:
  "Add responsive breakpoints based on the frames I have
  selected in Paper. Each frame is a different breakpoint."

Phase 4 - Deploy:
  "Deploy this to [Vercel/Netlify]. Set up a production
  build."
```

Figma to Paper to code migration:

```
Step 1: Read the design tokens from the Figma file and
create them in Paper.
Step 2: Copy each section from Figma to Paper, maintaining
layout fidelity.
Step 3: Verify the Paper design matches the Figma original.
Step 4: Generate React + Tailwind from the Paper design.
```

Pitch deck to video pipeline --- Open Design to Hyperframes:

```
Step 1: "Using the magazine-deck skill, create a 10-slide
pitch deck for [startup]. Use [design-system] visual style."

Step 2: "Take slide 1 (the hero) and turn it into a 5-second
intro animation using /hyperframes. Add a cinematic background
and title reveal."

Step 3: "Render all animations and assemble into a final
60-second pitch video with transitions between each slide."
```

Design system to component library --- Pencil + OpenPencil:

```
Step 1: "Design the Button, Input, and Card components in
the .pen file. Define color, spacing, and typography variables."

Step 2: "Export the components as React + Tailwind. Map
design variables to CSS custom properties."

Step 3: "Create a Storybook story for each component showing
all variants. Include accessibility testing in each story."
```

Each prompt in this appendix is designed to be copied directly into an agent session. Swap bracketed values for your project specifics. The prompts that work best are specific about output format, tool selection, and quality constraints. Vague prompts produce vague output.